

multipink

June 6, 2026

compile_options	-lang cpp -fpga-mem-th 4 -ct 1 -es 1 -mcd 16 -mdd 1024 -mdy 33 -singl
filename	multipink.dsp
filters.lib/fir:author	Julius O. Smith III
filters.lib/fir:copyright	Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.stanford.e
filters.lib/fir:license	MIT-style STK-4.3 license
filters.lib/iir:author	Julius O. Smith III
filters.lib/iir:copyright	Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.stanford.e
filters.lib/iir:license	MIT-style STK-4.3 license
filters.lib/lowpass0_highpass1	MIT-style STK-4.3 license, Copyright (C) 2003-2019 by Julius O. Smit
filters.lib/name	Faust Filters Library
filters.lib/version	1.7.1
maths.lib/author	GRAME
maths.lib/copyright	GRAME
maths.lib/license	LGPL with exception
maths.lib/name	Faust Math Library
maths.lib/version	2.9.0
name	multipink
noises.lib/name	Faust Noise Generator Library
noises.lib/version	1.5.0
seam.lib/author	Giuseppe Silvi, Davide Tedesco
seam.lib/license	CC3
seam.lib/name	Faust SEAM main lib
seam.lib/version	0.2
seam.noises.lib/author	Giuseppe Silvi
seam.noises.lib/license	CC3
seam.noises.lib/name	SEAM noises - Library
seam.noises.lib/version	0.2

This document provides a mathematical description of the Faust program text stored in the `multipink.dsp` file. See the notice in Section 3 (page 3) for details.

1 Mathematical definition of process

The *multipink* program evaluates the signal transformer denoted by **process**, which is mathematically defined as follows:

1. Output signals y_i for $i \in [1, 4]$ such that

$$\begin{aligned} y_1(t) &= 0.5 \cdot (0.049922035 \cdot r_1(t) + 0.050612699 \cdot r_1(t-2) \\ &\quad - 0.095993537 \cdot r_1(t-1) + 0.004408786 \cdot r_1(t-3)) \\ y_2(t) &= 0.5 \cdot (0.049922035 \cdot r_3(t) + 0.050612699 \cdot r_3(t-2) \\ &\quad - 0.095993537 \cdot r_3(t-1) + 0.004408786 \cdot r_3(t-3)) \\ y_3(t) &= 0.5 \cdot (0.049922035 \cdot r_5(t) + 0.050612699 \cdot r_5(t-2) \\ &\quad - 0.095993537 \cdot r_5(t-1) + 0.004408786 \cdot r_5(t-3)) \\ y_4(t) &= 0.5 \cdot (0.049922035 \cdot r_7(t) + 0.050612699 \cdot r_7(t-2) \\ &\quad - 0.095993537 \cdot r_7(t-1) + 0.004408786 \cdot r_7(t-3)) \end{aligned}$$

2. Input signal (none)

3. Intermediate signals s_i for $i \in [1, 3]$ and r_i for $i \in [1, 8]$ such that

$$\begin{aligned} s_1(t) &= 1103515245 \odot (r_2(t-1) \oplus 12345) \\ s_2(t) &= 1103515245 \odot (s_1(t) \oplus 12345) \\ s_3(t) &= 1103515245 \odot (s_2(t) \oplus 12345) \end{aligned}$$

$$\begin{aligned} r_2(t) &= 1103515245 \odot (s_3(t) \oplus 12345) \\ r_1(t) &= 0.5221894 \cdot r_1(t-3) + 4.6566128752458 \cdot 10^{-10} \cdot r_2(t) \\ &\quad + 2.494956002 \cdot r_1(t-1) - 2.017265875 \cdot r_1(t-2) \\ r_4(t) &= s_3(t) \\ r_3(t) &= 0.5221894 \cdot r_3(t-3) + 4.6566128752458 \cdot 10^{-10} \cdot r_4(t) \\ &\quad + 2.494956002 \cdot r_3(t-1) - 2.017265875 \cdot r_3(t-2) \\ r_6(t) &= s_2(t) \\ r_5(t) &= 0.5221894 \cdot r_5(t-3) + 4.6566128752458 \cdot 10^{-10} \cdot r_6(t) \\ &\quad + 2.494956002 \cdot r_5(t-1) - 2.017265875 \cdot r_5(t-2) \\ r_8(t) &= s_1(t) \\ r_7(t) &= 0.5221894 \cdot r_7(t-3) + 4.6566128752458 \cdot 10^{-10} \cdot r_8(t) \\ &\quad + 2.494956002 \cdot r_7(t-1) - 2.017265875 \cdot r_7(t-2) \end{aligned}$$

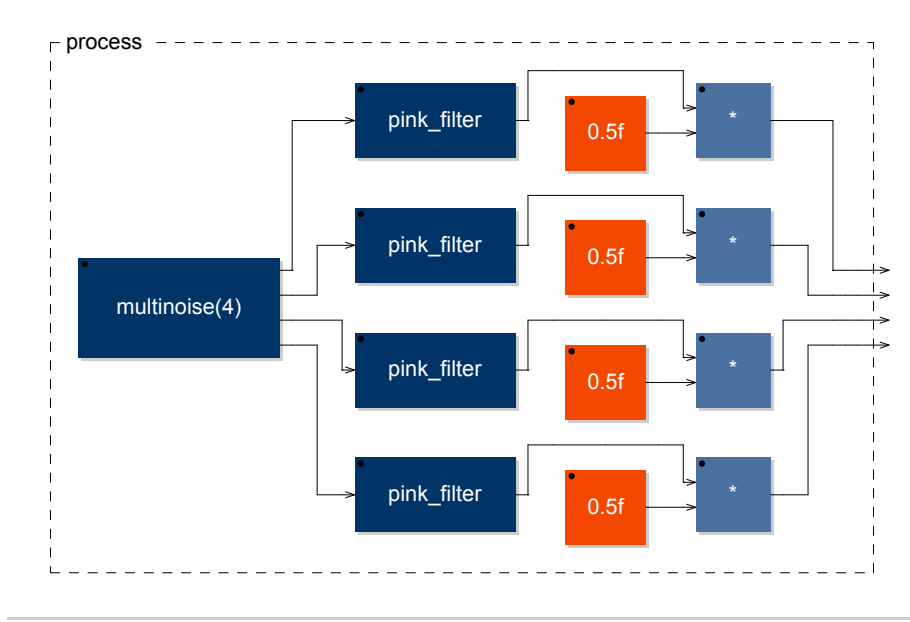


Figure 1: Block diagram of `process`

2 Block diagram of `process`

The block diagram of `process` is shown on Figure 1 (page 3).

3 Notice

- This document was generated using Faust version 2.85.5 on June 06, 2026.
- The value of a Faust program is the result of applying the signal transformer denoted by the expression to which the `process` identifier is bound to input signals, running at the f_s sampling frequency.
- Faust (*Functional Audio Stream*) is a functional programming language designed for synchronous real-time signal processing and synthesis applications. A Faust program is a set of bindings of identifiers to expressions that denote signal transformers. A signal s in S is a function mapping¹ times $t \in \mathbb{Z}$ to values $s(t) \in \mathbb{R}$, while a signal transformer is a function from S^n to S^m , where $n, m \in \mathbb{N}$. See the Faust manual for additional information (<http://faust.grame.fr>).
- Every mathematical formula derived from a Faust expression is assumed,

¹Faust assumes that $\forall s \in S, \forall t \in \mathbb{Z}, s(t) = 0$ when $t < 0$.

in this document, to having been normalized (in an implementation-dependent manner) by the Faust compiler.

- A block diagram is a graphical representation of the Faust binding of an identifier I to an expression E ; each graph is put in a box labeled by I . Subexpressions of E are recursively displayed as long as the whole picture fits in one page.
- This document uses the following integer operations:

<i>operation</i>	<i>name</i>	<i>semantics</i>
$i \oplus j$	integer addition	$\text{normalize}(i + j)$, in $\mathbb{Z}$
$i \odot j$	integer multiplication	$\text{normalize}(i \cdot j)$, in $\mathbb{Z}$

Integer operations in Faust are inspired by the semantics of operations on the n -bit two's complement representation of integer numbers; they are internal composition laws on the subset $[-2^{n-1}, 2^{n-1} - 1]$ of $\mathbb{Z}$, with $n = 32$. For any integer binary operation $\times$ on $\mathbb{Z}$, the $\otimes$ operation is defined as: $i \otimes j = \text{normalize}(i \times j)$, with

$$\text{normalize}(i) = i - N \cdot \text{sign}(i) \cdot \left\lfloor \frac{|i| + N/2 + (\text{sign}(i) - 1)/2}{N} \right\rfloor,$$

where $N = 2^n$ and $\text{sign}(i) = 0$ if $i = 0$ and $i/|i|$ otherwise. Unary integer operations are defined likewise.

- The `multipink-mdoc/` directory may also include the following subdirectories:
 - `cpp/` for Faust compiled code;
 - `pdf/` which contains this document;
 - `src/` for all Faust sources used (even libraries);
 - `svg/` for block diagrams, encoded using the Scalable Vector Graphics format (<http://www.w3.org/Graphics/SVG/>);
 - `tex/` for the $\text{\LaTeX}$ source of this document.

4 Faust code listings

This section provides the listings of the Faust code used to generate this document, including dependencies.

Listing 1: `multipink.dsp`

```

1 // SEAM-LTM · MULTIPINK -Faust spec / block-diagram source.
2 //
3 // Canonical DSP: sno.multipink in seam.noises.lib
```

```

4 // multipink(N,g) = no.multinoise(N) : par(i,N, no.pink_filter : *(g));
5 // N independent pink-noise streams at gain g. The C++ plugin re-implements
6 // this by hand (see plugins/multipink/source). This file exists to generate
7 // the block diagram and to document the canonical Faust reference.
8 //
9 // Regenerate diagrams:
10 // faust -I ../../../../faust-libraries/src -svg multipink.dsp
11 //
12 import("seam.lib");
13 process = sno.multipink(4, 0.5);

```

Listing 2: seam.lib

```

1 declare name "Faust SEAM main lib";
2 declare version "0.2";
3 declare author "Giuseppe Silvi";
4 declare author "Davide Tedesco";
5 declare license "CC3";
6 // calling standard faust libraries
7 import("stdfaust.lib");
8 // ### standard faust lib extensions
9 san = library("seam.analyzers.lib");
10 sba = library("seam.basic.lib");
11 sma = library("seam.math.lib");
12 sfi = library("seam.filters.lib");
13 sre = library("seam.reverbs.lib");
14 sno = library("seam.noises.lib");
15 // ### general Ambisonics theory
16 sam = library("seam.ambisonics.lib");
17 sdw = library("seam.dwt.lib");
18 // ### stereophony and perception
19 sst = library("seam.stereophony.lib");
20 // shr = library("seam.hrtf.lib");
21 // ### CSound and MaxMsp object cloning
22 scs = library("seam.csound.lib");
23 scy = library("seam.cyclone.lib");
24 // ### author specific literature
25 // sds = library("seam.discipio.lib");
26 smg = library("seam.gerzon.lib");
27 // sln = library("seam.nono.lib");
28 // sgn = library("seam.nottoli.lib");
29 sjm = library("seam.moorer.lib");
30 scr = library("seam.roads.lib");
31 sms = library("seam.schroeder.lib");
32 sfv = library("seam.freeverb.lib");
33
34 sff = library("seam.ffunctions.lib");
35
36 // ### live electronics stuff
37 // sgu = library("seam.gui.lib");
38 // shw = library("seam.hardware.lib");
39 // smx = library("seam.mixer.lib");
40
41 // ### instrument specific literature
42 // svc = library("seam.vcs3.lib");

```

Listing 3: stdfaust.lib

```

1 //##### stdfaust.lib #####
2 // The purpose of this library is to give access to all the Faust standard libraries

```

```

3 // through a series of environments.
4 //#####
5
6 aa = library("aanl.lib");
7 sf = library("all.lib");
8 an = library("analyzers.lib");
9 ba = library("basics.lib");
10 co = library("compressors.lib");
11 db = library("debug.lib");
12 de = library("delays.lib");
13 dm = library("demos.lib");
14 dx = library("dx7.lib");
15 en = library("envelopes.lib");
16 fd = library("fds.lib");
17 fi = library("filters.lib");
18 ho = library("hoa.lib");
19 hy = library("hysteresis.lib");
20 it = library("interpolators.lib");
21 la = library("linearalgebra.lib");
22 ma = library("maths.lib");
23 mi = library("mi.lib");
24 mo = library("motion.lib");
25 ef = library("misceffects.lib");
26 no = library("noises.lib");
27 os = library("oscillators.lib");
28 pf = library("phaflangers.lib");
29 pl = library("platform.lib");
30 pm = library("physmodels.lib");
31 qu = library("quantizers.lib");
32 rm = library("reducemaps.lib");
33 re = library("reverbs.lib");
34 ro = library("routes.lib");
35 sp = library("spats.lib");
36 si = library("signals.lib");
37 so = library("soundfiles.lib");
38 sy = library("synths.lib");
39 ve = library("vaeffects.lib");
40 vl = library("version.lib");
41 wa = library("webaudio.lib");
42 wd = library("wdmodels.lib");

```

Listing 4: seam.noises.lib

```

1 declare name "SEAM noises - Library";
2 declare version "0.2";
3 declare author "Giuseppe Silvi";
4 declare license "CC3";
5 //
6 import("seam.lib");
7 //===== NOISES ===
8 //=====
9 sno = library("seam.noises.lib");
10 //----- MULTICHANNEL PINK NOISE GENERATOR ---
11 multipink(N,g) = no.multinoise(N) : par(i,N,no.pink_filter : *(g));
12 // process = multipink(4,0.5);
13 //
14 rndw(f,imin,imax,omin,omax) = no.pink_noise :
15                               sfi.lpip(f) : sba.scale1(imin,imax,omin,omax);
16 //process = rndw(1,-0.01,0.01,0,1);

```

Listing 5: noises.lib

```

1 //##### noises.lib #####
2 // Noises library. Its official prefix is `no`.
3 //
4 // This library provides various noise generators and stochastic signal sources
5 // for audio synthesis and testing. It includes white, pink, brown, and blue noise,
6 // as well as pseudo-random number generators and utilities for decorrelated signals
7 // and random modulation in Faust DSP programs.
8 //
9 // The Noises library is organized into 1 section:
10 //
11 // * [Functions Reference](#functions-reference)
12 //
13 // #### References
14 //
15 // <https://github.com/grame-cncm/faustlibraries/blob/master/noises.lib>
16 //#####
17
18 ma = library("maths.lib");
19 ba = library("basics.lib");
20 fi = library("filters.lib");
21 os = library("oscillators.lib");
22 no = library("noises.lib"); // for compatible copy/paste out of this file
23
24 declare name "Faust Noise Generator Library";
25 declare version "1.5.0";
26
27 //=====Functions Reference=====
28 //=====
29
30 /*****
31 *****/
32 FAUST library file, GRAME section
33
34 Except where noted otherwise, Copyright (C) 2003-2017 by GRAME,
35 Centre National de Creation Musicale.
36 -----
37 GRAME LICENSE
38
39 This program is free software; you can redistribute it and/or modify
40 it under the terms of the GNU Lesser General Public License as
41 published by the Free Software Foundation; either version 2.1 of the
42 License, or (at your option) any later version.
43
44 This program is distributed in the hope that it will be useful,
45 but WITHOUT ANY WARRANTY; without even the implied warranty of
46 MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
47 GNU Lesser General Public License for more details.
48
49 You should have received a copy of the GNU Lesser General Public
50 License along with the GNU C Library; if not, write to the Free
51 Software Foundation, Inc., 59 Temple Place, Suite 330, Boston, MA
52 02111-1307 USA.
53
54 EXCEPTION TO THE LGPL LICENSE : As a special exception, you may create a
55 larger FAUST program which directly or indirectly imports this library
56 file and still distribute the compiled code generated by the FAUST
57 compiler, or a modified version of this compiled code, under your own
58 copyright and license. This EXCEPTION TO THE LGPL LICENSE explicitly
59 grants you the right to freely choose the license for the resulting
60 compiled code. In particular the resulting compiled code has no obligation
61 to be LGPL or GPL. For example you are free to choose a commercial or
62 closed source license or any other license if you decide so.
63 *****/
64 *****/
65
66 noise_env(seed) = environment {

```

```

67     RANDMAX = 2147483647.0; // = 2^31-1 = MAX_SIGNED_INT in 32 bits
68
69     noise = random / RANDMAX
70     with {
71         random = +(seed) ~ *(1103515245); // "linear congruential"
72     };
73
74     multirandom(N) = randomize(N) ~ _
75     with {
76         randomize(1) = +(seed) : *(1103515245);
77         randomize(N) = randomize(1) <: randomize(N-1),_;
78     };
79
80     multinoise(N) = multirandom(N) : par(i, N, /(RANDMAX)) : par(i, N, float);
81
82     noises(N,i) = multinoise(N) : ba.selector(i,N);
83
84     dnoise(t,sx) = random / RANDMAX
85     with {
86         random = +(int(seed*sx)*int(t)) ~ *(1103515245*int(t));
87     };
88 };
89
90 };
91
92 //-----`no.)noise`-----
93 // White noise generator (outputs random number between -1 and 1).
94 // `noise` is a standard Faust function.
95 //
96 // ### Usage
97 //
98 // ```
99 // noise : _
100 // ```
101 //
102 // Where:
103 //
104 // * output: white noise signal in [-1, 1].
105 //
106 // ### Test
107 //
108 // ```
109 // no = library("noises.lib");
110 // noise_test = no.noise;
111 // ```
112 //-----
113 noise = noise_env(12345).noise;
114
115 //-----`no.)multirandom`-----
116 // Generates multiple decorrelated random numbers in parallel.
117 //
118 // ### Usage
119 //
120 // ```
121 // multirandom(N) : si.bus(N)
122 // ```
123 //
124 // Where:
125 //
126 // * `N`: the number of decorrelated random numbers in parallel, a constant numerical
127 //   expression
128 //
129 // ### Test
130 //
131 // ```
132 // no = library("noises.lib");
133 // multirandom_test = no.multirandom(3);
134 // ```
135 //-----

```



```

134 multirandom(N) = noise_env(12345).multirandom(N);
135
136
137 //-----`(no.)multinoise`-----
138 // Generates multiple decorrelated noises in parallel.
139 //
140 // #### Usage
141 //
142 // ...
143 // multinoise(N) : si.bus(N)
144 // ...
145 //
146 // Where:
147 //
148 // * `N`: the number of decorrelated random numbers in parallel, a constant numerical
    expression
149 //
150 // #### Test
151 // ...
152 // no = library("noises.lib");
153 // multinoise_test = no.multinoise(3);
154 // ...
155 //-----
156 multinoise(N) = noise_env(12345).multinoise(N);
157
158
159 //-----`(no.)noises`-----
160 // A convenient wrapper around multinoise.
161 //
162 // #### Usage
163 //
164 // ...
165 // noises(N,i) : _
166 // ...
167 //
168 // Where:
169 //
170 // * `N`: the number of decorrelated random numbers in parallel, a constant numerical
    expression
171 // * `i`: the selected random number (i in [0..N])
172 //
173 // #### Test
174 // ...
175 // no = library("noises.lib");
176 // noises_test = no.noises(4, 2);
177 // ...
178 //-----
179 noises(N,i) = noise_env(12345).noises(N,i);
180
181
182 //-----`(no.)dnoise`-----
183 // A deterministic noise burst with a dynamically adjustable seed, enabling consistent recall
    .
184 // Useful for noise variation sensitive applications like replicable/recallable percussion
    sounds and waveguide excitation.
185 //
186 // #### Usage
187 //
188 // ...
189 // dnoise(t,sx) : _
190 // ...
191 //
192 // Where:
193 //
194 // * `t`: is a noise burst trigger
195 // * `sx`: defines the range of integer seed multipliers.
196 //
197 // #### Test

```

```

198 // ```
199 // no = library("noises.lib");
200 // ba = library("basics.lib");
201 // dnoise_test = (trigger, seedRange) : no.dnoise
202 // with {
203 //   trigger = 1 : ba.impulsify;
204 //   seedRange = 10.0;
205 // };
206 // ```
207 //
208 // #### Example
209 // This expression `sx = hslider("seed multiplier",1,1,1000,1)` allows 1000 distinct seed
210 // variations.
211 // To generate a burst with a fixed length, use `ba.spulse(bLength, t)` (as trigger for the `
212 //   t` parameter), where `bLength` is the burst duration in samples and `t` is a trigger.
213 //-----
214 dnoise(t,sx) = noise_env(12345).dnoise(t,sx);
215
216 //-----`(no.)randomseed`-----
217 // A random seed based on the foreign function `arc4random`
218 // (see man arc4random). Used in `rnoise`, `rmultirandom`, etc. to
219 // avoid having the same pseudo random sequence at each run.
220 //
221 // WARNING: using the foreign function `arc4random`, so only available in C/C++ and LLVM
222 // backends.
223 //
224 // #### Usage
225 //
226 // ```
227 // randomseed : _
228 // ```
229 //
230 // Where:
231 //
232 // * output: platform-specific random seed value.
233 //
234 // #### Test
235 // ```
236 // no = library("noises.lib");
237 // randomseed_test = no.randomseed;
238 // ```
239 //-----
240 randomseed = ffunction(int arc4random(), <stdlib.h>, ""), 12345 : select2(1');
241
242 //-----`(no.)rnoise`-----
243 // A randomized white noise generator (outputs random number between -1 and 1).
244 //
245 // WARNING: using the foreign function `arc4random`, so only available in C/C++ and LLVM
246 // backends.
247 //
248 // #### Usage
249 //
250 // ```
251 // rnoise : _
252 // ```
253 //
254 // #### Test
255 // ```
256 // no = library("noises.lib");
257 // rnoise_test = no.rnoise;
258 // ```
259 //-----
260 rnoise = noise_env(randomseed).noise;
261
262 //-----`(no.)rmultirandom`-----
263 // Generates multiple decorrelated random numbers in parallel.

```

```

262 //
263 // WARNING: using the foreign function `arc4random`, so only available in C/C++ and LLVM
    backends.
264 //
265 // #### Usage
266 // ```
267 // rmultirandom(N) : _
268 // ```
269 //
270 // Where:
271 //
272 // * `N`: the number of decorrelated random numbers in parallel, a constant numerical
    expression
273 //
274 // #### Test
275 // ```
276 // no = library("noises.lib");
277 // rmultirandom_test = no.rmultirandom(3);
278 // ```
279 //-----
280 rmultirandom(N) = noise_env(randomseed).multirandom(N);
281
282
283 //-----`(no.)rmultinoise`-----
284 // Generates multiple decorrelated noises in parallel.
285 //
286 // WARNING: using the foreign function `arc4random`, so only available in C/C++ and LLVM
    backends.
287 //
288 // #### Usage
289 // ```
290 // ```
291 // rmultinoise(N) : _
292 // ```
293 //
294 // Where:
295 //
296 // * `N`: the number of decorrelated random numbers in parallel, a constant numerical
    expression
297 //
298 // #### Test
299 // ```
300 // no = library("noises.lib");
301 // rmultinoise_test = no.rmultinoise(3);
302 // ```
303 //-----
304 rmultinoise(N) = noise_env(randomseed).multinoise(N);
305
306
307 //-----`(no.)rnoises`-----
308 // A convenient wrapper around rmultinoise.
309 //
310 // WARNING: using the foreign function `arc4random`, so only available in C/C++ and LLVM
    backends.
311 //
312 // #### Usage
313 // ```
314 // ```
315 // rnoises(N,i) : _
316 // ```
317 //
318 // Where:
319 //
320 // * `N`: the number of decorrelated random numbers in parallel
321 // * `i`: the selected random number (i in [0..N])
322 //
323 // #### Test
324 // ```

```

```

325 // no = library("noises.lib");
326 // rnoises_test = no.rnoises(4, 1);
327 // ```
328 //-----
329 rnoises(N,i) = noise_env(randomseed).noises(N,i);
330
331
332 //#####
333 /*****
334 FAUST library file, jos section
335
336 Except where noted otherwise, The Faust functions below in this
337 section are Copyright (C) 2003-2017 by Julius O. Smith III <jos@ccrma.stanford.edu>
338 ([jos](http://ccrma.stanford.edu/~jos/)), and released under the
339 (MIT-style) [STK-4.3](#stk-4.3-license) license.
340
341 All MarkDown comments in this section are Copyright 2016-2017 by Romain
342 Michon and/or Julius O. Smith III, and are released under the
343 [CCIA4I](https://creativecommons.org/licenses/by/4.0/) license (TODO: if/when Romain agrees)
344
345 *****/
346
347 //-----`no.)pink_noise`-----
348 // Pink noise (1/f noise) generator (third-order approximation covering the audio band well)
349
350 // `pink_noise` is a standard Faust function.
351 //
352 // #### Usage
353 // ```
354 // pink_noise : _
355 // ```
356 // Where:
357 //
358 // * output: pink (1/f) noise signal.
359 //
360 // #### Test
361 // ```
362 // no = library("noises.lib");
363 // pink_noise_test = no.pink_noise;
364 // ```
365 //
366 // #### Alternatives
367 // Higher-order approximations covering any frequency band can be obtained using
368 // ```
369 // no.noise : fi.spectral_tilt(order,lowerBandLimit,Bandwidth,p)
370 // ```
371 // where `p=-0.5` means filter rolloff `f-1/2` which gives 1/f rolloff in the
372 // power spectral density, and can be changed to other real values.
373 //
374 // #### Example
375 // pink_noise_compare.dsp - compare three pinking filters
376 // ```
377 // process = pink_noises with {
378 //   f0 = 35; // Lower bandlimit in Hz
379 //   bw3 = 0.7 * ma.SR/2.0 - f0; // Bandwidth in Hz, 3rd order case
380 //   bw9 = 0.8 * ma.SR/2.0 - f0; // Bandwidth in Hz, 9th order case
381 //   pink_tilt_3 = fi.spectral_tilt(3,f0,bw3,-0.5);
382 //   pink_tilt_9 = fi.spectral_tilt(9,f0,bw9,-0.5);
383 //   pink_noises = 1-1' <:
384 //     no.pink_filter, // original designed by invfreqz in Octave
385 //     pink_tilt_3, // newer method using the same filter order
386 //     pink_tilt_9; // newer method using a higher filter order
387 // };
388 // ```
389 //
390 // #### Output of Example
391 // ```

```

```

392 // faust2octave pink_noise_compare.dsp
393 // Octave:1> semilogx(20*log10(abs(fft(faustout,8192))(1:4096,:)));
394 // ...
395 // ...
396 // 
397 //
398 // #### References
399 //
400 // * <https://ccrma.stanford.edu/~jos/sasp/Example_Synthesis_1_F_Noise.html>
401 //-----
402 pink_filter = fi.iir((0.049922035, -0.095993537, 0.050612699, -0.004408786),
403                     (-2.494956002, 2.017265875, -0.522189400));
404
405 pink_noise = noise : pink_filter;
406
407 pink_noise_m = pink_noise * 12.5; // Equalizes loudness to that of no.noise (thanks Mykle
    Hansen) - beware of clipping
408
409
410 //-----`(no.)pink_noise_vm`-----
411 // Multi pink noise generator.
412 //
413 // #### Usage
414 //
415 // ...
416 // pink_noise_vm(N) : _
417 // ...
418 //
419 // Where:
420 //
421 // * `N`: number of latched white-noise processes to sum,
422 // not to exceed sizeof(int) in C++ (typically 32).
423 //
424 // #### Test
425 // ...
426 // no = library("noises.lib");
427 // pink_noise_vm_test = no.pink_noise_vm(4);
428 // ...
429 //
430 // #### References
431 //
432 // * <http://www.dsprelated.com/showarticle/908.php>
433 // * <http://www.firstpr.com.au/dsp/pink-noise/#Voss-McCartney>
434 //-----
435 pink_noise_vm(N) = noise <: _,par(i,N,ba.latch(clock(i))) :> _
436 with {
437     clock(i) = (ba.time>>i)&1; // i'th latch clock signal
438 };
439
440
441 //-----`(no.)lfnoise`, `(no.)lfnoise0` and `(no.)lfnoiseN`-----
442 // Low-frequency noise generators (Butterworth-filtered downsampled white noise).
443 //
444 // #### Usage
445 //
446 // ...
447 // lfnoise0(rate) : _ // new random number every int(ma.SR/rate) samples or so
448 // lfnoiseN(N,rate) : _ // same as "lfnoise0(rate) : fi.lowpass(N,rate)" [see filters.lib]
449 // lfnoise(rate) : _ // same as "lfnoise0(rate) : seq(i,5,fi.lowpass(N,rate))" (no overshoot)
450 // ...
451 //
452 // #### Example
453 //
454 // (view waveforms in faust2octave):
455 //
456 // ...
457 // rate = ma.SR/100.0; // new random value every 100 samples (ma.SR from maths.lib)

```

```

458 // process = lfnoise0(rate), // sampled/held noise (piecewise constant)
459 //       lfnoiseN(3,rate), // lfnoise0 smoothed by 3rd order Butterworth LPF
460 //       lfnoise(rate); // lfnoise0 smoothed with no overshoot
461 // ...
462 //
463 // #### Test
464 // ...
465 // no = library("noises.lib");
466 // lfnoise0_test = no.lfnoise0(10.0);
467 // ...
468 //-----
469 lfnoise0(freq) = noise : ba.latch(os.oscrs(freq));
470 lfnoiseN(N,freq) = lfnoise0(freq) : fi.lowpass(N,freq); // Nth-order Butterworth lowpass
471 lfnoise(freq) = lfnoise0(freq) : seq(i,5,fi.lowpass(1,freq)); // non-overshooting lowpass
472
473 //-----`no.`sparse_noise`-----
474 // Sparse noise generator.
475 //
476 // #### Usage
477 //
478 // ...
479 // sparse_noise(f0) : _
480 // ...
481 //
482 //
483 // Where:
484 //
485 // * `f0`: average frequency of noise impulses per second
486 //
487 // Random impulses in the amplitude range -1 to 1 are generated
488 // at an average rate of f0 impulses per second.
489 //
490 // #### Test
491 // ...
492 // no = library("noises.lib");
493 // sparse_noise_test = no.sparse_noise(5.0);
494 // ...
495 //
496 // #### References
497 //
498 // * See velvet_noise
499 //-----
500 sparse_noise(f0) = sn
501 with {
502   saw = os.lf_sawpos(f0);
503   sawdiff = saw - saw';
504   e = float(no.noise); // float() keeps 4.656613e-10f scaling here instead of later
505   eHeld = e : ba.latch(sawdiff);
506   eHeldPos = 0.5 + 0.5 * eHeld;
507   crossed = (saw >= eHeldPos) * (saw' < eHeldPos);
508   sn = e' * float(crossed);
509 };
510
511 //-----`no.`velvet_noise_vm`-----
512 // Velvet noise generator.
513 //
514 // #### Usage
515 //
516 // ...
517 // velvet_noise(amp, f0) : _
518 // ...
519 //
520 //
521 // Where:
522 //
523 // * `amp`: amplitude of noise impulses (positive and negative)
524 // * `f0`: average frequency of noise impulses per second
525 //

```

```

526 // #### Test
527 // ---
528 // no = library("noises.lib");
529 // velvet_noise_test = no.velvet_noise(0.5, 5.0);
530 // ---
531 //
532 // #### References
533 //
534 // * Matti Karjalainen and Hanna Jarvelainen,
535 // "Reverberation Modeling Using Velvet Noise",
536 // in Proc. 30th Int. Conf. Intelligent Audio Environments (AES07),
537 // March 2007.
538 //-----
539 velvet_noise(amp, f0) = vn
540 with {
541     sn = no.sparse_noise(f0);
542     vn = amp * ma.signum(sn);
543 };
544
545
546 //-----`no.`gnoise`-----
547 // Approximate zero-mean, unit-variance Gaussian white noise generator.
548 //
549 // #### Usage
550 //
551 // ---
552 // gnoise(N) : _
553 // ---
554 //
555 // Where:
556 //
557 // * `N`: number of uniform random numbers added to approximate Gaussian white noise
558 //
559 // #### Test
560 // ---
561 // no = library("noises.lib");
562 // gnoise_test = no.gnoise(8);
563 // ---
564 //
565 // #### References
566 //
567 // * See Central Limit Theorem
568 //-----
569 gnoise(N) = uvgwn
570 with {
571     uwn = no.multinoise(N); // uniform white noise in [-1,1] on N channels
572     gwn = uwn :> _; // sum of uniform approaches Gaussian by central limit thm
573     sigma = sqrt(N/3.0); // rms of each uwn channel
574     uvgwn = gwn / sigma; // approaches zero-mean, unit-variance Gaussian white noise, for
        large N
575 };
576
577 gnoisem(N) = gnoise(N) * 0.625; // Equalizes loudness to that of no.noise (thanks Mykle
        Hansen)
578
579
580 /*** END jos section ***/
581
582 //#####
583 /#####
584 FAUST library file, further contributions section
585
586 All contributions below should indicate both the contributor and terms
587 of license. If no such indication is found, "git blame" will say who
588 last edited each line, and that person can be emailed to inquire about
589 license disposition, if their license choice is not already indicated
590 elsewhere among the libraries. It is expected that all software will be
591 released under LGPL, STK-4.3, MIT, BSD, or a similar FOSS license.

```

```

592 *****/
593
594 //-----`no.colored_noise`-----
595 // Generates a colored noise signal with an arbitrary spectral
596 // roll-off factor (alpha) over the entire audible frequency range
597 // (20-20000 Hz). The output is normalized so that an equal RMS
598 // level is maintained for different values of alpha.
599 //
600 // #### Usage
601 //
602 // ```
603 // colored_noise(N,alpha) : _
604 // ```
605 //
606 // Where:
607 //
608 // * `N`: desired integer filter order (constant numerical expression)
609 // * `alpha`: slope of roll-off, between -1 and 1. -1 corresponds to
610 // brown/red noise, -1/2 pink noise, 0 white noise, 1/2 blue noise,
611 // and 1 violet/azure noise.
612 //
613 //
614 // #### Test
615 // ```
616 // no = library("noises.lib");
617 // colored_noise_test = no.colored_noise(4, 0.0);
618 // ```
619 //
620 // #### Examples
621 // See `dm.colored_noise_demo`.
622 //
623 //-----
624 declare colored_noise author "Constantinos Odysseas Economou";
625 declare colored_noise copyright "Copyright (C) 2022 Constantinos Odysseas Economou <c.
626     economou@sirenfx.io>";
627 declare colored_noise license "MIT-style STK-4.3 license";
628
629 colored_noise(N,alpha) = no.noise : fi.dcblocker : fi.spectral_tilt(N,fmin,bw,alpha) : *(
630     uniNorm(alpha * 2)) : max(-1.0) : min(1.0)
631 with {
632     fmin = 20.0;
633     fmax = 20000.0;
634     bw = fmax - fmin;
635
636     uniNorm(alpha) = (a*exp(-alpha*b))+(c*exp(-alpha*d)) : pow(-1)
637 with {
638     a = ba.if(ma.signum(alpha) > 0, 1, 0.8016);
639     b = ba.if(ma.signum(alpha) > 0, -4.28, -2.633);
640     c = ba.if(ma.signum(alpha) > 0, 0, 0.1984);
641     d = ba.if(ma.signum(alpha) > 0, 0, -0.7196);
642 };
643 };

```

Listing 6: filters.lib

```

1  /##### filters.lib #####
2  // Filters library. Its official prefix is `fi`.
3  //
4  // This library provides a comprehensive collection of linear and nonlinear
5  // filters used in audio and signal processing. It includes low-pass, high-pass,
6  // band-pass, allpass, shelving, equalizer, and crossover filters, as well as advanced
7  // analog and digital filter design sections for both educational and production use.
8  //
9  // The Filters library is organized into 23 sections:

```



```

10 //
11 // * [Basic Filters] (#basic-filters)
12 // * [Comb Filters] (#comb-filters)
13 // * [Direct-Form Digital Filter Sections] (#direct-form-digital-filter-sections)
14 // * [Direct-Form Second-Order Biquad Sections] (#direct-form-second-order-biquad-sections)
15 // * [Ladder/Lattice Digital Filters] (#ladderlattice-digital-filters)
16 // * [Useful Special Cases] (#useful-special-cases)
17 // * [Ladder/Lattice Allpass Filters] (#ladderlattice-allpass-filters)
18 // * [Digital Filter Sections Specified as Analog Filter Sections] (#digital-filter-sections-
    specified-as-analog-filter-sections)
19 // * [Simple Resonator Filters] (#simple-resonator-filters)
20 // * [Butterworth Lowpass/Highpass Filters] (#butterworth-lowpasshighpass-filters)
21 // * [Special Filter-Bank Delay-Equalizing Allpass Filters] (#special-filter-bank-delay-
    equalizing-allpass-filters)
22 // * [Elliptic (Cauer) Lowpass Filters] (#elliptic-cauer-lowpass-filters)
23 // * [Elliptic Highpass Filters] (#elliptic-highpass-filters)
24 // * [Butterworth Bandpass/Bandstop Filters] (#butterworth-bandpassbandstop-filters)
25 // * [Elliptic Bandpass Filters] (#elliptic-bandpass-filters)
26 // * [Parametric Equalizers (Shelf, Peaking)] (#parametric-equalizers-shelf-peaking)
27 // * [Mth-Octave Filter-Banks] (#mth-octave-filter-banks)
28 // * [Arbitrary-Crossover Filter-Banks and Spectrum Analyzers] (#arbitrary-crossover-filter-
    banks-and-spectrum-analyzers)
29 // * [State Variable Filters (SVF)] (#state-variable-filters)
30 // * [Linkwitz-Riley 4th-order 2-way, 3-way, and 4-way crossovers] (#linkwitz-riley-4th-order
    -2-way-3-way-and-4-way-crossovers)
31 // * [Standardized Filters] (#standardized-filters)
32 // * [Averaging Functions] (#averaging-functions)
33 // * [Kalman Filters] (#kalman-filters)
34 //
35 // #### References
36 //
37 // <https://github.com/grame-cncm/faustlibraries/blob/master/filters.lib>
38 //#####
39
40 // NOTE ABOUT LICENSES:
41 // Each function in this library has its own license. Licenses are declared
42 // before each function. Corresponding license terms can be found at the
43 // bottom of this file or in the Faust libraries documentation.
44
45 ma = library("maths.lib");
46 ba = library("basics.lib");
47 ro = library("routes.lib");
48 de = library("delays.lib");
49 an = library("analyzers.lib");
50 ef = library("misceffects.lib");
51 si = library("signals.lib");
52 fi = library("filters.lib"); // for compatible copy/paste out of this file
53 la = library("linearalgebra.lib");
54
55 declare name "Faust Filters Library";
56 declare version "1.7.1";
57
58 //=====Basic Filters=====
59 //=====
60
61 //-----`fi.`zero`-----
62 // One zero filter. Difference equation:  $y(n) = x(n) - zx(n-1)$ .
63 //
64 // #### Usage
65 //
66 // ```
67 // _ : zero(z) : _
68 // ```
69 //
70 // Where:
71 //
72 // * `z`: location of zero along real axis in z-plane
73 //

```

```

74 // #### Test
75 // ```
76 // fi = library("filters.lib");
77 // os = library("oscillators.lib");
78 // src = os.osc(440);
79 // zero_test = src : fi.zero(0.5);
80 // ```
81 //
82 // #### References
83 // <https://ccrma.stanford.edu/~jos/filters/One\_Zero.html>
84 //-----
85 declare zero author "Julius O. Smith III";
86 declare zero copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.stanford.
    edu>";
87 declare zero license "MIT-style STK-4.3 license";
88 zero(z) = _ <: _,mem : _,*(z) : -;
89
90 //-----`(fi.)pole`-----
91 // One pole filter. Could also be called a "leaky integrator".
92 // Difference equation:  $y(n) = x(n) + py(n-1)$ .
93 //
94 // #### Usage
95 //
96 // ```
97 // _ : pole(p) : _
98 // ```
99 //
100 // Where:
101 //
102 // * `p`: pole location = feedback coefficient
103 //
104 // #### Test
105 // ```
106 // fi = library("filters.lib");
107 // os = library("oscillators.lib");
108 // src = os.osc(440);
109 // pole_test = src : fi.pole(0.9);
110 // ```
111 //
112 // #### References
113 // <https://ccrma.stanford.edu/~jos/filters/One\_Pole.html>
114 //-----
115 declare pole author "Julius O. Smith III";
116 declare pole copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.stanford.
    edu>";
117 declare pole license "MIT-style STK-4.3 license";
118 pole(p) = + - *(p);
119
120 //-----`(fi.)integrator`-----
121 // Same as `pole(1)` [implemented separately for block-diagram clarity].
122 //
123 // #### Usage
124 //
125 // ```
126 // _ : integrator : _
127 // ```
128 //
129 // #### Test
130 // ```
131 // fi = library("filters.lib");
132 // os = library("oscillators.lib");
133 // src = os.osc(440);
134 // integrator_test = src : fi.integrator;
135 // ```
136 //-----
137 declare integrator author "Julius O. Smith III";
138 declare integrator copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
    stanford.edu>";

```

```

139 declare integrator license "MIT-style STK-4.3 license";
140 integrator = + ~ _;
141
142 //-----`(fi.)dcblokerat`-----
143 // DC blocker with configurable "break frequency".
144 // The amplitude response is substantially flat above `fb`,
145 // and sloped at about +6 dB/octave below `fb`.
146 // Derived from the analog transfer function:
147 //  $H(s) = \frac{s}{s + 2 \pi f_b}$ 
148 // (which can be seen as a 1st-order Butterworth highpass filter)
149 // by the low-frequency-matching bilinear transform method
150 // (i.e., using the typical frequency-scaling constant `2*SR`).
151 //
152 // #### Usage
153 //
154 // ...
155 // _ : dcblokerat(fb) : _
156 // ...
157 //
158 // Where:
159 //
160 // * `fb`: "break frequency" in Hz, i.e., -3 dB gain frequency (see 2nd reference below)
161 //
162 // #### Test
163 // ...
164 // fi = library("filters.lib");
165 // os = library("oscillators.lib");
166 // src = os.osc(440);
167 // dcblokerat_test = src : fi.dcblokerat(30);
168 // ...
169 //
170 // #### References
171 //
172 // * <https://ccrma.stanford.edu/~jos/pasp/Bilinear\_Transformation.html>
173 // * <https://ccrma.stanford.edu/~jos/spectilt/Bode\_Plots.html>
174 //-----
175
176 declare dcblokerat author "Julius O. Smith III";
177 declare dcblokerat copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.stanford.edu>";
178 declare dcblokerat license "MIT-style STK-4.3 license";
179 dcblokerat(fb) = *(b0) : zero(1) : pole(p)
180 with {
181     wn = ma.PI*fb/ma.SR;
182     b0 = 1.0 / (1 + wn);
183     p = (1 - wn) * b0;
184 };
185
186 //-----`(fi.)dcbloker`-----
187 // DC blocker. Default dc blocker has -3dB point near 35 Hz (at 44.1 kHz)
188 // and high-frequency gain near 1.0025 (due to no scaling).
189 // `dcbloker` is as standard Faust function.
190 //
191 // #### Usage
192 //
193 // ...
194 // _ : dcbloker : _
195 // ...
196 //
197 // #### Test
198 // ...
199 // fi = library("filters.lib");
200 // os = library("oscillators.lib");
201 // src = os.osc(440);
202 // dcbloker_test = src : fi.dcbloker;
203 // ...
204 //-----
205 declare dcbloker author "Julius O. Smith III";

```

```

206 declare dcblocker copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
      stanford.edu>";
207 declare dcblocker license "MIT-style STK-4.3 license";
208 dcblocker = zero(1) : pole(0.995);
209
210 //-----`(fi.)lptN`-----
211 // One-pole lowpass filter with arbitrary dis/charging factors set in dB and
212 // times set in seconds.
213 //
214 // #### Usage
215 //
216 // ``
217 // _ : lptN(N, tN) : _
218 // ``
219 //
220 // Where:
221 //
222 // * `N`: is the attenuation factor in dB
223 // * `tN`: is the filter period in seconds, that is, the time for the
224 // impulse response to decay by `N` dB
225 //
226 // #### Test
227 //
228 // fi = library("filters.lib");
229 // os = library("oscillators.lib");
230 // src = os.osc(440);
231 // lptN_test = src : fi.lptN(60, 0.1);
232 // ``
233 //
234 // #### References
235 //
236 // <https://ccrma.stanford.edu/~jos/mdft/Exponentials.html>
237 //-----
238 declare lptN author "Julius O. Smith III";
239 declare lptN copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.stanford.
      edu>";
240 declare lptN license "MIT-style STK-4.3 license";
241 lptN(N, tN, x) = x : si.smooth(ba.tau2pole(tN / log(10.0^(float(N)/20.0))));
242 // Special cases of lptN
243 //-----`(fi.)lptau`-----
244 // One-pole lowpass with a tau time constant (1/e attenuation after `tN` seconds).
245 //
246 // #### Usage
247 //
248 // ``
249 // _ : lptau(tN) : _
250 // ``
251 //
252 // Where:
253 //
254 // * `tN`: tau time constant in seconds (1/e attenuation after `tN` seconds)
255 //
256 // #### Test
257 //
258 // fi = library("filters.lib");
259 // os = library("oscillators.lib");
260 // src = os.osc(440);
261 // lptau_test = src : fi.lptau(0.1);
262 // ``
263 //-----
264 lptau(tN, x) = lptN(8.6858896381, tN, x); // Tau time constant, i.e., 1/e atten. after tN
      secs
265
266 //-----`(fi.)lpt60`-----
267 // One-pole lowpass with a T60 time constant (60 dB attenuation after `tN` seconds).
268 //
269 // #### Usage
270 //

```

```

271 // ``
272 // _ : lpt60(tN) : _
273 // ``
274 //
275 // Where:
276 //
277 // * `tN`: T60 time constant in seconds (60 dB attenuation after `tN` seconds)
278 //
279 // #### Test
280 // ``
281 // fi = library("filters.lib");
282 // os = library("oscillators.lib");
283 // src = os.osc(440);
284 // lpt60_test = src : fi.lpt60(0.3);
285 // ``
286 //-----
287 lpt60(tN, x) = lptN(60, tN, x); // T60 constant, i.e., 1/1000 atten. after tN secs
288
289 //-----`(fi.)lpt19`-----
290 // One-pole lowpass with a T19 time constant (approx. 19 dB attenuation after `tN` seconds).
291 //
292 // #### Usage
293 //
294 // ``
295 // _ : lpt19(tN) : _
296 // ``
297 //
298 // Where:
299 //
300 // * `tN`: T19 time constant in seconds (approximately 19 dB attenuation after `tN` seconds)
301 //
302 // #### Test
303 // ``
304 // fi = library("filters.lib");
305 // os = library("oscillators.lib");
306 // src = os.osc(440);
307 // lpt19_test = src : fi.lpt19(0.2);
308 // ``
309 //-----
310 lpt19(tN, x) = lptN(19, tN, x); // T19 constant, i.e., 1/e^2.2 atten. after tN secs
311
312 //=====Comb Filters=====
313 //=====
314
315 //-----`(fi.)ff_comb`-----
316 // Feed-Forward Comb Filter. Note that `ff_comb` requires integer delays
317 // (uses `delay` internally).
318 // `ff_comb` is a standard Faust function.
319 //
320 // #### Usage
321 //
322 // ``
323 // _ : ff_comb(maxdel,intdel,b0,bM) : _
324 // ``
325 //
326 // Where:
327 //
328 // * `maxdel`: maximum delay (a power of 2)
329 // * `intdel`: current (integer) comb-filter delay between 0 and maxdel
330 // * `del`: current (float) comb-filter delay between 0 and maxdel
331 // * `b0`: gain applied to delay-line input
332 // * `bM`: gain applied to delay-line output and then summed with input
333 //
334 // #### Test
335 // ``
336 // fi = library("filters.lib");
337 // os = library("oscillators.lib");
338 // src = os.osc(440);

```

```

339 // ff_comb_test = src : fi.ff_comb(2048, 64, 1, 0.7);
340 // ```
341 //
342 // #### References
343 //
344 // <https://ccrma.stanford.edu/~jos/pasp/Feedforward\_Comb\_Filters.html>
345 //-----
346 declare ff_comb author "Julius O. Smith III";
347 declare ff_comb copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
    stanford.edu>";
348 declare ff_comb license "MIT-style STK-4.3 license";
349 ff_comb(maxdel,M,b0,bM) = _ <: *(b0), bM * de.delay(maxdel,M) : +;
350
351 //-----`(fi.)ff_fcomb`-----
352 // Feed-Forward Comb Filter. Note that `ff_fcomb` takes floating-point delays
353 // (uses `fdelay` internally).
354 // `ff_fcomb` is a standard Faust function.
355 //
356 // #### Usage
357 //
358 // ```
359 // _ : ff_fcomb(maxdel,del,b0,bM) : _
360 // ```
361 //
362 // Where:
363 //
364 // * `maxdel`: maximum delay (a power of 2)
365 // * `del`: current (float) comb-filter delay between 0 and maxdel
366 // * `b0`: gain applied to delay-line input
367 // * `bM`: gain applied to delay-line output and then summed with input
368 //
369 // #### Test
370 // ```
371 // fi = library("filters.lib");
372 // os = library("oscillators.lib");
373 // src = os.osc(440);
374 // ff_fcomb_test = src : fi.ff_fcomb(2048, 64.5, 1, 0.7);
375 // ```
376 //
377 // #### References
378 //
379 // <https://ccrma.stanford.edu/~jos/pasp/Feedforward\_Comb\_Filters.html>
380 //-----
381 declare ff_fcomb author "Julius O. Smith III";
382 declare ff_fcomb copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
    stanford.edu>";
383 declare ff_fcomb license "MIT-style STK-4.3 license";
384 ff_fcomb(maxdel,del,b0,bM) = _ <: *(b0), bM * de.fdelay(maxdel,del) : +;
385
386 //-----`(fi.)ffcombfiler`-----
387 // Typical special case of `ff_comb()` where: `b0 = 1`.
388 //
389 // #### Test
390 // ```
391 // fi = library("filters.lib");
392 // os = library("oscillators.lib");
393 // src = os.osc(440);
394 // ffcombfiler_test = src : fi.ffcombfiler(2048, 64, 0.7);
395 // ```
396 //-----
397 declare ff_combfiler author "Julius O. Smith III";
398 declare ff_combfiler copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
    stanford.edu>";
399 declare ff_combfiler license "MIT-style STK-4.3 license";
400 ffcombfiler(maxdel,del,g) = ff_comb(maxdel,del,1,g);
401
402 //-----`(fi.)fb_comb_common`-----
403 // A generic feedback comb filter.

```

```

404 //
405 // #### Usage
406 //
407 // ```
408 // _ : fb_comb_common(dop,N,b0,aN) : _
409 // ```
410 //
411 // Where
412 //
413 // * `dop`: delay operator, e.g. `@` or `de.fdelay4a(2048)`
414 // * `N`: current delay
415 // * `b0`: gain applied to input
416 // * `aN`: gain applied to delay-line output
417 //
418 // #### Example test program
419 //
420 // ```
421 // process = fb_comb_common(@,N,b0,aN);
422 // ```
423 // implements the following difference equation:
424 // ```
425 // y[n] = b0 x[n] + aN y[n - N]
426 // ```
427 //
428 // See more examples in `filters.lib` below.
429 //
430 // #### Test
431 // ```
432 // fi = library("filters.lib");
433 // os = library("oscillators.lib");
434 // src = os.osc(440);
435 // fb_comb_common_test = src : fi.fb_comb_common(@, 64, 0.8, 0.6);
436 // ```
437 // -----
438 declare fb_comb_common author "Oleg Nesterov";
439 fb_comb_common(dop,N,b0,aN) = + ~ aN * dop(N-1) : *(b0);
440
441 //-----`(fi.)fb_comb`-----
442 // Feed-Back Comb Filter (integer delay).
443 //
444 // #### Usage
445 //
446 // ```
447 // _ : fb_comb(maxdel,del,b0,aN) : _
448 // ```
449 //
450 // Where:
451 //
452 // * `maxdel`: maximum delay (a power of 2)
453 // * `del`: current (float) comb-filter delay between 0 and maxdel
454 // * `b0`: gain applied to delay-line input and forwarded to output
455 // * `aN`: minus the gain applied to delay-line output before summing with the input
456 // and feeding to the delay line
457 //
458 // #### Test
459 // ```
460 // fi = library("filters.lib");
461 // os = library("oscillators.lib");
462 // src = os.osc(440);
463 // fb_comb_test = src : fi.fb_comb(2048, 64, 0.7, 0.6);
464 // ```
465 //
466 // #### References
467 //
468 // <https://ccrma.stanford.edu/~jos/pasp/Feedback\_Comb\_Filters.html>
469 //-----
470 declare fb_comb author "Julius O. Smith III";
471 declare fb_comb copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.

```

```

stanford.edu>, revised by Oleg Nesterov";
472 declare fb_comb license "MIT-style STK-4.3 license";
473 fb_comb(maxdel,del,b0,aN) = fb_comb_common(de.delay(maxdel),del,b0,-aN) : mem;
474
475 //-----`(fi.)fb_fcomb`-----
476 // Feed-Back Comb Filter (floating point delay).
477 //
478 // #### Usage
479 //
480 // ...
481 // _ : fb_fcomb(maxdel,del,b0,aN) : _
482 // ...
483 //
484 // Where:
485 //
486 // * `maxdel`: maximum delay (a power of 2)
487 // * `del`: current (float) comb-filter delay between 0 and maxdel
488 // * `b0`: gain applied to delay-line input and forwarded to output
489 // * `aN`: minus the gain applied to delay-line output before summing with the input
490 // and feeding to the delay line
491 //
492 // #### Test
493 // ...
494 // fi = library("filters.lib");
495 // os = library("oscillators.lib");
496 // src = os.osc(440);
497 // fb_fcomb_test = src : fi.fb_fcomb(2048, 64.5, 0.7, 0.6);
498 // ...
499 //
500 // #### References
501 //
502 // <https://ccrma.stanford.edu/~jos/pasp/Feedback\_Comb\_Filters.html>
503 //-----
504 declare fb_fcomb author "Julius O. Smith III";
505 declare fb_fcomb copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
stanford.edu>, revised by Oleg Nesterov";
506 declare fb_fcomb license "MIT-style STK-4.3 license";
507 fb_fcomb(maxdel,del,b0,aN) = fb_comb_common(de.fdelay(maxdel),del,b0,-aN) : mem;
508
509 //-----`(fi.)rev1`-----
510 // Special case of `fb_comb` (`rev1(maxdel,N,g)`).
511 // The "rev1 section" dates back to the 1960s in computer-music reverberation.
512 // See the `jcrev` and `brassrev` in `reverbs.lib` for usage examples.
513 //
514 // #### Test
515 // ...
516 // fi = library("filters.lib");
517 // os = library("oscillators.lib");
518 // src = os.osc(440);
519 // rev1_test = src : fi.rev1(2048, 64, 0.6);
520 // ...
521 //-----
522 declare rev1 author "Julius O. Smith III";
523 declare rev1 copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.stanford.
edu>";
524 declare rev1 license "MIT-style STK-4.3 license";
525 rev1(maxdel,N,g) = fb_comb(maxdel,N,1,-g);
526
527 //-----`(fi.)fbcombfiler` and `(fi.)ffbcmbfilter`-----
528 // Other special cases of Feed-Back Comb Filter.
529 //
530 // #### Usage
531 //
532 // ...
533 // _ : fbcombfiler(maxdel,intdel,g) : _
534 // _ : ffbcmbfilter(maxdel,del,g) : _
535 // ...
536 //

```



```

537 // Where:
538 //
539 // * `maxdel`: maximum delay (a power of 2)
540 // * `intdel`: current (integer) comb-filter delay between 0 and maxdel
541 // * `del`: current (float) comb-filter delay between 0 and maxdel
542 // * `g`: feedback gain
543 //
544 // #### Test
545 // ```
546 // fi = library("filters.lib");
547 // os = library("oscillators.lib");
548 // src = os.osc(440);
549 // fbcombfilter_test = src : fi.fbcombfilter(2048, 64, 0.6);
550 // ```
551 //
552 // #### References
553 //
554 // <https://ccrma.stanford.edu/~jos/pasp/Feedback\_Comb\_Filters.html>
555 //-----
556 declare fbcombfilter author "Julius O. Smith III";
557 declare fbcombfilter copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
    stanford.edu>";
558 declare fbcombfilter license "MIT-style STK-4.3 license";
559 fbcombfilter(maxdel,intdel,g) = (+ : de.delay(maxdel,intdel)) ~ *(g);
560
561 declare ffbcombfilter author "Julius O. Smith III";
562 declare ffbcombfilter copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
    stanford.edu>";
563 declare ffbcombfilter license "MIT-style STK-4.3 license";
564 ffbcombfilter(maxdel,del,g) = (+ : de.fdelay(maxdel,del)) ~ *(g);
565
566
567 //-----`(fi).allpass_comb`-----
568 // Schroeder Allpass Comb Filter. Note that:
569 //
570 // ```
571 // allpass_comb(maxlen,len,aN) = ff_comb(maxlen,len,aN,1) : fb_comb(maxlen,len-1,1,aN);
572 // ```
573 //
574 // which is a direct-form-1 implementation, requiring two delay lines.
575 // The implementation here is direct-form-2 requiring only one delay line.
576 //
577 // #### Usage
578 //
579 // ```
580 // _ : allpass_comb(maxdel,intdel,aN) : _
581 // ```
582 //
583 // Where:
584 //
585 // * `maxdel`: maximum delay (a power of 2)
586 // * `intdel`: current (integer) comb-filter delay between 0 and maxdel
587 // * `del`: current (float) comb-filter delay between 0 and maxdel
588 // * `aN`: minus the feedback gain
589 //
590 // #### Test
591 // ```
592 // fi = library("filters.lib");
593 // os = library("oscillators.lib");
594 // src = os.osc(440);
595 // allpass_comb_test = src : fi.allpass_comb(2048, 64, 0.6);
596 // ```
597 //
598 // #### References
599 //
600 // * <https://ccrma.stanford.edu/~jos/pasp/Allpass\_Two\_Combs.html>
601 // * <https://ccrma.stanford.edu/~jos/pasp/Schroeder\_Allpass\_Sections.html>
602 // * <https://ccrma.stanford.edu/~jos/filters/Four\_Direct\_Forms.html>

```

```

603 //-----
604 declare allpass_comb author "Julius O. Smith III";
605 declare allpass_comb copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
    stanford.edu>";
606 declare allpass_comb license "MIT-style STK-4.3 license";
607 allpass_comb(maxdel,N,aN) = (+ <: de.delay(maxdel,N-1),*(aN)) ~ *(-aN) : mem,_ : +;
608
609
610 //-----`(fi.)allpass_fcomb`-----
611 // Schroeder Allpass Comb Filter. Note that:
612 //
613 // ...
614 // allpass_comb(maxlen,len,aN) = ff_comb(maxlen,len,aN,1) : fb_comb(maxlen,len-1,1,aN);
615 // ...
616 //
617 // which is a direct-form-1 implementation, requiring two delay lines.
618 // The implementation here is direct-form-2 requiring only one delay line.
619 //
620 // `allpass_fcomb` is a standard Faust library.
621 //
622 // #### Usage
623 //
624 // ...
625 // _ : allpass_comb(maxdel,intdel,aN) : _
626 // _ : allpass_fcomb(maxdel,del,aN) : _
627 // ...
628 //
629 // Where:
630 //
631 // * `maxdel`: maximum delay (a power of 2)
632 // * `intdel`: current (float) comb-filter delay between 0 and maxdel
633 // * `del`: current (float) comb-filter delay between 0 and maxdel
634 // * `aN`: minus the feedback gain
635 //
636 // #### Test
637 // ...
638 // fi = library("filters.lib");
639 // os = library("oscillators.lib");
640 // src = os.osc(440);
641 // allpass_fcomb_test = src : fi.allpass_fcomb(2048, 64.5, 0.6);
642 // ...
643 //
644 // #### References
645 //
646 // * <https://ccrma.stanford.edu/~jos/pasp/Allpass_Two_Combs.html>
647 // * <https://ccrma.stanford.edu/~jos/pasp/Schroeder_Allpass_Sections.html>
648 // * <https://ccrma.stanford.edu/~jos/filters/Four_Direct_Forms.html>
649 //-----
650 declare allpass_fcomb author "Julius O. Smith III";
651 declare allpass_fcomb copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
    stanford.edu>";
652 declare allpass_fcomb license "MIT-style STK-4.3 license";
653 allpass_fcomb(maxdel,N,aN) = (+ <: de.fdelay(maxdel,N-1),*(aN)) ~ *(-aN) : mem,_ : +;
654
655
656 //-----`(fi.)rev2`-----
657 // Special case of `allpass_comb` (`rev2(maxlen,len,g)`).
658 // The "rev2 section" dates back to the 1960s in computer-music reverberation.
659 // See the `jcrev` and `brassrev` in `reverbs.lib` for usage examples.
660 //
661 // #### Test
662 // ...
663 // fi = library("filters.lib");
664 // os = library("oscillators.lib");
665 // src = os.osc(440);
666 // rev2_test = src : fi.rev2(2048, 64, 0.6);
667 // ...
668 //-----

```

```

669 declare rev2 author "Julius O. Smith III";
670 declare rev2 copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.stanford.
    edu>";
671 declare rev2 license "MIT-style STK-4.3 license";
672 rev2(maxlen,len,g) = allpass_comb(maxlen,len,-g);
673
674 //-----`(fi.)allpass_fcomb5` and `(fi.)allpass_fcomb1a`-----
675 // Same as `allpass_fcomb` but use `fdelay5` and `fdelay1a` internally
676 // (Interpolation helps - look at an fft of faust2octave on:
677 // `1-1` <: allpass_fcomb(1024,10.5,0.95), allpass_fcomb5(1024,10.5,0.95);`)
678 //
679 // #### Test
680 // ```
681 // fi = library("filters.lib");
682 // os = library("oscillators.lib");
683 // src = os.osc(440);
684 // allpass_fcomb5_test = src : fi.allpass_fcomb5(2048, 64.5, 0.6);
685 // ```
686 //-----
687 declare allpass_fcomb5 author "Julius O. Smith III";
688 declare allpass_fcomb5 copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
    stanford.edu>";
689 declare allpass_fcomb5 license "MIT-style STK-4.3 license";
690 allpass_fcomb5(maxdel,N,aN) = (+ <: de.fdelay5(maxdel,N-1),*(aN)) ~ *(-aN) : mem,_ : +;
691
692 declare allpass_fcomb1a author "Julius O. Smith III";
693 declare allpass_fcomb1a copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
    stanford.edu>";
694 declare allpass_fcomb1a license "MIT-style STK-4.3 license";
695 allpass_fcomb1a(maxdel,N,aN) = (+ <: de.fdelay1a(maxdel,N-1),*(aN)) ~ *(-aN) : mem,_ : +;
696
697
698 //=====Direct-Form Digital Filter Sections=====
699 //=====
700
701 // Specified by transfer-function polynomials B(z)/A(z) as in matlab
702
703 //-----`(fi.)iir`-----
704 // Nth-order Infinite-Impulse-Response (IIR) digital filter,
705 // implemented in terms of the Transfer-Function (TF) coefficients.
706 // Such filter structures are termed "direct form".
707 //
708 // `iir` is a standard Faust function.
709 //
710 // #### Usage
711 //
712 // ```
713 // _ : iir(bcoeffs,acoeffs) : _
714 // ```
715 //
716 // Where:
717 //
718 // * `bcoeffs`: (b0,b1,...,b_order) = TF numerator coefficients
719 // * `acoeffs`: (a1,...,a_order) = TF denominator coeffs (a0=1)
720 //
721 // #### Test
722 // ```
723 // fi = library("filters.lib");
724 // os = library("oscillators.lib");
725 // src = os.osc(440);
726 // iir_test = src : fi.iir((0.5, 0.5), (0.3));
727 // ```
728 //
729 // #### References
730 //
731 // <https://ccrma.stanford.edu/~jos/filters/Four_Direct_Forms.html>
732 //-----
733 declare iir author "Julius O. Smith III";

```

```

734 declare iir copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.stanford.
      edu>";
735 declare iir license "MIT-style STK-4.3 license";
736 iir(bv,av) = ma.sub ~ fir(av) : fir(bv);
737
738 //-----`fi.fir`-----
739 // FIR filter (convolution of FIR filter coefficients with a signal). `fir` is standard Faust
function.
740 //
741 // #### Usage
742 //
743 // ```
744 // _ : fir(bv) : _
745 // ```
746 //
747 // Where:
748 //
749 // * `bv`: `b0,b1,...,bn`, a parallel bank of coefficient signals
750 //
751 // #### Note
752 //
753 // `bv` is processed using pattern-matching at compile time,
754 // so it must have this normal form (parallel signals).
755 //
756 // #### Example test program
757 //
758 // Smoothing white noise with a five-point moving average:
759 //
760 // ```
761 // bv = .2,.2,.2,.2,.2;
762 // process = noise : fir(bv);
763 // ```
764 //
765 // Equivalent (note double parens):
766 //
767 // ```
768 // process = noise : fir((.2,.2,.2,.2,.2));
769 // ```
770 //-----
771 //fir(bv) = conv(bv);
772 //
773 // #### Test
774 // ```
775 // fi = library("filters.lib");
776 // os = library("oscillators.lib");
777 // src = os.osc(440);
778 // fir_test = src : fi.fir((0.2, 0.2, 0.2, 0.2, 0.2));
779 // ```
780 //-----
781 declare fir author "Julius O. Smith III";
782 declare fir copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.stanford.
      edu>";
783 declare fir license "MIT-style STK-4.3 license";
784 fir((b0,bv)) = _ <: *(b0), R(1,bv) :> _ with {
785     R(n,(bn,bv)) = (@(n):*(bn)), R(n+1,bv);
786     R(n, bn) = (@(n):*(bn)); };
787 fir(b0) = *(b0);
788
789 //-----`fi.conv` and `fi.convN`-----
790 // Convolution of input signal with given coefficients.
791 //
792 // #### Usage
793 //
794 // ```
795 // _ : conv((k1,k2,k3,...,kN)) : _ // Argument = one signal bank
796 // _ : convN(N,(k1,k2,k3,...)) : _ // Useful when N < count((k1,...))
797 // ```
798 //-----

```

```

799 //convN(N,kv,x) = sum(i,N,ba.take(i+1,kv) * x@i); // take() defined in basics.lib
800 //
801 // #### Test
802 // ```
803 // fi = library("filters.lib");
804 // os = library("oscillators.lib");
805 // si = library("signals.lib");
806 // src = os.osc(440);
807 // convN_test = (src <: si.bus(3)) : fi.convN(3, (0.3, 0.2, 0.1, 0.05));
808 // ```
809
810 declare convN author "Julius O. Smith III";
811 declare convN copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.stanford.
    edu>";
812 declare convN license "MIT-style STK-4.3 license";
813 convN(N,kv) = sum(i,N, @i)*ba.take(i+1,kv)); // take() defined in basics.lib
814 //conv(kv,x) = sum(i,count(kv),take(i+1,kv) * x@i); // count() from math.lib
815
816 declare conv author "Julius O. Smith III";
817 declare conv copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.stanford.
    edu>";
818 declare conv license "MIT-style STK-4.3 license";
819 conv(kv) = fir(kv);
820
821 //-----`(fi.)tf1`, `(fi.)tf2` and `(fi.)tf3`-----
822 // tfN = N'th-order direct-form digital filter.
823 //
824 // #### Usage
825 //
826 // ```
827 // _ : tf1(b0,b1,a1) : _
828 // _ : tf2(b0,b1,b2,a1,a2) : _
829 // _ : tf3(b0,b1,b2,b3,a1,a2,a3) : _
830 // ```
831 //
832 // Where:
833 //
834 // * `b`: transfer-function numerator
835 // * `a`: transfer-function denominator (monic)
836 //
837 // #### Test
838 // ```
839 // fi = library("filters.lib");
840 // os = library("oscillators.lib");
841 // src = os.osc(440);
842 // tf1_test = src : fi.tf1(0.5, 0.25, -0.4);
843 // ```
844 //
845 // #### References
846 //
847 // <https://ccrma.stanford.edu/~jos/fp/Direct\_Form\_I.html>
848 //-----
849 declare tf1 author "Julius O. Smith III";
850 declare tf1 copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.stanford.
    edu>";
851 declare tf1 license "MIT-style STK-4.3 license";
852 tf1(b0,b1,a1) = _ <: *(b0), (mem : *(b1)) :> + ~ *(0-a1);
853
854 declare tf2 author "Julius O. Smith III";
855 declare tf2 copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.stanford.
    edu>";
856 declare tf2 license "MIT-style STK-4.3 license";
857 tf2(b0,b1,b2,a1,a2) = iir((b0,b1,b2),(a1,a2));
858 // tf2 is a variant of tf22 below with duplicated mems
859
860 declare tf3 author "Julius O. Smith III";
861 declare tf3 copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.stanford.
    edu>";

```

```

862 declare tf3 license "MIT-style STK-4.3 license";
863 tf3(b0,b1,b2,b3,a1,a2,a3) = iir((b0,b1,b2,b3),(a1,a2,a3));
864
865 // "Original" version for music.lib. This is here for comparison but people should
866 // use tf2 instead
867 TF2(b0,b1,b2,a1,a2) = sub ~ conv2(a1,a2) : conv3(b0,b1,b2)
868 with {
869     conv3(k0,k1,k2,x) = k0*x + k1*x' + k2*x'';
870     conv2(k0,k1,x) = k0*x + k1*x';
871     sub(x,y) = y-x;
872 };
873
874 //-----`(fi.)notchw`-----
875 // Simple notch filter based on a biquad (`tf2`).
876 // `notchw` is a standard Faust function.
877 //
878 // #### Usage:
879 //
880 // ...
881 // _ : notchw(width,freq) : _
882 // ...
883 //
884 // Where:
885 //
886 // * `width`: "notch width" in Hz (approximate)
887 // * `freq`: "notch frequency" in Hz
888 //
889 // #### Test
890 // ...
891 // fi = library("filters.lib");
892 // os = library("oscillators.lib");
893 // src = os.osc(440);
894 // notchw_test = src : fi.notchw(200, 1000);
895 // ...
896 //
897 // #### References
898 //
899 // <https://ccrma.stanford.edu/~jos/pasp/Phasing\_2nd\_Order\_Allpass\_Filters.html>
900 //-----
901 declare notchw author "Julius O. Smith III";
902 declare notchw copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.stanford.edu>";
903 declare notchw license "MIT-style STK-4.3 license";
904 notchw(width,freq) = tf2(b0,b1,b2,a1,a2)
905 with {
906     fb = 0.5*width; // First design a dcblocker at (width/2)
907     wn = ma.PI*fb/ma.SR;
908     b0db = 1.0 / (1 + wn);
909     p = (1 - wn) * b0db; // This is our pole radius.
910     // Now place unit-circle zeros at desired angles:
911     tn = 2*ma.PI*freq/ma.SR;
912     a2 = p * p;
913     a2p1 = 1+a2;
914     a1 = -a2p1*cos(tn);
915     b1 = a1;
916     b0 = 0.5*a2p1;
917     b2 = b0;
918 };
919
920 //=====Direct-Form Second-Order Biquad Sections=====
921 // Direct-Form Second-Order Biquad Sections
922 //
923 // #### References
924 //
925 // <https://ccrma.stanford.edu/~jos/filters/Four\_Direct\_Forms.html>
926 //=====
927
928 //-----`(fi.)tf21`, `(fi.)tf22`, `(fi.)tf22t` and `(fi.)tf21t`

```

```

929 // tfN = N'th-order direct-form digital filter where:
930 //
931 // * `tf21` is tf2, direct-form 1
932 // * `tf22` is tf2, direct-form 2
933 // * `tf22t` is tf2, direct-form 2 transposed
934 // * `tf21t` is tf2, direct-form 1 transposed
935 //
936 // #### Usage
937 //
938 // ```
939 // _ : tf21(b0,b1,b2,a1,a2) : _
940 // _ : tf22(b0,b1,b2,a1,a2) : _
941 // _ : tf22t(b0,b1,b2,a1,a2) : _
942 // _ : tf21t(b0,b1,b2,a1,a2) : _
943 // ```
944 //
945 // Where:
946 //
947 // * `b` : transfer-function numerator
948 // * `a` : transfer-function denominator (monic)
949 //
950 // #### Test
951 // ```
952 // fi = library("filters.lib");
953 // os = library("oscillators.lib");
954 // src = os.osc(440);
955 // tf21_test = src : fi.tf21(0.1, 0.2, 0.1, -0.5, 0.06);
956 // ```
957 //
958 // #### References
959 //
960 // <https://ccrma.stanford.edu/~jos/fp/Direct\_Form\_I.html>
961 //-----
962 declare tf21 author "Julius O. Smith III";
963 declare tf21 copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.stanford.edu>";
964 declare tf21 license "MIT-style STK-4.3 license";
965 tf21(b0,b1,b2,a1,a2) = // tf2, direct-form 1:
966   _ <: (mem<:(mem:*(b2)),*(b1))),*(b0) :>_
967   : ((_,_,_>_) ~(<:*(-a1),(mem:*(-a2))));
968
969 declare tf22 author "Julius O. Smith III";
970 declare tf22 copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.stanford.edu>";
971 declare tf22 license "MIT-style STK-4.3 license";
972 tf22(b0,b1,b2,a1,a2) = // tf2, direct-form 2:
973   _ : (((_,_,_>_)~*(-a1)<:mem,*(b0))~*(-a2))
974   : (<:mem,*(b1)),_ : *(b2),_,_> _;
975
976 declare tf22t author "Julius O. Smith III";
977 declare tf22t copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.stanford.edu>";
978 declare tf22t license "MIT-style STK-4.3 license";
979 tf22t(b0,b1,b2,a1,a2) = // tf2, direct-form 2 transposed:
980   _ : (_,_,_<: *(b2)',*(b1)',*(b0))
981   : _,+',_,_> _~*(-a1)~*(-a2) : _;
982
983 declare tf21t author "Julius O. Smith III";
984 declare tf21t copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.stanford.edu>";
985 declare tf21t license "MIT-style STK-4.3 license";
986 tf21t(b0,b1,b2,a1,a2) = // tf2, direct-form 1 transposed:
987   tf22t(1,0,0,a1,a2) : tf22t(b0,b1,b2,0,0); // or write it out if you want
988
989 //===== Ladder/Lattice Digital Filters =====
990 // Ladder and lattice digital filters generally have superior numerical
991 // properties relative to direct-form digital filters. They can be derived

```

```

992 // from digital waveguide filters, which gives them a physical interpretation.
993
994 // #### References
995 //
996 // * F. Itakura and S. Saito: "Digital Filtering Techniques for Speech Analysis and
    Synthesis",
997 // 7th Int. Cong. Acoustics, Budapest, 25 C 1, 1971.
998 // * J. D. Markel and A. H. Gray: Linear Prediction of Speech, New York: Springer Verlag,
    1976.
999 // * <https://ccrma.stanford.edu/~jos/pasp/Conventional\_Ladder\_Filters.html>
1000 //=====
1001
1002 //-----`(fi.)av2sv`-----
1003 // Compute reflection coefficients sv from transfer-function denominator av.
1004 //
1005 // #### Usage
1006 //
1007 // ```
1008 // sv = av2sv(av)
1009 // ```
1010 //
1011 // Where:
1012 //
1013 // * `av`: parallel signal bank `a1,...,aN`
1014 // * `sv`: parallel signal bank `s1,...,sN`
1015 //
1016 // where `ro` = ith` reflection coefficient, and
1017 // `ai` = coefficient of `z^(-i)` in the filter
1018 // transfer-function denominator `A(z)`.
1019 //
1020 // #### Test
1021 // ```
1022 // fi = library("filters.lib");
1023 // si = library("signals.lib");
1024 // av2sv_test = fi.av2sv((-0.4, 0.1)) : si.bus(2);
1025 // ```
1026 //
1027 // #### References
1028 //
1029 // <https://ccrma.stanford.edu/~jos/filters/Step\_Down\_Procedure.html>
1030 // (where reflection coefficients are denoted by k rather than s).
1031 //-----
1032 declare av2sv author "Julius O. Smith III";
1033 declare av2sv copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.stanford.edu>";
1034 declare av2sv license "MIT-style STK-4.3 license";
1035 av2sv(av) = par(i,M,s(i+1)) with {
1036   M = ba.count(av);
1037   s(m) = sr(M-m+1); // m=1..M
1038   sr(m) = Ari(m,M-m+1); // s_{M-1-m}
1039   Ari(m,i) = ba.take(i+1,Ar(m-1));
1040   //step-down recursion for lattice/ladder digital filters:
1041   Ar(0) = (1,av); // Ar(m) is order M-m (i.e. "reverse-indexed")
1042   Ar(m) = 1,par(i,M-m, (Ari(m,i+1) - sr(m)*Ari(m,M-m-i))/(1-sr(m)*sr(m)));
1043 };
1044
1045 //-----`(fi.)bvav2nuv`-----
1046 // Compute lattice tap coefficients from transfer-function coefficients.
1047 //
1048 // #### Usage
1049 //
1050 // ```
1051 // nuv = bvav2nuv(bv,av)
1052 // ```
1053 //
1054 // Where:
1055 //
1056 // * `av`: parallel signal bank `a1,...,aN`

```



```

1057 // * `bv`: parallel signal bank `b0,b1,...,aN`
1058 // * `nuv`: parallel signal bank `nu1,...,nuN`
1059 //
1060 // where `nui` is the i'th tap coefficient,
1061 //     `bi` is the coefficient of `z^(-i)` in the filter numerator,
1062 //     `ai` is the coefficient of `z^(-i)` in the filter denominator
1063 //
1064 // #### Test
1065 // ...
1066 // fi = library("filters.lib");
1067 // si = library("signals.lib");
1068 // bvav2nuv_test = fi.bvav2nuv((0.1, 0.2, 0.3), (-0.4, 0.1)) : si.bus(3);
1069 // ...
1070 //-----
1071 declare bvav2nuv author "Julius O. Smith III";
1072 declare bvav2nuv copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
    stanford.edu>";
1073 declare bvav2nuv license "MIT-style STK-4.3 license";
1074 bvav2nuv(bv,av) = par(m,M+1,nu(m)) with {
1075     M = ba.count(av);
1076     nu(m) = ba.take(m+1,Pr(M-m)); // m=0..M
1077     // lattice/ladder tap parameters:
1078     Pr(0) = bv; // Pr(m) is order M-m, 'r' means "reversed"
1079     Pr(m) = par(i,M-m+1, (Pri(m,i) - nu(M-m+1)*Ari(m,M-m-i+1)));
1080     Pri(m,i) = ba.take(i+1,Pr(m-1));
1081     Ari(m,i) = ba.take(i+1,Ar(m-1));
1082     //step-down recursion for lattice/ladder digital filters:
1083     Ar(0) = (1,av); // Ar(m) is order M-m (recursion index must start at constant)
1084     Ar(m) = 1,par(i,M-m, (Ari(m,i+1) - sr(m)*Ari(m,M-m-i))/(1-sr(m)*sr(m)));
1085     sr(m) = Ari(m,M-m+1); // s_{M-i-m}
1086 };
1087
1088 //-----`fi.iir_lat2`-----
1089 // Two-multiply lattice IIR filter of arbitrary order.
1090 //
1091 // #### Usage
1092 //
1093 // ...
1094 // _ : iir_lat2(bv,av) : _
1095 // ...
1096 //
1097 // Where:
1098 //
1099 // * `bv`: transfer-function numerator
1100 // * `av`: transfer-function denominator (monic)
1101 //
1102 // #### Test
1103 // ...
1104 // fi = library("filters.lib");
1105 // os = library("oscillators.lib");
1106 // src = os.osc(440);
1107 // iir_lat2_test = src : fi.iir_lat2((0.1, 0.2, 0.3), (-0.4, 0.1));
1108 // ...
1109 //-----
1110 declare iir_lat2 author "Julius O. Smith III";
1111 declare iir_lat2 copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
    stanford.edu>";
1112 declare iir_lat2 license "MIT-style STK-4.3 license";
1113 iir_lat2(bv,av) = allpassnt(M,sv) : sum(i,M+1,*(ba.take(M-i+1,tg)))
1114 with {
1115     M = ba.count(av);
1116     sv = av2sv(av); // sv = vector of sin(theta) reflection coefficients
1117     tg = bvav2nuv(bv,av); // tg = vector of tap gains
1118 };
1119
1120 //-----`fi.allpassnt`-----
1121 // Two-multiply lattice allpass (nested order-1 direct-form-ii allpasses), with taps.
1122 //

```

```

1123 // #### Usage
1124 //
1125 // ...
1126 // _ : allpassnt(n,sv) : si.bus(n+1)
1127 // ...
1128 //
1129 // Where:
1130 //
1131 // * `n`: the order of the filter
1132 // * `sv`: the reflection coefficients (-1 1)
1133 //
1134 // The first output is the n-th order allpass output,
1135 // while the remaining outputs are taps taken from the
1136 // input of each delay element from the input to the output.
1137 // See (fi.)allpassn for the single-output case.
1138 //
1139 // #### Test
1140 // ...
1141 // fi = library("filters.lib");
1142 // os = library("oscillators.lib");
1143 // si = library("signals.lib");
1144 // src = os.osc(440);
1145 // allpassnt_test = src : fi.allpassnt(2, (0.3, -0.2)) : si.bus(3);
1146 // ...
1147 //-----
1148 declare allpassnt author "Julius O. Smith III";
1149 declare allpassnt copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
    stanford.edu>";
1150 declare allpassnt license "MIT-style STK-4.3 license";
1151 allpassnt(0,sv) = _;
1152 allpassnt(n,sv) = _ : ((+ <: (allpassnt(n-1,sv),*(s)))*(-s)) : fsec(n)
1153 with {
1154   fsec(1) = ro.crossnn(1) : _, (<:mem,_) : +,_;
1155   fsec(n) = ro.crossn1(n) : _, (<:mem,_,par(i,n-1,_) : +, par(i,n,_)
1156   innertaps(n) = par(i,n,_)
1157   s = ba.take(n,sv); // reflection coefficient s = sin(theta)
1158 };
1159
1160 //-----`fi.)iir_kl`-----
1161 // Kelly-Lochbaum ladder IIR filter of arbitrary order.
1162 //
1163 // #### Usage
1164 //
1165 // ...
1166 // _ : iir_kl(bv,av) : _
1167 // ...
1168 //
1169 // Where:
1170 //
1171 // * `bv`: transfer-function numerator
1172 // * `av`: transfer-function denominator (monic)
1173 //
1174 // #### Test
1175 // ...
1176 // fi = library("filters.lib");
1177 // os = library("oscillators.lib");
1178 // src = os.osc(440);
1179 // iir_kl_test = src : fi.iir_kl((0.1, 0.2, 0.3), (-0.4, 0.1));
1180 // ...
1181 //-----
1182 declare iir_kl author "Julius O. Smith III";
1183 declare iir_kl copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.stanford.
    edu>";
1184 declare iir_kl license "MIT-style STK-4.3 license";
1185 iir_kl(bv,av) = allpassnkt(M,sv) : sum(i,M+1,*(tghr(i)))
1186 with {
1187   M = ba.count(av);
1188   sv = av2sv(av); // sv = vector of sin(theta) reflection coefficients

```

```

1189   tg = bvav2nuv(bv,av); // tg = vector of tap gains for 2mul case
1190   tgr(i) = ba.take(M+1-i,tg);
1191   tghr(n) = tgr(n)/pi(n);
1192   pi(0) = 1;
1193   pi(n) = pi(n-1)*(1+ba.take(M-n+1,sv)); // all sign parameters '+'
1194 };
1195
1196 //-----`(fi.)allpassnkl`-----
1197 // Kelly-Lochbaum ladder allpass.
1198 //
1199 // #### Usage:
1200 //
1201 // ...
1202 // _ : allpassnkl(n,sv) : _
1203 // ...
1204 //
1205 // Where:
1206 //
1207 // * `n`: the order of the filter
1208 // * `sv`: the reflection coefficients (-1 1)
1209 //
1210 // #### Test
1211 // ...
1212 // fi = library("filters.lib");
1213 // os = library("oscillators.lib");
1214 // si = library("signals.lib");
1215 // src = os.osc(440);
1216 // allpassnkl_test = src : fi.allpassnkl(2, (0.3, -0.2)) : si.bus(3);
1217 // ...
1218 //-----
1219 declare allpassnkl author "Julius O. Smith III";
1220 declare allpassnkl copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
    stanford.edu>";
1221 declare allpassnkl license "MIT-style STK-4.3 license";
1222 allpassnkl(0,sv) = _;
1223 allpassnkl(n,sv) = _ <: *(s),*(1+s) : (+
    : allpassnkl(n-1,sv))-*(-s))) : fsec(n)
1224
1225 with {
1226   fsec(1) = _, (_<:mem*(1-s),_) : sumandtaps(n);
1227   fsec(n) = _, (_<:mem*(1-s),_) , par(i,n-1,_) : sumandtaps(n);
1228   s = ba.take(n,sv);
1229   sumandtaps(n) = +,par(i,n,_)
1230 };
1231
1232 //-----`(fi.)iir_lat1`-----
1233 // One-multiply lattice IIR filter of arbitrary order.
1234 //
1235 // #### Usage
1236 //
1237 // ...
1238 // _ : iir_lat1(bv,av) : _
1239 // ...
1240 //
1241 // Where:
1242 //
1243 // * `bv`: transfer-function numerator as a bank of parallel signals
1244 // * `av`: transfer-function denominator as a bank of parallel signals
1245 //
1246 // #### Test
1247 // ...
1248 // fi = library("filters.lib");
1249 // os = library("oscillators.lib");
1250 // src = os.osc(440);
1251 // iir_lat1_test = src : fi.iir_lat1((0.1, 0.2, 0.3), (-0.4, 0.1));
1252 // ...
1253 //-----
1254 declare iir_lat1 author "Julius O. Smith III";
1255 declare iir_lat1 copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.

```

```

stanford.edu>";
1256 declare iir_lat1 license "MIT-style STK-4.3 license";
1257 iir_lat1(bv,av) = allpassn1mt(M,sv) : sum(i,M+1,*(tghr(i+1)))
1258 with {
1259     M = ba.count(av);
1260     sv = av2sv(av); // sv = vector of sin(theta) reflection coefficients
1261     tg = bvav2nuv(bv,av); // tg = vector of tap gains
1262     tgr(i) = ba.take(M+2-i,tg); // i=1..M+1 (for "takability")
1263     tghr(n) = tgr(n)/pi(n);
1264     pi(1) = 1;
1265     pi(n) = pi(n-1)*(1+ba.take(M-n+2,sv)); // all sign parameters '+'
1266 };
1267
1268 //-----`(fi.)allpassn1mt`-----
1269 // One-multiply lattice allpass with tap lines.
1270 //
1271 // #### Usage
1272 //
1273 // ...
1274 // _ : allpassn1mt(N,sv) : _
1275 // ...
1276 //
1277 // Where:
1278 //
1279 // * `N`: the order of the filter (fixed at compile time)
1280 // * `sv`: the reflection coefficients (-1 1)
1281 //
1282 // #### Test
1283 // ...
1284 // fi = library("filters.lib");
1285 // os = library("oscillators.lib");
1286 // si = library("signals.lib");
1287 // src = os.osc(440);
1288 // allpassn1mt_test = src : fi.allpassn1mt(2, (0.3, -0.2)) : si.bus(3);
1289 // ...
1290 //-----
1291 declare allpassn1mt author "Julius O. Smith III";
1292 declare allpassn1mt copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
stanford.edu>";
1293 declare allpassn1mt license "MIT-style STK-4.3 license";
1294 allpassn1mt(0,sv) = _;
1295 allpassn1mt(n,sv) = _ <: _,_ : ((+:(s) <: _,_) : _,+ : ro.crossnn(1)
1296 : allpassn1mt(n-1,sv),_)-(*(-1)) : fsec(n)
1297 with {
1298     fsec(1) = ro.crossnn(1) : _, (_<:mem,_) : +,_;
1299     fsec(n) = ro.crossnn(1) : _, (_<:mem,_) : par(i,n-1,_) : +, par(i,n,_) ;
1300     innertaps(n) = par(i,n,_) ;
1301     s = ba.take(n,sv); // reflection coefficient s = sin(theta)
1302 };
1303
1304 //-----`(fi.)iir_nl`-----
1305 // Normalized ladder filter of arbitrary order.
1306 //
1307 // #### Usage
1308 //
1309 // ...
1310 // _ : iir_nl(bv,av) : _
1311 // ...
1312 //
1313 // Where:
1314 //
1315 // * `bv`: transfer-function numerator
1316 // * `av`: transfer-function denominator (monic)
1317 //
1318 // #### Test
1319 // ...
1320 // fi = library("filters.lib");
1321 // os = library("oscillators.lib");

```

```

1322 // src = os.osc(440);
1323 // iir_nl_test = src : fi.iir_nl((0.1, 0.2, 0.3), (-0.4, 0.1));
1324 // ...
1325 //
1326 // #### References
1327 //
1328 // * J. D. Markel and A. H. Gray, Linear Prediction of Speech, New York: Springer Verlag,
1329 //   1976.
1330 // * <https://ccrma.stanford.edu/~jos/pasp/Normalized\_Scattering\_Junctions.html>
1331 //-----
1332 declare iir_nl author "Julius O. Smith III";
1333 declare iir_nl copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.stanford.edu>";
1334 declare iir_nl license "MIT-style STK-4.3 license";
1335 iir_nl(bv,av) = allpassnlt(M,sv) : sum(i,M+1,*(tghr(i)))
1336 with {
1337   M = ba.count(av);
1338   sv = av2sv(av); // sv = vector of sin(theta) reflection coefficients
1339   tg = bvav2nuv(bv,av); // tg = vector of tap gains for 2mul case
1340   tgr(i) = ba.take(M+1-i,tg);
1341   tghr(n) = tgr(n)/pi(n);
1342   pi(0) = 1;
1343   s(n) = ba.take(M-n+1,sv); // reflection coefficient = sin(theta)
1344   c(n) = sqrt(max(0,1-s(n)*s(n))); // compiler crashes on sqrt(-)
1345   pi(n) = pi(n-1)*c(n);
1346 };
1347 //-----`fi.)allpassnlt`-----
1348 // Normalized ladder allpass filter of arbitrary order.
1349 //
1350 // #### Usage:
1351 //
1352 // ...
1353 // _ : allpassnlt(N,sv) : _
1354 // ...
1355 //
1356 // Where:
1357 //
1358 // * `N`: the order of the filter (fixed at compile time)
1359 // * `sv`: the reflection coefficients (-1 1)
1360 //
1361 // #### Test
1362 // ...
1363 // fi = library("filters.lib");
1364 // os = library("oscillators.lib");
1365 // si = library("signals.lib");
1366 // src = os.osc(440);
1367 // allpassnlt_test = src : fi.allpassnlt(2, (0.3, -0.2)) : si.bus(3);
1368 // ...
1369 //
1370 // #### References
1371 //
1372 // * J. D. Markel and A. H. Gray, Linear Prediction of Speech, New York: Springer Verlag,
1373 //   1976.
1374 // * <https://ccrma.stanford.edu/~jos/pasp/Normalized\_Scattering\_Junctions.html>
1375 //-----
1376 declare allpassnlt author "Julius O. Smith III";
1377 declare allpassnlt copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.stanford.edu>";
1378 declare allpassnlt license "MIT-style STK-4.3 license";
1379 allpassnlt(0,sv) = _;
1380 allpassnlt(n,scl*(sv)) = allpassnlt(n,par(i,count(sv),scl*(sv(i))));
1381 allpassnlt(n,sv) = _ <: *(s),*(c) : (+
1382   : allpassnlt(n-1,sv))-*(s)) : fsec(n)
1383 with {
1384   fsec(1) = _, (_<:mem*(c),_) : sumandtaps(n);
1385   fsec(n) = _, (_<:mem*(c),_) : par(i,n-1,_) : sumandtaps(n);
1386   s = ba.take(n,sv);

```

```

1386     c = sqrt(max(0,1-s*s));
1387     sumandtaps(n) = +,par(i,n,_);
1388 };
1389
1390 //=====Useful Special Cases=====
1391 //=====
1392 //-----`(fi.)tf2np`-----
1393 // Biquad based on a stable second-order Normalized Ladder Filter
1394 // (more robust to modulation than `tf2` and protected against instability).
1395 //
1396 // #### Usage
1397 //
1398 // ```
1399 // _ : tf2np(b0,b1,b2,a1,a2) : _
1400 // ```
1401 //
1402 // Where:
1403 //
1404 // * `b`: transfer-function numerator
1405 // * `a`: transfer-function denominator (monic)
1406 //
1407 // #### Test
1408 // ```
1409 // fi = library("filters.lib");
1410 // os = library("oscillators.lib");
1411 // src = os.osc(440);
1412 // tf2np_test = src : fi.tf2np(0.6, 0.3, 0.2, -0.5, 0.2);
1413 // ```
1414 //-----
1415
1416 declare tf2np author "Julius O. Smith III";
1417 declare tf2np copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.stanford.edu>";
1418 declare tf2np license "MIT-style STK-4.3 license";
1419
1420 tf2np(b0,b1,b2,a1,a2) = allpassnlt(M,sv) : sum(i,M+1,*(tghr(i)))
1421 with {
1422     smax = 1.0-ma.EPSILON; // maximum reflection-coefficient magnitude allowed
1423     s2 = max(-smax, min(smax,a2)); // Project both reflection-coefficients
1424     s1 = max(-smax, min(smax,a1/(1+a2))); // into the defined stability-region.
1425     sv = (s1,s2); // vector of sin(theta) reflection coefficients
1426     M = 2;
1427     nu(2) = b2;
1428     nu(1) = b1 - b2*a1;
1429     nu(0) = (b0-b2*a2) - nu(1)*s1;
1430     tg = (nu(0),nu(1),nu(2));
1431     tgr(i) = ba.take(M+1-i,tg); // vector of tap gains for 2mul case
1432     tghr(n) = tgr(n)/pi(n); // apply pi parameters for NLF case
1433     pi(0) = 1;
1434     s(n) = ba.take(M-n+1,sv);
1435     c(n) = sqrt(1-s(n)*s(n));
1436     pi(n) = pi(n-1)*c(n);
1437 };
1438
1439 //-----`(fi.)wgr`-----
1440 // Second-order transformer-normalized digital waveguide resonator.
1441 //
1442 // #### Usage
1443 //
1444 // ```
1445 // _ : wgr(f,r) : _
1446 // ```
1447 //
1448 // Where:
1449 //
1450 // * `f`: resonance frequency (Hz)
1451 // * `r`: loss factor for exponential decay (set to 1 to make a numerically stable oscillator
1452 )

```

```

1452 //
1453 // #### Test
1454 // ```
1455 // fi = library("filters.lib");
1456 // os = library("oscillators.lib");
1457 // src = os.osc(440);
1458 // wgr_test = fi.wgr(440, 0.995, src);
1459 // ```
1460 //
1461 // #### References
1462 //
1463 // * <https://ccrma.stanford.edu/~jos/pasp/Power\_Normalized\_Waveguide\_Filters.html>
1464 // * <https://ccrma.stanford.edu/~jos/pasp/Digital\_Waveguide\_Oscillator.html>
1465 //-----
1466 declare wgr author "Julius O. Smith III";
1467 declare wgr copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.stanford.
    edu>";
1468 declare wgr license "MIT-style STK-4.3 license";
1469 wgr(f,r,x) = (*G),_<:_,(+:(*(C))<:_,_)+:_,(x,-) ~ cross : _,*(0-gi)
1470 with {
1471   C = cos(2*ma.PI*f/ma.SR);
1472   gi = sqrt(max(0,(1+C)/(1-C))); // compensate amplitude (only needed when
1473   G = r*(1-1' + gi')/gi; // frequency changes substantially)
1474   cross = _,_ <: !,_,_,!;
1475 };
1476
1477 //-----`fi.)nlf2`-----
1478 // Second order normalized digital waveguide resonator.
1479 //
1480 // #### Usage
1481 //
1482 // ```
1483 // _ : nlf2(f,r) : _
1484 // ```
1485 //
1486 // Where:
1487 //
1488 // * `f`: resonance frequency (Hz)
1489 // * `r`: loss factor for exponential decay (set to 1 to make a sinusoidal oscillator)
1490 //
1491 // #### Test
1492 // ```
1493 // fi = library("filters.lib");
1494 // os = library("oscillators.lib");
1495 // src = os.osc(440);
1496 // nlf2_test = fi.nlf2(440, 0.995, src);
1497 // ```
1498 //
1499 // #### References
1500 //
1501 // <https://ccrma.stanford.edu/~jos/pasp/Power\_Normalized\_Waveguide\_Filters.html>
1502 //-----
1503 declare nlf2 author "Julius O. Smith III";
1504 declare nlf2 copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.stanford.
    edu>";
1505 declare nlf2 license "MIT-style STK-4.3 license";
1506 nlf2(f,r,x) = ((_<:_,_),(_<:_,_)) : (*(s),*(c),*(c),*(0-s)) :>
1507   (*(r),+(x)) ~ cross
1508 with {
1509   th = 2*ma.PI*f/ma.SR;
1510   c = cos(th);
1511   s = sin(th);
1512   cross = _,_ <: !,_,_,!;
1513 };
1514
1515 //-----`fi.)apnl`-----
1516 // Passive Nonlinear Allpass based on Pierce switching springs idea.

```

```

1518 // Switch between allpass coefficient `a1` and `a2` at signal zero crossings.
1519 //
1520 // #### Usage
1521 //
1522 // ```
1523 // _ : apnl(a1,a2) : _
1524 // ```
1525 //
1526 // Where:
1527 //
1528 // * `a1`: first allpass coefficient
1529 // * `a2`: second allpass coefficient
1530 //
1531 // #### Test
1532 // ```
1533 // fi = library("filters.lib");
1534 // os = library("oscillators.lib");
1535 // src = os.osc(440);
1536 // apnl_test = fi.apnl(0.5, -0.5, src);
1537 // ```
1538 //
1539 // #### References
1540 //
1541 // * "A Passive Nonlinear Digital Filter Design ..." by John R. Pierce and Scott
1542 //   A. Van Duyne, JASA, vol. 101, no. 2, pp. 1120-1126, 1997
1543 //-----
1544 declare apnl author "Julius O. Smith III";
1545 declare apnl copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.stanford.
    edu>";
1546 declare apnl license "MIT-style STK-4.3 license";
1547 apnl(a1,a2,x) = nonLinFilter
1548 with {
1549   condition = _>0;
1550   nonLinFilter = (x - _ <: _*(condition*a1 + (1-condition)*a2),_')~_ :> +;
1551 };
1552
1553
1554 //=====Ladder/Lattice Allpass Filters=====
1555 // An allpass filter has gain 1 at every frequency, but variable phase.
1556 // Ladder/lattice allpass filters are specified by reflection coefficients.
1557 // They are defined here as nested allpass filters, hence the names `allpassn*`.
1558 //
1559 // #### References
1560 //
1561 // * <https://ccrma.stanford.edu/~jos/pasp/Conventional_Ladder_Filters.html>
1562 // * <https://ccrma.stanford.edu/~jos/pasp/Nested_Allpass_Filters.html>
1563 // * Linear Prediction of Speech, Markel and Gray, Springer Verlag, 1976
1564 //=====
1565
1566 //-----`(fi.)scatN`-----
1567 // N-port scattering junction.
1568 //
1569 // #### Usage
1570 //
1571 // ```
1572 // si.bus(N) : scatN(N,av,filter) : si.bus(N)
1573 // ```
1574 //
1575 // Where:
1576 //
1577 // * `N`: number of incoming/outgoing waves
1578 // * `av`: vector (list) of `N` alpha parameters (each between 0 and 2, and normally summing
    to 2): <https://ccrma.stanford.edu/~jos/pasp/Alpha_Parameters.html>
1579 // * `filter`: optional junction filter to apply (`_` for none, see below)
1580 //
1581 // With no filter:
1582 //
1583 // - The junction is _lossless_ when the alpha parameters sum to 2 ("allpass").

```



```

1584 // - The junction is _passive_ but lossy when the alpha parameters sum to less than 2 ("
      resistive loss").
1585 // - Dynamic and reactive junctions are obtained using the `filter` argument.
1586 // For guaranteed stability, the filter should be _positive real_. (See 2nd ref. below).
1587 //
1588 // For  $(N=2)$  (two-port scattering), the reflection coefficient  $\rho$  corresponds
1589 // to alpha parameters  $\alpha_{pm\rho}$ .
1590 //
1591 // #### Example: Whacky echo chamber made of 16 lossless "acoustic tubes":
1592 //
1593 // ---
1594 // process = _ : *(1.0/sqrt(N)) <: daisyRev(16,2,0.9999) :> _,_ with {
1595 //   daisyRev(N,Dp2,G) = si.bus(N) : (si.bus(2*N) :> si.bus(N)
1596 //     : fi.scatN(N, par(i,N,2*G/float(N)), fi.lowpass(1,5000.0))
1597 //     : par(i,N,de.delay(DS(i),DS(i)-1))) ~ si.bus(N) with { DS(i) = 2^(Dp2+i); };
1598 // };
1599 // ---
1600 //
1601 // #### Test
1602 // ---
1603 // fi = library("filters.lib");
1604 // os = library("oscillators.lib");
1605 // dual_src = os.osc(440), os.osc(660);
1606 // scatN_test = dual_src : fi.scatN(2, (1, 1), _);
1607 // ---
1608 //
1609 // #### References
1610 //
1611 // * <https://ccrma.stanford.edu/~jos/pasp/Loaded_Waveguide_Junctions.html>
1612 // * <https://ccrma.stanford.edu/~jos/pasp/Passive_String_Terminations.html>
1613 // * <https://ccrma.stanford.edu/~jos/pasp/Unloaded_Junctions_Alpha_Parameters.html>
1614 //-----
1615 declare scatN author "Julius O. Smith III";
1616 declare scatN copyright "Copyright (C) 2024 by Julius O. Smith III <jos@ccrma.stanford.edu
      >";
1617 declare scatN license "MIT-style STK-4.3 license";
1618
1619 scatN(0,av,filter) = !;
1620 scatN(N,av,filter) = incomingWaves <: (junctionSum : filter <: si.bus(N)), par(i,N,*(-1)) :>
      si.bus(N)
1621 with {
1622   incomingWaves = si.bus(N);
1623   alpha(i) = ba.take(i+1,av);
1624   junctionSum = par(i,N,(alpha(i))) :> _; // Junction velocity/pressure for series/parallel
      junction
1625   outgoingWaves = junctionSum <: si.bus(N), negatedIncoming :> si.bus(N);
1626   alphaSum = sum(i,N,alpha(i));
1627 };
1628
1629 //-----`fi.)scat`-----
1630 // Scatter off of reflectance r with reflection coefficient s.
1631 //
1632 // #### Usage:
1633 //
1634 // ---
1635 // _ : scat(s,r) : _
1636 // ---
1637 // Where:
1638 //
1639 // * `s`: reflection coefficient between -1 and 1 for stability
1640 // * `r`: single-input, single-output block diagram,
1641 //   having gain less than 1 at all frequencies for stability.
1642 //
1643 // #### Example: the following program should produce all zeros:
1644 //
1645 // ---
1646 // process = fi.allpassn(3,(.3,.2,.1)), fi.scat(.1, fi.scat(.2, fi.scat(.3, _)))
1647 //   :> - : ^2) : +~_;

```

```

1648 // ```
1649 //
1650 // #### Test
1651 // ```
1652 // fi = library("filters.lib");
1653 // os = library("oscillators.lib");
1654 // src = os.osc(440);
1655 // scat_test = src : fi.scats(0.5, _);
1656 // ```
1657 //
1658 // #### References
1659 //
1660 // * <https://ccrma.stanford.edu/~jos/pasp/Scattering\_Impedance\_Changes.html>
1661 //-----
1662 declare scat author "Julius O. Smith III";
1663 declare scat copyright "Copyright (C) 2024 by Julius O. Smith III <jos@ccrma.stanford.edu>";
1664 declare scat license "MIT-style STK-4.3 license";
1665
1666 scat(s,r) = _ <: ((+ <: (r,*(s)))-(*(s))) : _',_ :+;
1667
1668 //-----`fi.allpassn`-----
1669 // Two-multiply lattice filter.
1670 //
1671 // #### Usage:
1672 //
1673 // ```
1674 // _ : allpassn(n,sv) : _
1675 // ```
1676 // Where:
1677 //
1678 // * `n`: the order of the filter
1679 // * `sv`: the reflection coefficients `(s1,s2,...,sN)`, each between -1 and 1
1680 //
1681 // Equivalent to `fi.allpassnt(n,sv) : _, par(i,n,!);`
1682 //
1683 // Equivalent to `fi.scats(s(n), fi.scats(s(n-1), ..., fi.scats(s(1), _)))`
1684 // with { s(k) = ba.take(k,sv); } ;`
1685 //
1686 // Identical to `allpassn` in `old/filter.lib`.
1687 //
1688 // #### Test
1689 // ```
1690 // fi = library("filters.lib");
1691 // os = library("oscillators.lib");
1692 // src = os.osc(440);
1693 // allpassn_test = src : fi.allpassn(3, (0.3, 0.2, 0.1));
1694 // ```
1695 //
1696 // #### References
1697 //
1698 // * J. D. Markel and A. H. Gray: Linear Prediction of Speech, New York: Springer Verlag, 1976.
1699 // * <https://ccrma.stanford.edu/~jos/pasp/Conventional\_Ladder\_Filters.html>
1700 //-----
1701 declare allpassn author "Julius O. Smith III";
1702 declare allpassn copyright "Copyright (c) 2003-2024 by Julius O. Smith III <jos@ccrma.stanford.edu>";
1703 declare allpassn license "MIT-style STK-4.3 license";
1704 allpassn(0,sv) = _;
1705 allpassn(n,sv) = _ <: ((+ <: (allpassn(n-1,sv)),*(s)))-(*(s))) : _',_ :+
1706 with { s = ba.take(n,sv); };
1707
1708 //-----`fi.allpassnn`-----
1709 // Normalized form - four multiplies and two adds per section,
1710 // but coefficients can be time varying and nonlinear without
1711 // "parametric amplification" (modulation of signal energy).
1712 //
1713 // #### Usage:

```

```

1714 //
1715 // ~~~
1716 // _ : allpassnn(n,tv) : _
1717 // ~~~
1718 //
1719 // Where:
1720 //
1721 // * `n`: the order of the filter
1722 // * `tv`: the reflection coefficients (-PI PI)
1723 //
1724 // #### Test
1725 // ~~~
1726 // fi = library("filters.lib");
1727 // os = library("oscillators.lib");
1728 // src = os.osc(440);
1729 // allpassnn_test = src : fi.allpassnn(3, (0.3, 0.2, 0.1));
1730 // ~~~
1731 //-----
1732 // power-normalized (reflection coefficients s = sin(t)):
1733 declare allpassnn author "Julius O. Smith III";
1734 declare allpassnn copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
    stanford.edu>";
1735 declare allpassnn license "MIT-style STK-4.3 license";
1736
1737 allpassnn(0,tv) = _;
1738 allpassnn(n,tv) = _ <: *(s), (*(c) : (+
1739     : allpassnn(n-1,tv))-*(-s))) : _, mem*c : +
1740 with { c = cos(ba.take(n,tv)); s = sin(ba.take(n,tv)); };
1741
1742 //-----`(fi.)allpassnkl`-----
1743 // Kelly-Lochbaum form - four multiplies and two adds per
1744 // section, but all signals have an immediate physical
1745 // interpretation as traveling pressure waves, etc.
1746 //
1747 // #### Usage:
1748 //
1749 // ~~~
1750 // _ : allpassnkl(n,sv) : _
1751 // ~~~
1752 //
1753 // Where:
1754 //
1755 // * `n`: the order of the filter
1756 // * `sv`: the reflection coefficients (-1 1)
1757 //
1758 // #### Test
1759 // ~~~
1760 // fi = library("filters.lib");
1761 // os = library("oscillators.lib");
1762 // src = os.osc(440);
1763 // allpassnkl_test = src : fi.allpassnkl(3, (0.3, 0.2, 0.1));
1764 // ~~~
1765 //-----
1766 // Kelly-Lochbaum:
1767 declare allpassnkl author "Julius O. Smith III";
1768 declare allpassnkl copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
    stanford.edu>";
1769 declare allpassnkl license "MIT-style STK-4.3 license";
1770
1771 allpassnkl(0,sv) = _;
1772 allpassnkl(n,sv) = _ <: *(s),(*(1+s) : (+
1773     : allpassnkl(n-1,sv))-*(-s))) : _, mem*(1-s) : +
1774 with { s = ba.take(n,sv); };
1775
1776 //-----`(fi.)allpass1m`-----
1777 // One-multiply form - one multiply and three adds per section.
1778 // Normally the most efficient in special-purpose hardware.
1779 //

```

```

1780 // #### Usage:
1781 //
1782 // ...
1783 // _ : allpassnm(n,sv) : _
1784 // ...
1785 //
1786 // Where:
1787 //
1788 // * `n`: the order of the filter
1789 // * `sv`: the reflection coefficients (-1 1)
1790 //
1791 // #### Test
1792 // ...
1793 // fi = library("filters.lib");
1794 // os = library("oscillators.lib");
1795 // src = os.osc(440);
1796 // allpassnm_test = src : fi.allpassnm(3, (0.3, 0.2, 0.1));
1797 // ...
1798 //-----
1799 // one-multiply:
1800 declare allpassnm author "Julius O. Smith III";
1801 declare allpassnm copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
    stanford.edu>";
1802 declare allpassnm license "MIT-style STK-4.3 license";
1803
1804 allpassnm(0,sv) = _;
1805 allpassnm(n,sv) = _ <: _,_ : ((+:(s) <: _,_),_ : _,+ : cross
1806     : allpassnm(n-1,sv),_)-(*(-1)) : _',_ : +
1807 with { s = ba.take(n,sv); cross = _,_ <: !,_,_!; };
1808
1809 //=====Digital Filter Sections Specified as Analog Filter Sections=====
1810 //=====
1811
1812 //-----`(fi.)tf2s` and `(fi.)tf2snp`-----
1813 // Second-order direct-form digital filter,
1814 // specified by ANALOG transfer-function polynomials B(s)/A(s),
1815 // and a frequency-scaling parameter. Digitization via the
1816 // bilinear transform is built in.
1817 //
1818 // #### Usage
1819 //
1820 // ...
1821 // _ : tf2s(b2,b1,b0,a1,a0,w1) : _
1822 // ...
1823 // Where:
1824 //
1825 // ...
1826 //      b2 s^2 + b1 s + b0
1827 // H(s) = -----
1828 //      s^2 + a1 s + a0
1829 // ...
1830 //
1831 // and `w1` is the desired digital frequency (in radians/second)
1832 // corresponding to analog frequency 1 rad/sec (i.e., `s = j`).
1833 //
1834 // #### Example test program
1835 //
1836 // A second-order ANALOG Butterworth lowpass filter,
1837 // normalized to have cutoff frequency at 1 rad/sec,
1838 // has transfer function:
1839 //
1840 // ...
1841 //      1
1842 // H(s) = -----
1843 //      s^2 + a1 s + 1
1844 // ...
1845 //
1846 // where `a1 = sqrt(2)`. Therefore, a DIGITAL Butterworth lowpass

```

```

1847 // cutting off at `SR/4` is specified as `tf2s(0,0,1,sqrt(2),1,PI*SR/2);`
1848 //
1849 // #### Method
1850 //
1851 // Bilinear transform scaled for exact mapping of w1.
1852 //
1853 // #### Test
1854 // ...
1855 // fi = library("filters.lib");
1856 // os = library("oscillators.lib");
1857 // ma = library("maths.lib");
1858 // src = os.osc(440);
1859 // tf2s_test = src : fi.tf2s(0, 0, 1, sqrt(2), 1, ma.PI*ma.SR/2);
1860 // ...
1861 //
1862 // #### References
1863 //
1864 // * <https://ccrma.stanford.edu/~jos/pasp/Bilinear\_Transformation.html>
1865 //-----
1866 declare tf2s author "Julius O. Smith III";
1867 declare tf2s copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.stanford.edu>";
1868 declare tf2s license "MIT-style STK-4.3 license";
1869 tf2s(b2,b1,b0,a1,a0,w1) = tf2(b0d,b1d,b2d,a1d,a2d)
1870 with {
1871   c = 1/tan(w1*0.5/ma.SR); // bilinear-transform scale-factor
1872   csq = c*c;
1873   d = a0 + a1 * c + csq;
1874   b0d = (b0 + b1 * c + b2 * csq)/d;
1875   b1d = 2 * (b0 - b2 * csq)/d;
1876   b2d = (b0 - b1 * c + b2 * csq)/d;
1877   a1d = 2 * (a0 - csq)/d;
1878   a2d = (a0 - a1*c + csq)/d;
1879 };
1880
1881 // tf2snp = tf2s but using a protected normalized ladder filter for tf2:
1882 tf2snp(b2,b1,b0,a1,a0,w1) = tf2np(b0d,b1d,b2d,a1d,a2d)
1883 with {
1884   c = 1/tan(w1*0.5/ma.SR); // bilinear-transform scale-factor
1885   csq = c*c;
1886   d = a0 + a1 * c + csq;
1887   b0d = (b0 + b1 * c + b2 * csq)/d;
1888   b1d = 2 * (b0 - b2 * csq)/d;
1889   b2d = (b0 - b1 * c + b2 * csq)/d;
1890   a1d = 2 * (a0 - csq)/d;
1891   a2d = (a0 - a1*c + csq)/d;
1892 };
1893
1894 //-----
1895 // #### Test
1896 // ...
1897 // fi = library("filters.lib");
1898 // os = library("oscillators.lib");
1899 // ma = library("maths.lib");
1900 // src = os.osc(440);
1901 // tf2snp_test = src : fi.tf2snp(0, 0, 1, sqrt(2), 1, ma.PI*ma.SR/2);
1902 // ...
1903 //-----
1904
1905 //-----`(fi.)tf1snp`-----
1906 // First-order special case of tf2snp above.
1907 //
1908 // #### Usage
1909 //
1910 // ...
1911 // _ : tf1snp(b1,b0,a0,w1) : _
1912 // ...
1913 //

```

```

1914 // Where:
1915 //
1916 // * `b1`: analog numerator coefficient for `s`
1917 // * `b0`: analog numerator constant coefficient
1918 // * `a0`: analog denominator constant coefficient
1919 // * `w1`: desired digital frequency (radians/second) corresponding to analog frequency `1
           rad/sec`
1920 //
1921 // #### Test
1922 // ```
1923 // fi = library("filters.lib");
1924 // os = library("oscillators.lib");
1925 // ma = library("maths.lib");
1926 // src = os.osc(440);
1927 // tf1snp_test = src : fi.tf1snp(0, 1, 1, ma.PI*ma.SR/2);
1928 // ```
1929 //-----
1930 declare tf1snp author "Julius O. Smith III";
1931 declare tf1snp copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.stanford
           .edu>";
1932 declare tf1snp license "MIT-style STK-4.3 license";
1933 tf1snp(b1,b0,a0,w1) = fi.tf2snp(b1,b0,0,a0,0,w1); // FIXME: Faust compiler does not fully
           optimize - does C++?
1934
1935 //-----`fi.`tf3s1f`-----
1936 // Analogous to `tf2s` above, but third order, and using the typical
1937 // low-frequency-matching bilinear-transform constant 2/T ("lf" series)
1938 // instead of the specific-frequency-matching value used in `tf2s` and `tf1s`.
1939 // Note the lack of a "w1" argument.
1940 //
1941 //
1942 // #### Usage
1943 //
1944 // ```
1945 // _ : tf3s1f(b3,b2,b1,b0,a3,a2,a1,a0) : _
1946 // ```
1947 //
1948 // Where:
1949 //
1950 // * `b3`: analog numerator coefficient of `s^3`
1951 // * `b2`: analog numerator coefficient of `s^2`
1952 // * `b1`: analog numerator coefficient of `s`
1953 // * `b0`: analog numerator constant term
1954 // * `a3`: analog denominator coefficient of `s^3`
1955 // * `a2`: analog denominator coefficient of `s^2`
1956 // * `a1`: analog denominator coefficient of `s`
1957 // * `a0`: analog denominator constant term
1958 //
1959 // #### Test
1960 // ```
1961 // fi = library("filters.lib");
1962 // os = library("oscillators.lib");
1963 // src = os.osc(440);
1964 // tf3s1f_test = src : fi.tf3s1f(0, 0, 0, 1, 1, 2, 2, 1);
1965 // ```
1966 //-----
1967 declare tf3s1f author "Julius O. Smith III";
1968 declare tf3s1f copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.stanford
           .edu>";
1969 declare tf3s1f license "MIT-style STK-4.3 license";
1970 tf3s1f(b3,b2,b1,b0,a3,a2,a1,a0) = tf3(b0d,b1d,b2d,b3d,a1d,a2d,a3d) with {
1971   c = 2.0 * ma.SR; // bilinear-transform scale-factor ("lf" case)
1972   csq = c*c;
1973   cc = csq*c;
1974   // Thank you maxima:
1975   b3d = (b3*c^3-b2*c^2+b1*c-b0)/d;
1976   b2d = (-3*b3*c^3+b2*c^2+b1*c-3*b0)/d;
1977   b1d = (3*b3*c^3+b2*c^2-b1*c-3*b0)/d;

```

```

1978     b0d = (-b3*c^3-b2*c^2-b1*c-b0)/d;
1979     a3d = (a3*c^3-a2*c^2+a1*c-a0)/d;
1980     a2d = (-3*a3*c^3+a2*c^2+a1*c-3*a0)/d;
1981     a1d = (3*a3*c^3+a2*c^2-a1*c-3*a0)/d;
1982     d = (-a3*c^3-a2*c^2-a1*c-a0);
1983 };
1984
1985 //-----`(fi.)tf1s`-----
1986 // First-order direct-form digital filter,
1987 // specified by ANALOG transfer-function polynomials B(s)/A(s),
1988 // and a frequency-scaling parameter.
1989 //
1990 // #### Usage
1991 //
1992 // ```
1993 // _ : tf1s(b1,b0,a0,w1) : _
1994 // ```
1995 //
1996 // Where:
1997 //
1998 // * `b1`: analog numerator coefficient of `s`
1999 // * `b0`: analog numerator constant term
2000 // * `a0`: analog denominator constant term in `s + a0`
2001 // * `w1`: desired digital frequency in radians per second corresponding to analog frequency
2002 //         `1 rad/s`
2003 //
2004 // Where:
2005 //
2006 //         b1 s + b0
2007 // H(s) = -----
2008 //         s + a0
2009 // and `w1` is the desired digital frequency (in radians/second)
2010 // corresponding to analog frequency 1 rad/sec (i.e., `s = j`).
2011 //
2012 // #### Example test program
2013 //
2014 // A first-order ANALOG Butterworth lowpass filter,
2015 // normalized to have cutoff frequency at 1 rad/sec,
2016 // has transfer function:
2017 //
2018 //         1
2019 // H(s) = -----
2020 //         s + 1
2021 //
2022 // so `b0 = a0 = 1` and `b1 = 0`. Therefore, a DIGITAL first-order
2023 // Butterworth lowpass with gain -3dB at `SR/4` is specified as
2024 //
2025 // ```
2026 // tf1s(0,1,1,PI*SR/2); // digital half-band order 1 Butterworth
2027 // ```
2028 //
2029 // #### Method
2030 //
2031 // Bilinear transform scaled for exact mapping of w1.
2032 //
2033 // #### Test
2034 //
2035 // fi = library("filters.lib");
2036 // os = library("oscillators.lib");
2037 // ma = library("maths.lib");
2038 // src = os.osc(440);
2039 // tf1s_test = src : fi.tf1s(0, 1, 1, ma.PI*ma.SR/2);
2040 // ```
2041 //
2042 // #### References
2043 //
2044 // * <https://ccrma.stanford.edu/~jos/pasp/Bilinear\_Transformation.html>

```

```

2045 //-----
2046 declare tf1s author "Julius O. Smith III";
2047 declare tf1s copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.stanford.
    edu>";
2048 declare tf1s license "MIT-style STK-4.3 license";
2049 tf1s(b1,b0,a0,w1) = tf1(b0d,b1d,a1d)
2050 with {
2051     c = 1/tan(w1*0.5/ma.SR); // bilinear-transform scale-factor
2052     d = a0 + c;
2053     b1d = (b0 - b1*c) / d;
2054     b0d = (b0 + b1*c) / d;
2055     a1d = (a0 - c) / d;
2056 };
2057
2058 //-----`fi.)tf2sb`-----
2059 // Bandpass mapping of `tf2s`: In addition to a frequency-scaling parameter
2060 // `w1` (set to HALF the desired passband width in rad/sec),
2061 // there is a desired center-frequency parameter wc (also in rad/s).
2062 // Thus, `tf2sb` implements a fourth-order digital bandpass filter section
2063 // specified by the coefficients of a second-order analog lowpass prototype
2064 // section. Such sections can be combined in series for higher orders.
2065 // The order of mappings is (1) frequency scaling (to set lowpass cutoff w1),
2066 // (2) bandpass mapping to wc, then (3) the bilinear transform, with the
2067 // usual scale parameter `2*SR`. Algebra carried out in maxima and pasted here.
2068 //
2069 // #### Usage
2070 //
2071 // ...
2072 // _ : tf2sb(b2,b1,b0,a1,a0,w1,wc) : _
2073 // ...
2074 //
2075 // Where:
2076 //
2077 // * `b2`, `b1`, `b0`: analog lowpass numerator coefficients
2078 // * `a1`, `a0`: analog lowpass denominator coefficients
2079 // * `w1`: half the desired passband width in radians/second
2080 // * `wc`: desired center frequency in radians/second
2081 //
2082 // #### Test
2083 // ...
2084 // fi = library("filters.lib");
2085 // os = library("oscillators.lib");
2086 // ma = library("maths.lib");
2087 // src = os.osc(440);
2088 // tf2sb_test = src : fi.tf2sb(0, 0, 1, sqrt(2), 1, 2*ma.PI*200, 2*ma.PI*1000);
2089 // ...
2090 //-----
2091 declare tf2sb author "Julius O. Smith III";
2092 declare tf2sb copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.stanford.
    edu>";
2093 declare tf2sb license "MIT-style STK-4.3 license";
2094 tf2sb(b2,b1,b0,a1,a0,w1,wc) =
2095     iir((b0d/a0d,b1d/a0d,b2d/a0d,b3d/a0d,b4d/a0d),(a1d/a0d,a2d/a0d,a3d/a0d,a4d/a0d)) with {
2096     T = 1.0/float(ma.SR);
2097     b0d = (4*b0*w1^2+8*b2*wc^2)*T^2+8*b1*w1*T+16*b2;
2098     b1d = 4*b2*wc^4*T^4+4*b1*wc^2*w1*T^3-16*b1*w1*T-64*b2;
2099     b2d = 6*b2*wc^4*T^4+(-8*b0*w1^2-16*b2*wc^2)*T^2+96*b2;
2100     b3d = 4*b2*wc^4*T^4-4*b1*wc^2*w1*T^3+16*b1*w1*T-64*b2;
2101     b4d = (b2*wc^4*T^4-2*b1*wc^2*w1*T^3+(4*b0*w1^2+8*b2*wc^2)*T^2-8*b1*w1*T+16*b2)
2102         + b2*wc^4*T^4+2*b1*wc^2*w1*T^3;
2103     a0d = wc^4*T^4+2*a1*wc^2*w1*T^3+(4*a0*w1^2+8*wc^2)*T^2+8*a1*w1*T+16;
2104     a1d = 4*wc^4*T^4+4*a1*wc^2*w1*T^3-16*a1*w1*T-64;
2105     a2d = 6*wc^4*T^4+(-8*a0*w1^2-16*wc^2)*T^2+96;
2106     a3d = 4*wc^4*T^4-4*a1*wc^2*w1*T^3+16*a1*w1*T-64;
2107     a4d = wc^4*T^4-2*a1*wc^2*w1*T^3+(4*a0*w1^2+8*wc^2)*T^2-8*a1*w1*T+16;
2108 };
2109
2110 //-----`fi.)tf1sb`-----

```



```

2111 // First-to-second-order lowpass-to-bandpass section mapping,
2112 // analogous to tf2sb above.
2113 //
2114 // #### Usage
2115 //
2116 // ...
2117 // _ : tf1sb(b1,b0,a0,w1,wc) : _
2118 // ...
2119 //
2120 // Where:
2121 //
2122 // * `b1`, `b0`: analog numerator coefficients
2123 // * `a0`: analog denominator constant coefficient
2124 // * `w1`: half the desired passband width in radians/second
2125 // * `wc`: desired center frequency in radians/second
2126 //
2127 // #### Test
2128 // ...
2129 // fi = library("filters.lib");
2130 // os = library("oscillators.lib");
2131 // ma = library("maths.lib");
2132 // src = os.osc(440);
2133 // tf1sb_test = src : fi.tf1sb(0, 1, 1, 2*ma.PI*200, 2*ma.PI*1000);
2134 // ...
2135 //-----
2136 declare tf1sb author "Julius O. Smith III";
2137 declare tf1sb copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.stanford.
    edu>";
2138 declare tf1sb license "MIT-style STK-4.3 license";
2139 tf1sb(b1,b0,a0,w1,wc) = tf2(b0d/a0d,b1d/a0d,b2d/a0d,a1d/a0d,a2d/a0d) with {
2140     T = 1.0/float(ma.SR);
2141     a0d = wc^2*T^2+2*a0*w1*T+4;
2142     b0d = b1*wc^2*T^2 +2*b0*w1*T+4*b1;
2143     b1d = 2*b1*wc^2*T^2-8*b1;
2144     b2d = b1*wc^2*T^2-2*b0*w1*T+4*b1;
2145     a1d = 2*wc^2*T^2-8;
2146     a2d = wc^2*T^2-2*a0*w1*T+4;
2147 };
2148
2149
2150 //=====Simple Resonator Filters=====
2151 //=====
2152
2153 //-----`(fi.)resonlp`-----
2154 // Simple resonant lowpass filter based on `tf2s` (virtual analog).
2155 // `resonlp` is a standard Faust function.
2156 //
2157 // #### Usage
2158 //
2159 // ...
2160 // _ : resonlp(fc,Q,gain) : _
2161 // _ : resonhp(fc,Q,gain) : _
2162 // _ : resonbp(fc,Q,gain) : _
2163 //
2164 // ...
2165 //
2166 // Where:
2167 //
2168 // * `fc`: center frequency (Hz)
2169 // * `Q`: q
2170 // * `gain`: gain (0-1)
2171 //
2172 // #### Test
2173 // ...
2174 // fi = library("filters.lib");
2175 // os = library("oscillators.lib");
2176 // src = os.osc(440);
2177 // resonlp_test = src : fi.resonlp(1000, 2, 0.8);

```

```

2178 //
2179 //-----
2180 // resonlp = 2nd-order lowpass with corner resonance:
2181 declare resonlp author "Julius O. Smith III";
2182 declare resonlp copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
    stanford.edu>";
2183 declare resonlp license "MIT-style STK-4.3 license";
2184
2185 resonlp(fc,Q,gain) = tf2s(b2,b1,b0,a1,a0,wc)
2186 with {
2187     wc = 2*ma.PI*fc;
2188     a1 = 1/Q;
2189     a0 = 1;
2190     b2 = 0;
2191     b1 = 0;
2192     b0 = gain;
2193 };
2194
2195 //-----`fi.`resonhp`-----
2196 // Simple resonant highpass filters based on `tf2s` (virtual analog).
2197 // `resonhp` is a standard Faust function.
2198 //
2199 //
2200 // #### Usage
2201 //
2202 //
2203 // _ : resonlp(fc,Q,gain) : _
2204 // _ : resonhp(fc,Q,gain) : _
2205 // _ : resonbp(fc,Q,gain) : _
2206 //
2207 //
2208 //
2209 // Where:
2210 //
2211 // * `fc`: center frequency (Hz)
2212 // * `Q`: q
2213 // * `gain`: gain (0-1)
2214 //
2215 // #### Test
2216 //
2217 // fi = library("filters.lib");
2218 // os = library("oscillators.lib");
2219 // src = os.osc(440);
2220 // resonhp_test = fi.resonhp(1000, 2, 0.8, src);
2221 //
2222 //-----
2223 // resonhp = 2nd-order highpass with corner resonance:
2224 declare resonhp author "Julius O. Smith III";
2225 declare resonhp copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
    stanford.edu>";
2226 declare resonhp license "MIT-style STK-4.3 license";
2227
2228 resonhp(fc,Q,gain,x) = gain*x-resonlp(fc,Q,gain,x);
2229
2230 //-----`fi.`resonbp`-----
2231 // Simple resonant bandpass filters based on `tf2s` (virtual analog).
2232 // `resonbp` is a standard Faust function.
2233 //
2234 //
2235 // #### Usage
2236 //
2237 //
2238 // _ : resonlp(fc,Q,gain) : _
2239 // _ : resonhp(fc,Q,gain) : _
2240 // _ : resonbp(fc,Q,gain) : _
2241 //
2242 //
2243 //

```

```

2244 // Where:
2245 //
2246 // * `fc`: center frequency (Hz)
2247 // * `Q`: q
2248 // * `gain`: gain (0-1)
2249 //
2250 // #### Test
2251 // ```
2252 // fi = library("filters.lib");
2253 // os = library("oscillators.lib");
2254 // src = os.osc(440);
2255 // resonbp_test = src : fi.resonbp(1000, 2, 0.8);
2256 // ```
2257 //-----
2258 // resonbp = 2nd-order bandpass
2259 declare resonbp author "Julius O. Smith III";
2260 declare resonbp copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
    stanford.edu>";
2261 declare resonbp license "MIT-style STK-4.3 license";
2262
2263 resonbp(fc,Q,gain) = tf2s(b2,b1,b0,a1,a0,wc)
2264 with {
2265     wc = 2*ma.PI*fc;
2266     a1 = 1/Q;
2267     a0 = 1;
2268     b2 = 0;
2269     b1 = gain;
2270     b0 = 0;
2271 };
2272
2273
2274 //=====Butterworth Lowpass/Highpass Filters=====
2275 //=====
2276
2277 //-----`(fi.)lowpass`-----
2278 // Nth-order Butterworth lowpass filter.
2279 // `lowpass` is a standard Faust function.
2280 //
2281 // #### Usage
2282 //
2283 // ```
2284 // _ : lowpass(N,fc) : _
2285 // ```
2286 //
2287 // Where:
2288 //
2289 // * `N`: filter order (number of poles), nonnegative constant numerical expression
2290 // * `fc`: desired cut-off frequency (-3dB frequency) in Hz
2291 //
2292 // #### Test
2293 // ```
2294 // fi = library("filters.lib");
2295 // os = library("oscillators.lib");
2296 // src = os.osc(440);
2297 // lowpass_test = src : fi.lowpass(4, 2000);
2298 // ```
2299 //
2300 // #### References
2301 //
2302 // * <https://ccrma.stanford.edu/~jos/filters/Butterworth\_Lowpass\_Design.html>
2303 // * `butter` function in Octave `("[z,p,g] = butter(N,1,'s');")`
2304 //-----
2305 declare lowpass author "Julius O. Smith III";
2306 declare lowpass copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
    stanford.edu>";
2307 declare lowpass license "MIT-style STK-4.3 license";
2308
2309 lowpass(N,fc) = lowpass0_highpass1(0,N,fc);

```

```

2310
2311
2312 //-----`(fi.)highpass`-----
2313 // Nth-order Butterworth highpass filter.
2314 // `highpass` is a standard Faust function.
2315 //
2316 // ### Usage
2317 //
2318 // ...
2319 // _ : highpass(N,fc) : _
2320 // ...
2321 //
2322 // Where:
2323 //
2324 // * `N`: filter order (number of poles), nonnegative constant numerical expression
2325 // * `fc`: desired cut-off frequency (-3dB frequency) in Hz
2326 //
2327 // ### Test
2328 // ...
2329 // fi = library("filters.lib");
2330 // os = library("oscillators.lib");
2331 // src = os.osc(440);
2332 // highpass_test = src : fi.highpass(4, 500);
2333 // ...
2334 //
2335 // ### References
2336 //
2337 // * <https://ccrma.stanford.edu/~jos/filters/Butterworth_Lowpass_Design.html>
2338 // * `butter` function in Octave `("[z,p,g] = butter(N,1,'s');")`
2339 //-----
2340 declare highpass author "Julius O. Smith III";
2341 declare highpass copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
    stanford.edu>";
2342 declare highpass license "MIT-style STK-4.3 license";
2343
2344 highpass(N,fc) = lowpass0_highpass1(1,N,fc);
2345
2346
2347 //-----`(fi.)lowpass0_highpass1`-----
2348 //
2349 // ### Test
2350 // ...
2351 // fi = library("filters.lib");
2352 // os = library("oscillators.lib");
2353 // src = os.osc(440);
2354 // lowpass0_highpass1_test = src : fi.lowpass0_highpass1(0, 2, 1000);
2355 // ...
2356 //-----
2357 declare lowpass0_highpass1 author "Julius O. Smith III";
2358 declare lowpass0_highpass1 "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
    stanford.edu>";
2359 declare lowpass0_highpass1 "MIT-style STK-4.3 license";
2360
2361 lowpass0_highpass1(s,N,fc) = lphpr(s,N,N,fc)
2362 with {
2363   lphpr(s,0,N,fc) = _;
2364   lphpr(s,1,N,fc) = tf1s(s,1-s,1,2*ma.PI*fc);
2365   lphpr(s,0,N,fc) = lphpr(s,(0-2),N,fc) : tf2s(s,0,1-s,a1s,1,w1) with {
2366     parity = N % 2;
2367     S = (0-parity)/2; // current section number
2368     a1s = -2*cos((ma.PI)*-1 + (1-parity)*ma.PI/(2*N) + (S-1+parity)*ma.PI/N);
2369     w1 = 2*ma.PI*fc;
2370   };
2371 };
2372
2373
2374 //=====Special Filter-Bank Delay-Equalizing Allpass Filters=====
2375 // These special allpass filters are needed by filterbank et al. below.

```

```

2376 // They are equivalent to (lowpass(N,fc) + highpass(N,fc))/2, but with
2377 // canceling pole-zero pairs removed (which occurs for odd N).
2378 //=====
2379
2380 //-----`fi.`highpass_plus_lowpass`-----
2381 //
2382 // #### Test
2383 // ...
2384 // fi = library("filters.lib");
2385 // os = library("oscillators.lib");
2386 // highpass_plus_lowpass_test = os.osc(440) : fi.highpass_plus_lowpass(3, 1000);
2387 // ...
2388 //-----
2389 declare highpass_plus_lowpass author "Julius O. Smith III";
2390 declare highpass_plus_lowpass copyright "Copyright (C) 2003-2019 by Julius O. Smith III <
      jos@ccrma.stanford.edu>";
2391 declare highpass_plus_lowpass license "MIT-style STK-4.3 license";
2392
2393 highpass_plus_lowpass(1,fc) = _;
2394 highpass_plus_lowpass(3,fc) = tf2s(1,-1,1,1,w1) with { w1 = 2*ma.PI*fc; };
2395 highpass_plus_lowpass(5,fc) = tf2s(1,-a11,1,a11,1,w1)
2396 with {
2397     a11 = 1.618033988749895;
2398     w1 = 2*ma.PI*fc;
2399 };
2400
2401 // Catch-all definitions for generality - even order is done:
2402 highpass_plus_lowpass(N,fc) = _ <: switch_odd_even(N%2,N,fc) with {
2403     switch_odd_even(0,N,fc) = highpass_plus_lowpass_even(N,fc);
2404     switch_odd_even(1,N,fc) = highpass_plus_lowpass_odd(N,fc);
2405 };
2406
2407 //-----`fi.`highpass_minus_lowpass`-----
2408 //
2409 // #### Test
2410 // ...
2411 // fi = library("filters.lib");
2412 // os = library("oscillators.lib");
2413 // highpass_minus_lowpass_test = os.osc(440) : fi.highpass_minus_lowpass(3, 1000);
2414 // ...
2415 //-----
2416 declare highpass_minus_lowpass author "Julius O. Smith III";
2417 declare highpass_minus_lowpass copyright "Copyright (C) 2003-2019 by Julius O. Smith III <
      jos@ccrma.stanford.edu>";
2418 declare highpass_minus_lowpass license "MIT-style STK-4.3 license";
2419 highpass_minus_lowpass(3,fc) = tf1s(-1,1,1,w1) with { w1 = 2*ma.PI*fc; };
2420 highpass_minus_lowpass(5,fc) = tf1s(1,-1,1,w1) : tf2s(1,-a12,1,a12,1,w1)
2421 with {
2422     a12 = 0.618033988749895;
2423     w1 = 2*ma.PI*fc;
2424 };
2425
2426 // Catch-all definitions for generality - even order is done:
2427 highpass_minus_lowpass(N,fc) = _ <: switch_odd_even(N%2,N,fc) with {
2428     switch_odd_even(0,N,fc) = highpass_minus_lowpass_even(N,fc);
2429     switch_odd_even(1,N,fc) = highpass_minus_lowpass_odd(N,fc);
2430 };
2431
2432 //-----`fi.`highpass_minus_lowpass_even`-----
2433 //
2434 // #### Test
2435 // ...
2436 // fi = library("filters.lib");
2437 // os = library("oscillators.lib");
2438 // highpass_plus_lowpass_even_test = os.osc(440), os.osc(440) : fi.highpass_plus_lowpass_even
      (4, 1000);
2439 // ...
2440 //-----

```

```

2441 declare highpass_plus_lowpass_even author "Julius O. Smith III";
2442 declare highpass_plus_lowpass_even copyright "Copyright (C) 2003-2019 by Julius O. Smith III
      <jos@ccrma.stanford.edu>";
2443 declare highpass_plus_lowpass_even license "MIT-style STK-4.3 license";
2444 highpass_plus_lowpass_even(N,fc) = highpass(N,fc) + lowpass(N,fc);
2445
2446 //-----`(fi.)highpass_plus_lowpass_even`-----
2447 //
2448 // #### Test
2449 // ...
2450 // fi = library("filters.lib");
2451 // os = library("oscillators.lib");
2452 // highpass_minus_lowpass_even_test = os.osc(440), os.osc(440) : fi.
      highpass_minus_lowpass_even(4, 1000);
2453 // ...
2454 //-----
2455 declare highpass_minus_lowpass_even author "Julius O. Smith III";
2456 declare highpass_minus_lowpass_even copyright "Copyright (C) 2003-2019 by Julius O. Smith
      III <jos@ccrma.stanford.edu>";
2457 declare highpass_minus_lowpass_even license "MIT-style STK-4.3 license";
2458 highpass_minus_lowpass_even(N,fc) = highpass(N,fc) - lowpass(N,fc);
2459
2460
2461 //-----`(fi.)highpass_plus_lowpass_odd`-----
2462 // Sum of the odd-order highpass and lowpass responses.
2463 //
2464 // #### Usage
2465 //
2466 // ...
2467 // _ : highpass_plus_lowpass_odd(N,fc) : _
2468 // ...
2469 //
2470 // Where:
2471 //
2472 // * `N`: odd Butterworth order
2473 // * `fc`: cutoff frequency in Hz
2474 //
2475 // #### Test
2476 // ...
2477 // fi = library("filters.lib");
2478 // os = library("oscillators.lib");
2479 // highpass_plus_lowpass_odd_test = os.osc(440), os.osc(440) : fi.highpass_plus_lowpass_odd
      (3, 1000);
2480 // ...
2481 // FLXME: Rewrite the following, as for orders 3 and 5 above,
2482 // to eliminate pole-zero cancellations:
2483 //-----
2484 declare highpass_plus_lowpass_odd author "Julius O. Smith III";
2485 declare highpass_plus_lowpass_odd copyright "Copyright (C) 2003-2019 by Julius O. Smith III
      <jos@ccrma.stanford.edu>";
2486 declare highpass_plus_lowpass_odd license "MIT-style STK-4.3 license";
2487 highpass_plus_lowpass_odd(N,fc) = highpass(N,fc) + lowpass(N,fc);
2488
2489 //-----`(fi.)highpass_minus_lowpass_odd`-----
2490 // Difference between the odd-order highpass and lowpass responses.
2491 //
2492 // #### Usage
2493 //
2494 // ...
2495 // _ : highpass_minus_lowpass_odd(N,fc) : _
2496 // ...
2497 //
2498 // Where:
2499 //
2500 // * `N`: odd Butterworth order
2501 // * `fc`: cutoff frequency in Hz
2502 //
2503 // #### Test

```

```

2504 // ```
2505 // fi = library("filters.lib");
2506 // os = library("oscillators.lib");
2507 // highpass_minus_lowpass_odd_test = os.osc(440), os.osc(440) : fi.highpass_minus_lowpass_odd
      (3, 1000);
2508 // ```
2509 // FIXME: Rewrite the following, as for orders 3 and 5 above,
2510 // to eliminate pole-zero cancellations/
2511 //-----
2512 highpass_minus_lowpass_odd(N,fc) = highpass(N,fc) - lowpass(N,fc);
2513 declare highpass_minus_lowpass_odd author "Julius O. Smith III";
2514 declare highpass_minus_lowpass_odd copyright "Copyright (C) 2003-2019 by Julius O. Smith III
      <jos@ccrma.stanford.edu>";
2515 declare highpass_plus_lowpass_odd license "MIT-style STK-4.3 license";
2516
2517
2518 //=====Elliptic (Cauer) Lowpass Filters=====
2519 // Elliptic (Cauer) Lowpass Filters
2520 //
2521 // #### References
2522 //
2523 // * <http://en.wikipedia.org/wiki/Elliptic_filter>
2524 // * functions `ncauer` and `ellip` in Octave.
2525 //=====
2526
2527 //-----`fi().lowpass3e`-----
2528 // Third-order Elliptic (Cauer) lowpass filter.
2529 //
2530 // #### Usage
2531 //
2532 // ```
2533 // _ : lowpass3e(fc) : _
2534 // ```
2535 //
2536 // Where:
2537 //
2538 // * `fc`: -3dB frequency in Hz
2539 //
2540 // #### Test
2541 // ```
2542 // fi = library("filters.lib");
2543 // os = library("oscillators.lib");
2544 // src = os.osc(440);
2545 // lowpass3e_test = src : fi.lowpass3e(1000);
2546 // ```
2547 //
2548 // #### Design
2549 //
2550 // For spectral band-slice level display (see `octave_analyzer3e`):
2551 // ```
2552 //
2553 // [z,p,g] = ncauer(Rp,Rs,3); % analog zeros, poles, and gain, where
2554 // Rp = 60 % dB ripple in stopband
2555 // Rs = 0.2 % dB ripple in passband
2556 // ```
2557 //-----
2558 declare lowpass3e author "Julius O. Smith III";
2559 declare lowpass3e copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
      stanford.edu>";
2560 declare lowpass3e license "MIT-style STK-4.3 license";
2561
2562 lowpass3e(fc) = tf2s(b21,b11,b01,a11,a01,w1) : tf1s(0,1,a02,w1)
2563 with {
2564     a11 = 0.802636764161030; // format long; poly(p(1:2)) % in octave
2565     a01 = 1.412270893774204;
2566     a02 = 0.822445908998816; // poly(p(3)) % in octave
2567     b21 = 0.019809144837789; // poly(z)
2568     b11 = 0;

```

```

2569     b01 = 1.161516418982696;
2570     w1 = 2*ma.PI*fc;
2571 };
2572
2573 //-----`(fi.)lowpass6e`-----
2574 // Sixth-order Elliptic/Cauer lowpass filter.
2575 //
2576 // #### Usage
2577 //
2578 // ```
2579 // _ : lowpass6e(fc) : _
2580 // ```
2581 //
2582 // Where:
2583 //
2584 // * `fc`: -3dB frequency in Hz
2585 //
2586 // #### Test
2587 // ```
2588 // fi = library("filters.lib");
2589 // os = library("oscillators.lib");
2590 // src = os.osc(440);
2591 // lowpass6e_test = src : fi.lowpass6e(1000);
2592 // ```
2593 //
2594 // #### Design
2595 //
2596 // For spectral band-slice level display (see octave_analyzer6e):
2597 //
2598 // ```
2599 // [z,p,g] = ncauer(Rp,Rs,6); % analog zeros, poles, and gain, where
2600 // Rp = 80 % dB ripple in stopband
2601 // Rs = 0.2 % dB ripple in passband
2602 // ```
2603 //-----
2604 declare lowpass6e author "Julius O. Smith III";
2605 declare lowpass6e copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
    stanford.edu>";
2606 declare lowpass6e license "MIT-style STK-4.3 license";
2607
2608 lowpass6e(fc) =
2609     tf2s(b21,b11,b01,a11,a01,w1) :
2610     tf2s(b22,b12,b02,a12,a02,w1) :
2611     tf2s(b23,b13,b03,a13,a03,w1)
2612 with {
2613     b21 = 0.000099999997055;
2614     a21 = 1;
2615     b11 = 0;
2616     a11 = 0.782413046821645;
2617     b01 = 0.000433227200555;
2618     a01 = 0.245291508706160;
2619     b22 = 1;
2620     a22 = 1;
2621     b12 = 0;
2622     a12 = 0.512478641889141;
2623     b02 = 7.621731298870603;
2624     a02 = 0.689621364484675;
2625     b23 = 1;
2626     a23 = 1;
2627     b13 = 0;
2628     a13 = 0.168404871113589;
2629     b03 = 53.536152954556727;
2630     a03 = 1.069358407707312;
2631     w1 = 2*ma.PI*fc;
2632 };
2633
2634
2635 //=====Elliptic Highpass Filters=====

```



```

2636 //=====
2637
2638 //-----`(fi.)highpass3e`-----
2639 // Third-order Elliptic (Cauer) highpass filter. Inversion of `lowpass3e` wrt unit
2640 // circle in s plane (s <- 1/s).
2641 //
2642 // #### Usage
2643 //
2644 // ...
2645 // _ : highpass3e(fc) : _
2646 // ...
2647 //
2648 // Where:
2649 //
2650 // * `fc`: -3dB frequency in Hz
2651 //
2652 // #### Test
2653 // ...
2654 // fi = library("filters.lib");
2655 // os = library("oscillators.lib");
2656 // src = os.osc(440);
2657 // highpass3e_test = src : fi.highpass3e(1000);
2658 // ...
2659 //-----
2660 declare highpass3e author "Julius O. Smith III";
2661 declare highpass3e copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
    stanford.edu>";
2662 declare highpass3e license "MIT-style STK-4.3 license";
2663 highpass3e(fc) = tf2s(b01/a01,b11/a01,b21/a01,a11/a01,1/a01,w1) :
2664     tf1s(1/a02,0,1/a02,w1)
2665 with {
2666     a11 = 0.802636764161030;
2667     a01 = 1.412270893774204;
2668     a02 = 0.822445908998816;
2669     b21 = 0.019809144837789;
2670     b11 = 0;
2671     b01 = 1.161516418982696;
2672     w1 = 2*ma.PI*fc;
2673 };
2674
2675 //-----`(fi.)highpass6e`-----
2676 // Sixth-order Elliptic/Cauer highpass filter. Inversion of `lowpass3e` wrt unit
2677 // circle in s plane (s <- 1/s).
2678 //
2679 // #### Usage
2680 //
2681 // ...
2682 // _ : highpass6e(fc) : _
2683 // ...
2684 //
2685 // Where:
2686 //
2687 // * `fc`: -3dB frequency in Hz
2688 //
2689 // #### Test
2690 // ...
2691 // fi = library("filters.lib");
2692 // os = library("oscillators.lib");
2693 // src = os.osc(440);
2694 // highpass6e_test = src : fi.highpass6e(1000);
2695 // ...
2696 //-----
2697 declare highpass6e author "Julius O. Smith III";
2698 declare highpass6e copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
    stanford.edu>";
2699 declare highpass6e license "MIT-style STK-4.3 license";
2700 highpass6e(fc) =
2701     tf2s(b01/a01,b11/a01,b21/a01,a11/a01,1/a01,w1) :

```

```

2702         tf2s(b02/a02,b12/a02,b22/a02,a12/a02,1/a02,w1) :
2703         tf2s(b03/a03,b13/a03,b23/a03,a13/a03,1/a03,w1)
2704     with {
2705         b21 = 0.000099999997055;
2706         a21 = 1;
2707         b11 = 0;
2708         a11 = 0.782413046821645;
2709         b01 = 0.000433227200555;
2710         a01 = 0.245291508706160;
2711         b22 = 1;
2712         a22 = 1;
2713         b12 = 0;
2714         a12 = 0.512478641889141;
2715         b02 = 7.621731298870603;
2716         a02 = 0.689621364484675;
2717         b23 = 1;
2718         a23 = 1;
2719         b13 = 0;
2720         a13 = 0.168404871113589;
2721         b03 = 53.536152954556727;
2722         a03 = 1.069358407707312;
2723         w1 = 2*ma.PI*fc;
2724     };
2725
2726
2727     //=====Butterworth Bandpass/Bandstop Filters=====
2728     //=====
2729
2730     //-----`(fi.)bandpass`-----
2731     // Order 2*Nh Butterworth bandpass filter made using the transformation
2732     // `s <- s + wc^2/s` on `lowpass(Nh)`, where `wc` is the desired bandpass center
2733     // frequency. The `lowpass(Nh)` cutoff `w1` is half the desired bandpass width.
2734     // `bandpass` is a standard Faust function.
2735     //
2736     // #### Usage
2737     //
2738     // ```
2739     // _ : bandpass(Nh,fl,fu) : _
2740     // ```
2741     //
2742     // Where:
2743     //
2744     // * `Nh`: HALF the desired bandpass order (which is therefore even)
2745     // * `fl`: lower -3dB frequency in Hz
2746     // * `fu`: upper -3dB frequency in Hz
2747     // Thus, the passband width is `fu-fl`,
2748     // and its center frequency is `(fl+fu)/2`.
2749     //
2750     // #### Test
2751     // ```
2752     // fi = library("filters.lib");
2753     // os = library("oscillators.lib");
2754     // src = os.osc(440);
2755     // bandpass_test = src : fi.bandpass(2, 500, 1500);
2756     // ```
2757     //-----
2758     declare bandpass author "Julius O. Smith III";
2759     declare bandpass copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
2760         stanford.edu>";
2761     declare bandpass license "MIT-style STK-4.3 license";
2762     bandpass(Nh,fl,fu) = bandpass0_bandstop1(0,Nh,fl,fu);
2763
2764     //-----`(fi.)bandpass0_bandstop1`-----
2765     // Order 2*Nh Butterworth bandstop filter made using the transformation
2766     // `s <- s + wc^2/s` on `highpass(Nh)`, where `wc` is the desired bandpass center
2767     // frequency. The `highpass(Nh)` cutoff `w1` is half the desired bandpass width.
2768     // `bandstop` is a standard Faust function.

```

```

2769 //
2770 // ##### Usage
2771 //
2772 // ...
2773 // _ : bandstop(Nh,fl,fu) : _
2774 // ...
2775 // Where:
2776 //
2777 // * `Nh`: HALF the desired bandstop order (which is therefore even)
2778 // * `fl`: lower -3dB frequency in Hz
2779 // * `fu`: upper -3dB frequency in Hz
2780 // Thus, the passband (stopband) width is `fu-fl`,
2781 // and its center frequency is `(fl+fu)/2`.
2782 //
2783 // ##### Test
2784 // ...
2785 // fi = library("filters.lib");
2786 // os = library("oscillators.lib");
2787 // src = os.osc(440);
2788 // bandstop_test = src : fi.bandstop(2, 500, 1500);
2789 // ...
2790 //-----
2791 declare bandstop author "Julius O. Smith III";
2792 declare bandstop copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
    stanford.edu>";
2793 declare bandstop license "MIT-style STK-4.3 license";
2794 bandstop(Nh,fl,fu) = bandpass0_bandstop1(1,Nh,fl,fu);
2795
2796 //-----`(fi.)bandstop`-----
2797 //
2798 // ##### Test
2799 // ...
2800 // fi = library("filters.lib");
2801 // os = library("oscillators.lib");
2802 // src = os.osc(440);
2803 // bandpass0_bandstop1_test = src : fi.bandpass0_bandstop1(0, 2, 500, 1500);
2804 // ...
2805 //-----
2806 declare bandpass0_bandstop1 author "Julius O. Smith III";
2807 declare bandpass0_bandstop1 copyright "Copyright (C) 2003-2019 by Julius O. Smith III <
    jos@ccrma.stanford.edu>";
2808 declare bandpass0_bandstop1 license "MIT-style STK-4.3 license";
2809 bandpass0_bandstop1(s,Nh,fl,fu) = bpbsr(s,Nh,Nh,fl,fu)
2810 with {
2811     wl = 2*ma.PI*fl; // digital (z-plane) lower passband edge
2812     wu = 2*ma.PI*fu; // digital (z-plane) upper passband edge
2813
2814     c = 2.0*ma.SR; // bilinear transform scaling used in tf2sb, tf1sb
2815     wla = c*tan(wl/c); // analog (s-plane) lower cutoff
2816     wua = c*tan(wu/c); // analog (s-plane) upper cutoff
2817
2818     wc = sqrt(wla*wua); // s-plane center frequency
2819     w1 = wua - wc^2/wua; // s-plane lowpass prototype cutoff
2820
2821     bpbsr(s,0,Nh,fl,fu) = _;
2822     bpbsr(s,1,Nh,fl,fu) = tf1sb(s,1-s,1,w1,wc);
2823     bpbsr(s,0,Nh,fl,fu) = bpbsr(s,0-2,Nh,fl,fu) : tf2sb(s,0,(1-s),a1s,1,w1,wc)
2824     with {
2825         parity = Nh % 2;
2826         S = (0-parity)/2; // current section number
2827         a1s = -2*cos(-1*ma.PI + (1-parity)*ma.PI/(2*Nh) + (S-1+parity)*ma.PI/Nh);
2828     };
2829 };
2830
2831
2832 //=====Elliptic Bandpass Filters=====
2833 //=====
2834

```

```

2835 //-----`(fi.)bandpass6e`-----
2836 // Order 12 elliptic bandpass filter analogous to `bandpass(6)`.
2837 //-----
2838 declare bandpass6e author "Julius O. Smith III";
2839 declare bandpass6e copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
    stanford.edu>";
2840 declare bandpass6e license "MIT-style STK-4.3 license";
2841 bandpass6e(fl,fu) = tf2sb(b21,b11,b01,a11,a01,w1,wc) : tf1sb(0,1,a02,w1,wc)
2842 with {
2843     a11 = 0.802636764161030; // In octave: format long; poly(p(1:2))
2844     a01 = 1.412270893774204;
2845     a02 = 0.822445908998816; // poly(p(3))
2846     b21 = 0.019809144837789; // poly(z)
2847     b11 = 0;
2848     b01 = 1.161516418982696;
2849
2850     w1 = 2*ma.PI*fl; // digital (z-plane) lower passband edge
2851     wu = 2*ma.PI*fu; // digital (z-plane) upper passband edge
2852
2853     c = 2.0*ma.SR; // bilinear transform scaling used in tf2sb, tf1sb
2854     wla = c*tan(w1/c); // analog (s-plane) lower cutoff
2855     wua = c*tan(wu/c); // analog (s-plane) upper cutoff
2856
2857     wc = sqrt(wla*wua); // s-plane center frequency
2858     w1 = wua - wc^2/wua; // s-plane lowpass cutoff
2859 };
2860
2861 //-----`(fi.)bandpass12e`-----
2862 // Order 24 elliptic bandpass filter analogous to `bandpass(6)`.
2863 //-----
2864 declare bandpass12e author "Julius O. Smith III";
2865 declare bandpass12e copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
    stanford.edu>";
2866 declare bandpass12e license "MIT-style STK-4.3 license";
2867 bandpass12e(fl,fu) =
2868     tf2sb(b21,b11,b01,a11,a01,w1,wc) :
2869     tf2sb(b22,b12,b02,a12,a02,w1,wc) :
2870     tf2sb(b23,b13,b03,a13,a03,w1,wc)
2871 with { // octave script output:
2872     b21 = 0.000099999997055;
2873     a21 = 1;
2874     b11 = 0;
2875     a11 = 0.782413046821645;
2876     b01 = 0.000433227200555;
2877     a01 = 0.245291508706160;
2878     b22 = 1;
2879     a22 = 1;
2880     b12 = 0;
2881     a12 = 0.512478641889141;
2882     b02 = 7.621731298870603;
2883     a02 = 0.689621364484675;
2884     b23 = 1;
2885     a23 = 1;
2886     b13 = 0;
2887     a13 = 0.168404871113589;
2888     b03 = 53.536152954556727;
2889     a03 = 1.069358407707312;
2890
2891     w1 = 2*ma.PI*fl; // digital (z-plane) lower passband edge
2892     wu = 2*ma.PI*fu; // digital (z-plane) upper passband edge
2893
2894     c = 2.0*ma.SR; // bilinear transform scaling used in tf2sb, tf1sb
2895     wla = c*tan(w1/c); // analog (s-plane) lower cutoff
2896     wua = c*tan(wu/c); // analog (s-plane) upper cutoff
2897
2898     wc = sqrt(wla*wua); // s-plane center frequency
2899     w1 = wua - wc^2/wua; // s-plane lowpass cutoff
2900 };

```

```

2901 //-----`fi.)pospass`-----
2902 // Positive-Pass Filter (single-side-band filter).
2903 //
2904 // ### Usage
2905 //
2906 // ...
2907 // _ : pospass(N,fc) : _,_
2908 // ...
2909 //
2910 // where
2911 //
2912 // * `N`: filter order (Butterworth bandpass for positive frequencies).
2913 // * `fc`: lower bandpass cutoff frequency in Hz.
2914 // - Highpass cutoff frequency at  $\text{ma.SR}/2 - \text{fc}$  Hz.
2915 //
2916 // ### Example test program
2917 //
2918 // * See `dm.pospass_demo`
2919 // * Look at frequency response
2920 //
2921 //
2922 // ### Method
2923 //
2924 // A filter passing only positive frequencies can be made from a
2925 // half-band lowpass by modulating it up to the positive-frequency range.
2926 // Equivalently, down-modulate the input signal using a complex sinusoid at  $-\text{SR}/4$  Hz,
2927 // lowpass it with a half-band filter, and modulate back up by  $\text{SR}/4$  Hz.
2928 // In Faust/math notation:
2929 //  $\text{\$pospass}(N) = \text{\texttt{ast}(e^{-j\frac{\pi}{2}n})} : \text{\texttt{mbox{lowpass}(N,SR/4)}} : \text{\texttt{ast}(e^{j\frac{\pi}{2}n})}$ 
2930 //
2931 // An approximation to the Hilbert transform is given by the
2932 // imaginary output signal:
2933 //
2934 // ...
2935 // hilbert(N) = pospass(N) : !,*(2);
2936 // ...
2937 //
2938 // ### References
2939 //
2940 // * <https://ccrma.stanford.edu/~jos/mdft/Analytic\_Signals\_Hilbert\_Transform.html>
2941 // * <https://ccrma.stanford.edu/~jos/sasp/Comparison\_Optimal\_Chebyshev\_FIR\_I.html>
2942 // * <https://ccrma.stanford.edu/~jos/sasp/Hilbert\_Transform.html>
2943 //-----
2944 declare pospass author "Julius O. Smith III";
2945 declare pospass copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.stanford.edu>";
2946 declare pospass license "MIT-style STK-4.3 license";
2947 pospass(N,fc) = fi.pospass0(lpf) with {
2948   fcs = ma.SR/4 - fc; // Upper lowpass cutoff = (SR/2 - fc) - SR/4
2949   lpf = fi.lowpass(N,fcs); // Butterworth lowpass
2950 };
2951
2952 declare pospass6e author "Julius O. Smith III";
2953 declare pospass6e copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.stanford.edu>";
2954 declare pospass6e license "MIT-style STK-4.3 license";
2955 pospass6e(fc) = fi.pospass0(lpf) with {
2956   lpf = fi.lowpass6e(ma.SR/4 - fc); // Elliptic lowpass, order 6
2957 };
2958
2959 declare pospass0 author "Julius O. Smith III";
2960 declare pospass0 copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.stanford.edu>";
2961 declare pospass0 license "MIT-style STK-4.3 license";
2962 pospass0(lpf) = unmodulate : lpf, lpf : modulate with {
2963   c = 1-1' : +*(-1):mem); // complex sinusoid rotating at SR/4
2964   s = c'; // ||: 0, 1, 0, -1 :||

```

```

2965     unmodulate = _ <: *(c),*(-s); // subtract SR/4 from all input frequencies
2966     modulate(x,y) = c*x-s*y, c*y + s*x; // add SR/4 to all frequencies
2967 };
2968
2969
2970 //=====Parametric Equalizers (Shelf, Peaking)=====
2971 // Parametric Equalizers (Shelf, Peaking).
2972 //
2973 // #### References
2974 //
2975 // * <http://en.wikipedia.org/wiki/Equalization>
2976 // * <https://webaudio.github.io/Audio-EQ-Cookbook/Audio-EQ-Cookbook.txt>
2977 // * Digital Audio Signal Processing, Udo Zolzer, Wiley, 1999, p. 124
2978 // * <https://ccrma.stanford.edu/~jos/filters/Low_High_Shelving_Filters.html>
2979 // * <https://ccrma.stanford.edu/~jos/filters/Peaking_Equalizers.html>
2980 // * maxmsp.lib in the Faust distribution
2981 // * bandfilter.dsp in the faust2pd distribution
2982 //=====
2983
2984 //-----`fi.`lowshelf`-----
2985 // First-order "low shelf" filter (gain boost/cut between dc and some frequency)
2986 // `low_shelf` is a standard Faust function.
2987 //
2988 // #### Usage
2989 //
2990 // ```
2991 // _ : lowshelf(N,L0,fx) : _
2992 // _ : low_shelf(L0,fx) : _ // default case (order 3)
2993 // _ : lowshelf_other_freq(N,L0,fx) : _
2994 // ```
2995 //
2996 // Where:
2997 //
2998 // * `N`: filter order 1, 3, 5, ... (odd only, default should be 3, a constant numerical
2999 //   expression)
3000 // * `L0`: desired level (dB) between dc and fx (boost `L0>0` or cut `L0<0`)
3001 // * `fx`: -3dB frequency of lowpass band (`L0>0`) or upper band (`L0<0`)
3002 //   (see "SHELF SHAPE" below).
3003 //
3004 // The gain at SR/2 is constrained to be 1.
3005 // The generalization to arbitrary odd orders is based on the well known
3006 // fact that odd-order Butterworth band-splits are allpass-complementary
3007 // (see filterbank documentation below for references).
3008 //
3009 // #### Test
3010 // ```
3011 // fi = library("filters.lib");
3012 // os = library("oscillators.lib");
3013 // src = os.osc(440);
3014 // lowshelf_test = src : fi.lowshelf(3, 6, 500);
3015 // ```
3016 //
3017 // #### Shelf Shape
3018 // The magnitude frequency response is approximately piecewise-linear
3019 // on a log-log plot ("BODE PLOT"). The Bode "stick diagram" approximation
3020 // L(lf) is easy to state in dB versus dB-frequency lf = dB(f):
3021 //
3022 // * L0 > 0:
3023 // * L(lf) = L0, f between 0 and fx = 1st corner frequency;
3024 // * L(lf) = L0 - N * (lf - lfx), f between fx and f2 = 2nd corner frequency;
3025 // * L(lf) = 0, lf > lf2.
3026 // * lf2 = lfx + L0/N = dB-frequency at which level gets back to 0 dB.
3027 // * L0 < 0:
3028 // * L(lf) = L0, f between 0 and f1 = 1st corner frequency;
3029 // * L(lf) = - N * (lfx - lf), f between f1 and lfx = 2nd corner frequency;
3030 // * L(lf) = 0, lf > lfx.
3031 // * lf1 = lfx + L0/N = dB-frequency at which level goes up from L0.
3032 //

```

```

3032 // See `lowshelf_other_freq`.
3033 //
3034 // #### References
3035 //
3036 // * See "Parametric Equalizers" above for references regarding `low_shelf`, `high_shelf`,
3037 //   and `peak_eq`.
3038 //-----
3039 declare lowshelf author "Julius O. Smith III";
3040 declare lowshelf copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
3041 stanford.edu>";
3042 declare lowshelf license "MIT-style STK-4.3 license";
3043 lowshelf(N,L0,fx) = filterbank(N,(fx)) : _, *(ba.db2linear(L0)) :> _;
3044
3045 // Special cases and optimization:
3046 //-----
3047 // (fi.)low_shelf`-----
3048 // #### Test
3049 // ...
3050 // fi = library("filters.lib");
3051 // os = library("oscillators.lib");
3052 // src = os.osc(440);
3053 // low_shelf_test = src : fi.low_shelf(6, 500);
3054 // ...
3055 //-----
3056 declare low_shelf author "Julius O. Smith III";
3057 declare low_shelf copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
3058 stanford.edu>";
3059 declare low_shelf license "MIT-style STK-4.3 license";
3060 low_shelf = lowshelf(3); // default = 3rd order Butterworth
3061
3062 //-----
3063 // (fi.)low_shelf1`-----
3064 // #### Test
3065 // ...
3066 // fi = library("filters.lib");
3067 // os = library("oscillators.lib");
3068 // src = os.osc(440);
3069 // low_shelf1_test = fi.low_shelf1(2, 500, src);
3070 // ...
3071 //-----
3072 declare low_shelf1 author "Julius O. Smith III";
3073 declare low_shelf1 copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
3074 stanford.edu>";
3075 declare low_shelf1 license "MIT-style STK-4.3 license";
3076 low_shelf1(L0,fx,x) = x + (ba.db2linear(L0)-1)*lowpass(1,fx,x); // optimized
3077
3078 //-----
3079 // (fi.)low_shelf1_l`-----
3080 // #### Test
3081 // ...
3082 // fi = library("filters.lib");
3083 // os = library("oscillators.lib");
3084 // src = os.osc(440);
3085 // low_shelf1_l_test = fi.low_shelf1_l(2, 500, src);
3086 // ...
3087 //-----
3088 declare low_shelf1_l author "Julius O. Smith III";
3089 declare low_shelf1_l copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
3090 stanford.edu>";
3091 declare low_shelf1_l license "MIT-style STK-4.3 license";
3092 low_shelf1_l(G0,fx,x) = x + (G0-1)*lowpass(1,fx,x); // optimized
3093
3094 //-----
3095 // (fi.)lowshelf_other_freq`-----
3096 // #### Test
3097 // ...
3098 // fi = library("filters.lib");
3099 // lowshelf_other_freq_test = fi.lowshelf_other_freq(3, 6, 500);

```

```

3095 // ---
3096 //-----
3097 declare lowshelf_other_freq author "Julius O. Smith III";
3098 declare lowshelf_other_freq copyright "Copyright (C) 2003-2019 by Julius O. Smith III <
      jos@ccrma.stanford.edu>";
3099 declare lowshelf_other_freq license "MIT-style STK-4.3 license";
3100 lowshelf_other_freq(N, L0, fx) = ba.db2linear(ba.linear2db(fx) + L0/N); // convenience
3101
3102 //-----`fi.)highshelf`-----
3103 // First-order "high shelf" filter (gain boost/cut above some frequency).
3104 // `high_shelf` is a standard Faust function.
3105 //
3106 // #### Usage
3107 //
3108 // ---
3109 // _ : highshelf(N,Lpi,fx) : _
3110 // _ : high_shelf(L0,fx) : _ // default case (order 3)
3111 // _ : highshelf_other_freq(N,Lpi,fx) : _
3112 // ---
3113 //
3114 // Where:
3115 //
3116 // * `N`: filter order 1, 3, 5, ... (odd only, a constant numerical expression).
3117 // * `Lpi`: desired level (dB) between fx and SR/2 (boost Lpi>0 or cut Lpi<0)
3118 // * `fx`: -3dB frequency of highpass band (L0>0) or lower band (L0<0)
3119 //         (Use highshelf_other_freq() below to find the other one.)
3120 //
3121 // The gain at dc is constrained to be 1.
3122 // See `lowshelf` documentation above for more details on shelf shape.
3123 //
3124 // #### Test
3125 // ---
3126 // fi = library("filters.lib");
3127 // os = library("oscillators.lib");
3128 // src = os.osc(440);
3129 // highshelf_test = src : fi.highshelf(3, 6, 2000);
3130 // ---
3131 //
3132 // #### References
3133 //
3134 // * See "Parametric Equalizers" above for references regarding `low_shelf`, `high_shelf`,
      and `peak_eq`.
3135 //-----
3136 declare highshelf author "Julius O. Smith III";
3137 declare highshelf copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
      stanford.edu>";
3138 declare highshelf license "MIT-style STK-4.3 license";
3139 highshelf(N,Lpi,fx) = filterbank(N,(fx)) : *(ba.db2linear(Lpi)), _ :> _;
3140
3141 // Special cases and optimization:
3142
3143 //-----`fi.)high_shelf`-----
3144 // #### Test
3145 // ---
3146 // fi = library("filters.lib");
3147 // os = library("oscillators.lib");
3148 // src = os.osc(440);
3149 // high_shelf_test = src : fi.high_shelf(6, 2000);
3150 // ---
3151 //-----
3152 high_shelf = highshelf(3); // default = 3rd order Butterworth
3153
3154 //-----`fi.)high_shelf1`-----
3155 // #### Test
3156 // ---
3157 // fi = library("filters.lib");
3158 // os = library("oscillators.lib");
3159 // src = os.osc(440);

```



```

3160 // high_shelf1_test = fi.high_shelf1(6, 2000, src);
3161 // ...
3162 //-----
3163 declare high_shelf1 author "Julius O. Smith III";
3164 declare high_shelf1 copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
stanford.edu>";
3165 declare high_shelf1 license "MIT-style STK-4.3 license";
3166 high_shelf1(Lpi,fx,x) = x + (ba.db2linear(Lpi)-1)*highpass(1,fx,x); // optimized
3167
3168 //-----`fi.)high_shelf1_l`-----
3169 // #### Test
3170 // ...
3171 // fi = library("filters.lib");
3172 // os = library("oscillators.lib");
3173 // src = os.osc(440);
3174 // high_shelf1_l_test = fi.high_shelf1_l(2, 2000, src);
3175 // ...
3176 //-----
3177 declare high_shelf1_l author "Julius O. Smith III";
3178 declare high_shelf1_l copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
stanford.edu>";
3179 declare high_shelf1_l license "MIT-style STK-4.3 license";
3180 high_shelf1_l(Gpi,fx,x) = x + (Gpi-1)*highpass(1,fx,x); //optimized
3181
3182 // shelf transitions between frequency fx and this one:
3183
3184 //-----`fi.)highshelf_other_freq`-----
3185 // #### Test
3186 // ...
3187 // fi = library("filters.lib");
3188 // highshelf_other_freq_test = fi.highshelf_other_freq(3, 6, 2000);
3189 // ...
3190 //-----
3191 declare highshelf_other_freq author "Julius O. Smith III";
3192 declare highshelf_other_freq copyright "Copyright (C) 2003-2019 by Julius O. Smith III <
jos@ccrma.stanford.edu>";
3193 declare highshelf_other_freq license "MIT-style STK-4.3 license";
3194 highshelf_other_freq(N, Lpi, fx) = ba.db2linear(ba.linear2db(fx) - Lpi/N);
3195
3196
3197 //-----`fi.)peak_eq`-----
3198 // Second order "peaking equalizer" section (gain boost or cut near some frequency)
3199 // Also called a "parametric equalizer" section.
3200 // `peak_eq` is a standard Faust function.
3201 //
3202 // #### Usage
3203 //
3204 // ...
3205 // _ : peak_eq(Lfx,fx,B) : _
3206 // ...
3207 //
3208 // Where:
3209 //
3210 // * `Lfx`: level (dB) at fx (boost Lfx>0 or cut Lfx<0)
3211 // * `fx`: peak frequency (Hz)
3212 // * `B`: bandwidth (B) of peak in Hz
3213 //
3214 // #### Test
3215 // ...
3216 // fi = library("filters.lib");
3217 // os = library("oscillators.lib");
3218 // src = os.osc(440);
3219 // peak_eq_test = src : fi.peak_eq(6, 1000, 200);
3220 // ...
3221 //
3222 // #### References
3223 //
3224 // * See "Parametric Equalizers" above for references regarding `low_shelf`, `high_shelf`,

```

```

3225         and `peak_eq`.
3226 //-----
3227 declare peak_eq author "Julius O. Smith III";
3228 declare peak_eq copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
3229     stanford.edu>";
3230 declare peak_eq license "MIT-style STK-4.3 license";
3231 peak_eq(Lfx,fx,B) = tf2s(1,b1s,1,a1s,1,wx) with {
3232     T = float(1.0/ma.SR);
3233     Bw = B*T/sin(wx*T); // prewarp s-bandwidth for more accuracy in z-plane
3234     a1 = ma.PI*Bw;
3235     b1 = g*a1;
3236     g = ba.db2linear(abs(Lfx));
3237     b1s = select2(Lfx>0,a1,b1); // When Lfx>0, pole dominates bandwidth
3238     a1s = select2(Lfx>0,b1,a1); // When Lfx<0, zero dominates
3239     wx = 2*ma.PI*fx;
3240 };
3241 //-----`(fi.)peak_eq_cq`-----
3242 // Constant-Q second order peaking equalizer section.
3243 //
3244 // #### Usage
3245 //
3246 // _ : peak_eq_cq(Lfx,fx,Q) : _
3247 //
3248 //
3249 // Where:
3250 //
3251 // * `Lfx`: level (dB) at fx
3252 // * `fx`: boost or cut frequency (Hz)
3253 // * `Q`: "Quality factor" = fx/B where B = bandwidth of peak in Hz
3254 //
3255 // #### Test
3256 //
3257 // fi = library("filters.lib");
3258 // os = library("oscillators.lib");
3259 // src = os.osc(440);
3260 // peak_eq_cq_test = src : fi.peak_eq_cq(6, 1000, 4);
3261 //
3262 //
3263 // #### References
3264 //
3265 // * See "Parametric Equalizers" above for references regarding `low_shelf`, `high_shelf`,
3266     and `peak_eq`.
3267 //-----
3268 declare peak_eq_cq author "Julius O. Smith III";
3269 declare peak_eq_cq copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
3270     stanford.edu>";
3271 declare peak_eq_cq license "MIT-style STK-4.3 license";
3272 peak_eq_cq(Lfx,fx,Q) = peak_eq(Lfx,fx,fx/Q);
3273 //-----`(fi.)peak_eq_rm`-----
3274 // Regalia-Mitra second order peaking equalizer section.
3275 //
3276 // #### Usage
3277 //
3278 // _ : peak_eq_rm(Lfx,fx,tanPiBT) : _
3279 //
3280 //
3281 // Where:
3282 //
3283 // * `Lfx`: level (dB) at fx
3284 // * `fx`: boost or cut frequency (Hz)
3285 // * `tanPiBT`: `tan(PI*B/SR)`, where B = -3dB bandwidth (Hz) when  $10^{(Lfx/20)} = 0$ 
3286     ~  $PI*B/SR$  for narrow bandwidths B
3287 //
3288 // #### Test

```

```

3289 // ````
3290 // fi = library("filters.lib");
3291 // os = library("oscillators.lib");
3292 // ma = library("maths.lib");
3293 // src = os.osc(440);
3294 // peak_eq_rm_test = src : fi.peak_eq_rm(6, 1000, tan(ma.PI*200/ma.SR));
3295 // ````
3296 //
3297 // #### References
3298 //
3299 // P.A. Regalia, S.K. Mitra, and P.P. Vaidyanathan,
3300 // "The Digital All-Pass Filter: A Versatile Signal Processing Building Block"
3301 // Proceedings of the IEEE, 76(1):19-37, Jan. 1988. (See pp. 29-30.)
3302 // See also "Parametric Equalizers" above for references on shelf
3303 // and peaking equalizers in general.
3304 //-----
3305 declare peak_eq_rm author "Julius O. Smith III";
3306 declare peak_eq_rm copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
    stanford.edu>";
3307 declare peak_eq_rm license "MIT-style STK-4.3 license";
3308 peak_eq_rm(Lfx,fx,tanPiBT) = _ <: _,A,_ : +,- : *(0.5),*(K/2.0) : + with {
3309     A = tf2(k2, k1*(1+k2), 1, k1*(1+k2), k2) <: _,_; // allpass
3310     k1 = 0.0 - cos(2.0*ma.PI*fx/ma.SR);
3311     k2 = (1.0 - tanPiBT)/(1.0 + tanPiBT);
3312     K = ba.db2linear(Lfx);
3313 };
3314
3315 //-----`(fi.)spectral_tilt`-----
3316 // Spectral tilt filter, providing an arbitrary spectral rolloff factor
3317 // alpha in (-1,1), where
3318 // -1 corresponds to one pole (-6 dB per octave), and
3319 // +1 corresponds to one zero (+6 dB per octave).
3320 // In other words, alpha is the slope of the ln magnitude versus ln frequency.
3321 // For a "pinking filter" (e.g., to generate 1/f noise from white noise),
3322 // set alpha to -1/2.
3323 //
3324 //
3325 // #### Usage
3326 //
3327 // ````
3328 // _ : spectral_tilt(N,f0,bw,alpha) : _
3329 // ````
3330 // Where:
3331 //
3332 // * `N`: desired integer filter order (fixed at compile time)
3333 // * `f0`: lower frequency limit for desired roll-off band > 0
3334 // * `bw`: bandwidth of desired roll-off band
3335 // * `alpha`: slope of roll-off desired in nepers per neper,
3336 //           between -1 and 1 (ln mag / ln radian freq)
3337 //
3338 // #### Test
3339 // ````
3340 // fi = library("filters.lib");
3341 // os = library("oscillators.lib");
3342 // src = os.osc(440);
3343 // spectral_tilt_test = src : fi.spectral_tilt(4, 200, 2000, -0.5);
3344 // ````
3345 //
3346 // #### Example test program
3347 //
3348 // See `dm.spectral_tilt_demo` and the documentation for `no.pink_noise`.
3349 //
3350 // #### References
3351 //
3352 // J.O. Smith and H.F. Smith,
3353 // "Closed Form Fractional Integration and Differentiation via Real Exponentially Spaced Pole
    -Zero Pairs",
3354 // arXiv.org publication arXiv:1606.06154 [cs.CE], June 7, 2016, <http://arxiv.org/abs

```

```

3355 /1606.06154>
3356 //-----
3357 declare spectral_tilt author "Julius O. Smith III";
3358 declare spectral_tilt copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
stanford.edu>";
3359 declare spectral_tilt license "MIT-style STK-4.3 license";
3360 spectral_tilt(N,f0,bw,alpha) = seq(i,N,sec(i)) with {
3361     sec(i) = g * tfis(b1,b0,a0,1) with {
3362         g = a0/b0; // unity dc-gain scaling
3363         b1 = 1.0;
3364         b0 = mzh(i);
3365         a0 = mph(i);
3366         mzh(i) = prewarp(mz(i),ma.SR,w0); // prewarping for bilinear transform
3367         mph(i) = prewarp(mp(i),ma.SR,w0);
3368         prewarp(w,SR,wp) = wp * tan(w*T/2) / tan(wp*T/2) with { T = 1/ma.SR; };
3369         mz(i) = w0 * r ^ (-alpha+i); // minus zero i in s plane
3370         mp(i) = w0 * r ^ i; // minus pole i in s plane
3371         f0p = max(f0,ma.EPSILON); // cannot go to zero
3372         w0 = 2 * ma.PI * f0p; // radian frequency of first pole
3373         f1 = f0p + bw; // upper band limit
3374         r = (f1/f0p)^(1.0/float(N-1)); // pole ratio (2 => octave spacing)
3375     };
3376 };
3377
3378 //-----`(fi.)levelfilter`-----
3379 // Dynamic level lowpass filter.
3380 // `levelfilter` is a standard Faust function.
3381 //
3382 // #### Usage
3383 //
3384 // ...
3385 // _ : levelfilter(L,freq) : _
3386 // ...
3387 //
3388 // Where:
3389 //
3390 // * `L`: desired level (in dB) at Nyquist limit (SR/2), e.g., -60
3391 // * `freq`: corner frequency (-3dB point) usually set to fundamental freq
3392 // * `N`: Number of filters in series where L = L/N
3393 //
3394 // #### Test
3395 // ...
3396 // fi = library("filters.lib");
3397 // os = library("oscillators.lib");
3398 // src = os.osc(440);
3399 // levelfilter_test = fi.levelfilter(0.1, 200, src);
3400 // ...
3401 //
3402 // #### References
3403 //
3404 // * <https://ccrma.stanford.edu/realsimple/faust_strings/Dynamic_Level_Lowpass_Filter.html>
3405 //-----
3406 declare levelfilter author "Julius O. Smith III";
3407 declare levelfilter copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
stanford.edu>";
3408 declare levelfilter license "MIT-style STK-4.3 license";
3409 levelfilter(L,freq,x) = (L * LO * x) + ((1.0-L) * lp2out(x))
3410 with {
3411     LO = pow(L,1/3);
3412     Lw = ma.PI*freq/ma.SR; // = w1 T / 2
3413     Lgain = Lw / (1.0 + Lw);
3414     Lpole2 = (1.0 - Lw) / (1.0 + Lw);
3415     lp2out = *(Lgain) : + ~ *(Lpole2);
3416 };
3417
3418 //-----`(fi.)levelfilterN`-----
3419

```

```

3420 // Dynamic level lowpass filter.
3421 //
3422 // #### Usage
3423 //
3424 // ...
3425 // _ : levelfilterN(N,freq,L) : _
3426 // ...
3427 //
3428 // Where:
3429 //
3430 // * `N`: Number of filters in series where  $L = L/N$ , a constant numerical expression
3431 // * `freq`: corner frequency (-3dB point) usually set to fundamental freq
3432 // * `L`: desired level (in dB) at Nyquist limit ( $SR/2$ ), e.g., -60
3433 //
3434 // #### Test
3435 // ...
3436 // fi = library("filters.lib");
3437 // os = library("oscillators.lib");
3438 // src = os.osc(440);
3439 // levelfilterN_test = src : fi.levelfilterN(3, 200, 0.1);
3440 // ...
3441 //
3442 // #### References
3443 //
3444 // * <https://ccrma.stanford.edu/real/simple/faust\_strings/Dynamic\_Level\_Lowpass\_Filter.html>
3445 //-----
3446 declare levelfilterN author "Julius O. Smith III";
3447 declare levelfilterN copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.stanford.edu>";
3448 declare levelfilterN license "MIT-style STK-4.3 license";
3449 levelfilterN(N,freq,L) = seq(i,N,levelfilter((L/N),freq));
3450
3451
3452 //-----Mth-Octave Filter-Banks-----
3453 // Mth-octave filter-banks split the input signal into a bank of parallel signals, one
3454 // for each spectral band. They are related to the Mth-Octave Spectrum-Analyzers in
3455 // `analysis.lib`.
3456 // The documentation of this library contains more details about the implementation.
3457 // The parameters are:
3458 //
3459 // * `M`: number of band-slices per octave (>1), a constant numerical expression
3460 // * `N`: total number of bands (>2), a constant numerical expression
3461 // * `ftop`: upper bandlimit of the Mth-octave bands (< $SR/2$ )
3462 //
3463 // In addition to the Mth-octave output signals, there is a highpass signal
3464 // containing frequencies from ftop to  $SR/2$ , and a "dc band" lowpass signal
3465 // containing frequencies from 0 (dc) up to the start of the Mth-octave bands.
3466 // Thus, the N output signals are
3467 //
3468 // ...
3469 // highpass(ftop), MthOctaveBands(M,N-2,ftop), dcBand(ftop*2^(-M*(N-1)))
3470 // ...
3471 //
3472 // A Filter-Bank is defined here as a signal bandsplitter having the
3473 // property that summing its output signals gives an allpass-filtered
3474 // version of the filter-bank input signal. A more conventional term for
3475 // this is an "allpass-complementary filter bank". If the allpass filter
3476 // is a pure delay (and possible scaling), the filter bank is said to be
3477 // a "perfect-reconstruction filter bank" (see Vaidyanathan-1993 cited
3478 // below for details). A "graphic equalizer", in which band signals
3479 // are scaled by gains and summed, should be based on a filter bank.
3480 //
3481 // The filter-banks below are implemented as Butterworth or Elliptic
3482 // spectrum-analyzers followed by delay equalizers that make them
3483 // allpass-complementary.
3484 //
3485 // #### Increasing Channel Isolation
3486 //

```

```

3487 // Go to higher filter orders - see Regalia et al. or Vaidyanathan (cited
3488 // below) regarding the construction of more aggressive recursive
3489 // filter-banks using elliptic or Chebyshev prototype filters.
3490 //
3491 // #### References
3492 //
3493 // * "Tree-structured complementary filter banks using all-pass sections", Regalia et al.,
3494 //   IEEE Trans. Circuits & Systems, CAS-34:1470-1484, Dec. 1987
3495 // * "Multirate Systems and Filter Banks", P. Vaidyanathan, Prentice-Hall, 1993
3496 // * Elementary filter theory: <https://ccrma.stanford.edu/~jos/filters/>
3497 //=====
3498 //-----`(fi.)mth_octave_filterbank[n]`-----
3499 // Allpass-complementary filter banks based on Butterworth band-splitting.
3500 // For Butterworth band-splits, the needed delay equalizer is easily found.
3501 //
3502 // #### Usage
3503 //
3504 // ...
3505 // _ : mth_octave_filterbank(0,M,ftop,N) : par(i,N,_) // 0th-order
3506 // _ : mth_octave_filterbank_alt(0,M,ftop,N) : par(i,N,_) // dc-inverted version
3507 // ...
3508 //
3509 // Also for convenience:
3510 //
3511 // ...
3512 // _ : mth_octave_filterbank3(M,ftop,N) : par(i,N,_) // 3rd-order Butterworth
3513 // _ : mth_octave_filterbank5(M,ftop,N) : par(i,N,_) // 5th-order Butterworth
3514 // mth_octave_filterbank_default = mth_octave_filterbank5;
3515 // ...
3516 //
3517 // Where:
3518 //
3519 // * `0`: order of filter used to split each frequency band into two, a constant numerical
3520 //   expression
3521 // * `M`: number of band-slices per octave, a constant numerical expression
3522 // * `ftop`: highest band-split crossover frequency (e.g., 20 kHz)
3523 // * `N`: total number of bands (including dc and Nyquist), a constant numerical expression
3524 //
3525 // #### Test
3526 // ...
3527 // fi = library("filters.lib");
3528 // os = library("oscillators.lib");
3529 // sig = os.osc(440);
3530 // mth_octave_filterbank_test = sig : fi.mth_octave_filterbank(3, 2, 8000, 2);
3531 // ...
3532 //-----
3533 declare mth_octave_filterbank author "Julius O. Smith III";
3534 declare mth_octave_filterbank copyright "Copyright (C) 2003-2019 by Julius O. Smith III <
3535   jos@ccrma.stanford.edu>";
3536 declare mth_octave_filterbank license "MIT-style STK-4.3 license";
3537
3538 mth_octave_filterbank(0,M,ftop,N) = an.mth_octave_analyzer(0,M,ftop,N) : delayeq(N)
3539 with {
3540   fc(n) = ftop * 2^(float(n-N+1)/float(M)); // -3dB crossover frequencies
3541   ap(n) = highpass_plus_lowpass(0,fc(n)); // delay-equalizing allpass
3542   delayeq(N) = par(i,N-2,apchain(i+1)), _, _;
3543   apchain(i) = seq(j,N-1-i,ap(j+1));
3544 };
3545
3546 // dc-inverted version. This reduces the delay-equalizer order for odd 0.
3547 // Negating the input signal makes the dc band noninverting
3548 // and all higher bands sign-inverted (if preferred).
3549 declare mth_octave_filterbank_alt author "Julius O. Smith III";
3550 declare mth_octave_filterbank_alt copyright "Copyright (C) 2003-2019 by Julius O. Smith III
3551   <jos@ccrma.stanford.edu>";
3552 declare mth_octave_filterbank_alt license "MIT-style STK-4.3 license";

```

```

3551 //-----`(fi.)mth_octave_filterbank_alt`-----
3552 //
3553 // #### Usage
3554 //
3555 // ...
3556 // _ : mth_octave_filterbank_alt(0,M,ftop,N) : par(i,N,_)
3557 // ...
3558 //
3559 // Where:
3560 //
3561 // * `0`: filter order used to split each band
3562 // * `M`: number of band-slices per octave
3563 // * `ftop`: highest band-split crossover frequency
3564 // * `N`: total number of bands
3565 //
3566 // #### Test
3567 // ...
3568 // fi = library("filters.lib");
3569 // os = library("oscillators.lib");
3570 // sig = os.osc(440);
3571 // mth_octave_filterbank_alt_test = sig : fi.mth_octave_filterbank_alt(3, 2, 8000, 2);
3572 // ...
3573 //-----
3574 mth_octave_filterbank_alt(0,M,ftop,N) = an.mth_octave_analyzer(0,M,ftop,N) : delayeqi(0,N)
3575 with {
3576   fc(n) = ftop * 2^(float(n-N+1)/float(M)); // -3dB crossover frequencies
3577   ap(n) = highpass_minus_lowpass(0,fc(n)); // half the order of 'plus' case
3578   delayeqi(N) = par(i,N-2,apchain(i+1)), _, *(-1.0);
3579   apchain(i) = seq(j,N-1-i,ap(j+1));
3580 };
3581
3582 // Note that even-order cases require complex coefficients.
3583 // See Vaidyanathan 1993 and papers cited there for more info.
3584 declare mth_octave_filterbank3 author "Julius O. Smith III";
3585 declare mth_octave_filterbank3 copyright "Copyright (C) 2003-2019 by Julius O. Smith III <
    jos@ccrma.stanford.edu>";
3586 declare mth_octave_filterbank3 license "MIT-style STK-4.3 license";
3587
3588 //-----`(fi.)mth_octave_filterbank3`-----
3589 //
3590 // #### Usage
3591 //
3592 // ...
3593 // _ : mth_octave_filterbank3(M,ftop,N) : par(i,N,_)
3594 // ...
3595 //
3596 // Where:
3597 //
3598 // * `M`: number of band-slices per octave
3599 // * `ftop`: highest band-split crossover frequency
3600 // * `N`: total number of bands
3601 //
3602 // #### Test
3603 // ...
3604 // fi = library("filters.lib");
3605 // os = library("oscillators.lib");
3606 // sig = os.osc(440);
3607 // mth_octave_filterbank3_test = sig : fi.mth_octave_filterbank3(2, 8000, 2);
3608 // ...
3609 //-----
3610 mth_octave_filterbank3(M,ftop,N) = mth_octave_filterbank_alt(3,M,ftop,N);
3611
3612 declare mth_octave_filterbank5 author "Julius O. Smith III";
3613 declare mth_octave_filterbank5 copyright "Copyright (C) 2003-2019 by Julius O. Smith III <
    jos@ccrma.stanford.edu>";
3614 declare mth_octave_filterbank5 license "MIT-style STK-4.3 license";
3615
3616 //-----`(fi.)mth_octave_filterbank5`-----

```

```

3617 //
3618 // ##### Usage
3619 //
3620 // ...
3621 // _ : mth_octave_filterbank5(M,ftop,N) : par(i,N,_)
3622 // ...
3623 //
3624 // Where:
3625 //
3626 // * `M`: number of band-slices per octave
3627 // * `ftop`: highest band-split crossover frequency
3628 // * `N`: total number of bands
3629 //
3630 // ##### Test
3631 // ...
3632 // fi = library("filters.lib");
3633 // os = library("oscillators.lib");
3634 // sig = os.osc(440);
3635 // mth_octave_filterbank5_test = sig : fi.mth_octave_filterbank5(2, 8000, 2);
3636 // ...
3637 //-----
3638 mth_octave_filterbank5(M,ftop,N) = mth_octave_filterbank(5,M,ftop,N);
3639
3640 declare mth_octave_filterbank_default author "Julius O. Smith III";
3641 declare mth_octave_filterbank_default copyright "Copyright (C) 2003-2019 by Julius O. Smith
    III <jos@ccrma.stanford.edu>";
3642 declare mth_octave_filterbank_default license "MIT-style STK-4.3 license";
3643
3644 //-----`(fi.)mth_octave_filterbank_default`-----
3645 //
3646 // ##### Test
3647 // ...
3648 // fi = library("filters.lib");
3649 // os = library("oscillators.lib");
3650 // sig = os.osc(440);
3651 // mth_octave_filterbank_default_test = sig : fi.mth_octave_filterbank_default(2, 8000, 2);
3652 // ...
3653 //-----
3654 mth_octave_filterbank_default = mth_octave_filterbank5;
3655
3656 //=====Arbitrary-Crossover Filter-Banks and Spectrum Analyzers=====
3657 // These are similar to the Mth-octave analyzers above, except that the
3658 // band-split frequencies are passed explicitly as arguments.
3659 //=====
3660
3661 // ACKNOWLEDGMENT
3662 // Technique for processing a variable number of signal arguments due
3663 // to Yann Ornlarey (as is the entire Faust framework!)
3664
3665 //-----`(fi.)filterbank`-----
3666 // Filter bank.
3667 // `filterbank` is a standard Faust function.
3668 //
3669 // ##### Usage
3670 //
3671 // ...
3672 // _ : filterbank (O,freqs) : par(i,N,_) // Butterworth band-splits
3673 // ...
3674 //
3675 // Where:
3676 //
3677 // * `O`: band-split filter order (odd integer required for filterbank[i], a constant
    numerical expression)
3678 // * `freqs`: (fc1,fc2,...,fcNs) [in numerically ascending order], where
    Ns=N-1 is the number of octave band-splits
3679 //
3680 // (total number of bands N=Ns+1).
3681 //
3682 // If frequencies are listed explicitly as arguments, enclose them in parens:

```



```

3683 //
3684 // ---
3685 // _ : filterbank(3,(fc1,fc2)) : _,_,_
3686 // ---
3687 //
3688 // #### Test
3689 // ---
3690 // fi = library("filters.lib");
3691 // os = library("oscillators.lib");
3692 // src = os.osc(440);
3693 // filterbank_test = src : fi.filterbank(3, (500, 2000));
3694 // ---
3695 //-----
3696 declare filterbank author "Julius O. Smith III";
3697 declare filterbank copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
    stanford.edu>";
3698 declare filterbank license "MIT-style STK-4.3 license";
3699 filterbank(0,lfreqs) = an.analyzer(0,lfreqs) : delayeq(nb)
3700 with {
3701     nb = ba.count(lfreqs);
3702     fc(n) = ba.take(n, lfreqs);
3703     ap(n) = highpass_plus_lowpass(0,fc(n));
3704     delayeq(1) = _,_; // par(i,0,...) does not fly
3705     delayeq(nb) = par(i,nb-1,apchain(nb-1-i)),_,_ ;
3706     apchain(0) = _;
3707     apchain(i) = ap(i) : apchain(i-1);
3708 };
3709
3710 //-----`fi.filterbanki`-----
3711 // Inverted-dc filter bank.
3712 //
3713 // #### Usage
3714 //
3715 // ---
3716 // _ : filterbanki(0,freqs) : par(i,N,_) // Inverted-dc version
3717 // ---
3718 //
3719 // Where:
3720 //
3721 // * `0`: band-split filter order (odd integer required for `filterbank[i]`, a constant
    numerical expression)
3722 // * `freqs`: (fc1,fc2,...,fcNs) [in numerically ascending order], where
    Ns=N-1 is the number of octave band-splits
3723 // (total number of bands N=Ns+1).
3724 //
3725 //
3726 // If frequencies are listed explicitly as arguments, enclose them in parens:
3727 //
3728 // ---
3729 // _ : filterbanki(3,(fc1,fc2)) : _,_,_
3730 // ---
3731 //
3732 // #### Test
3733 // ---
3734 // fi = library("filters.lib");
3735 // os = library("oscillators.lib");
3736 // src = os.osc(440);
3737 // filterbanki_test = src : fi.filterbanki(3, (500, 2000));
3738 // ---
3739 //-----
3740 declare filterbanki author "Julius O. Smith III";
3741 declare filterbanki copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
    stanford.edu>";
3742 declare filterbanki license "MIT-style STK-4.3 license";
3743 filterbanki(0,lfreqs) = _ <: bsplit(nb)
3744 with {
3745     nb = ba.count(lfreqs);
3746     fc(n) = ba.take(n, lfreqs);
3747     lp(n) = lowpass(0,fc(n));

```

```

3748     hp(n) = highpass(0,fc(n));
3749     ap(n) = highpass_minus_lowpass(0,fc(n));
3750     bsplit(0) = *(-1.0);
3751     bsplit(i) = (hp(i) : delayeq(i-1)), (lp(i) <: bsplit(i-1));
3752     delayeq(0) = _; // moving the *(-1) here inverts all outputs BUT dc
3753     delayeq(i) = ap(i) : delayeq(i-1);
3754 };
3755
3756
3757 //=====State Variable Filters
3758 //##### References
3759 //
3760 // * Solving the continuous SVF equations using trapezoidal integration
3761 // * <https://cytomic.com/files/dsp/SvfLinearTrapOptimised2.pdf>
3762 //=====
3763
3764 //-----`(fi.)svf`-----
3765 // An environment with `lp`, `bp`, `hp`, `notch`, `peak`, `ap`, `bell`, `ls`, `hs` SVF based
3766 // filters.
3767 // All filters have `freq` and `Q` parameters, the `bell`, `ls`, `hs` ones also have a `gain`
3768 // third parameter.
3769 //
3770 // ##### Usage
3771 //
3772 // ~~~
3773 // _ : svf.xx(freq, Q, [gain]) : _
3774 // ~~~
3775 //
3776 // Where:
3777 //
3778 // * `freq`: cut frequency
3779 // * `Q`: quality factor
3780 // * `[gain]`: gain in dB
3781 //
3782 // ##### Test
3783 // ~~~
3784 // fi = library("filters.lib");
3785 // os = library("oscillators.lib");
3786 // sig = os.osc(440);
3787 // svf_lp_test = fi.svf.lp(1000, 0.707, sig);
3788 // ~~~
3789 //-----
3790 declare svf author "Oleg Nesterov";
3791 declare svf copyright "Copyright (C) 2020 Oleg Nesterov <oleg@redhat.com>";
3792 declare svf license "MIT-style STK-4.3 license";
3793
3794 svf = environment {
3795
3796     // Internal implementation
3797     svf(T,F,Q,G) = tick ~ (_,_) : !,!,si.dot(3, mix)
3798     with {
3799         tick(ic1eq, ic2eq, v0) =
3800             2*v1 - ic1eq,
3801             2*v2 - ic2eq,
3802             v0, v1, v2
3803         with {
3804             v1 = ic1eq + g *(v0-ic2eq) : /(1 + g*(g+k));
3805             v2 = ic2eq + g * v1;
3806         };
3807
3808         A = pow(10.0, G/40.0);
3809
3810         g = tan(F * ma.PI/ma.SR) : case {
3811             (7) => /(sqrt(A));
3812             (8) => *(sqrt(A));
3813             (t) => _;
3814         } (T);
3815     };
3816 } (T);

```

```

3813
3814     k = case {
3815         (6) => 1/(Q*A);
3816         (t) => 1/Q;
3817     } (T);
3818
3819     mix = case {
3820         (0) => 0, 0, 1;
3821         (1) => 0, 1, 0;
3822         (2) => 1, -k, -1;
3823         (3) => 1, -k, 0;
3824         (4) => 1, -k, -2;
3825         (5) => 1, -2*k, 0;
3826         (6) => 1, k*(A*A-1), 0;
3827         (7) => 1, k*(A-1), A*A-1;
3828         (8) => A*A, k*(1-A)*A, 1-A*A;
3829     } (T);
3830 };
3831
3832 // External API
3833 lp(f,q) = svf(0, f, q, 0);
3834 bp(f,q) = svf(1, f, q, 0);
3835 hp(f,q) = svf(2, f, q, 0);
3836 notch(f,q) = svf(3, f, q, 0);
3837 peak(f,q) = svf(4, f, q, 0);
3838 ap(f,q) = svf(5, f, q, 0);
3839 bell(f,q,g) = svf(6, f, q, g);
3840 ls(f,q,g) = svf(7, f, q, g);
3841 hs(f,q,g) = svf(8, f, q, g);
3842 };
3843
3844
3845 //-----`fi.)svf_morph`-----
3846 // An SVF-based filter that can smoothly morph between
3847 // being lowpass, bandpass, and highpass.
3848 //
3849 // #### Usage
3850 //
3851 // ```
3852 // _ : svf_morph(freq, Q, blend) : _
3853 // ```
3854 //
3855 // Where:
3856 //
3857 // * `freq`: cutoff frequency
3858 // * `Q`: quality factor
3859 // * `blend`: [0..2] continuous, where 0 is `lowpass`, 1 is `bandpass`, and 2 is `highpass`.
3860 // For performance, the value is not clamped to [0..2].
3861 //
3862 // #### Test
3863 // ```
3864 // fi = library("filters.lib");
3865 // os = library("oscillators.lib");
3866 // sig = os.osc(440);
3867 // svf_morph_test = fi.svf_morph(1000, 0.707, 1, sig);
3868 // ```
3869 //
3870 // #### Example test program
3871 //
3872 // ```
3873 // process = no.noise : svf_morph(freq, q, blend)
3874 // with {
3875 //   blend = hslider("Blend", 0, 0, 2, .01) : si.smoo;
3876 //   q = hslider("Q", 1, 0.1, 10, .01) : si.smoo;
3877 //   freq = hslider("freq", 5000, 100, 18000, 1) : si.smoo;
3878 // };
3879 // ```

```

```

3880 // #### References
3881 //
3882 // <https://github.com/mtytel/vital/blob/636ca0ef517a4db087a6a08a6a8a5e704e21f836/src/
3883 // synthesis/filters/digital_svf.cpp#L292-L295>
3884 //-----
3885 declare svf_morph author "David Braun and Clarence W. Rowley";
3886 declare svf_morph copyright "Copyright (C) 2024 David Braun <braun@ccrma.stanford.edu>";
3887 declare svf_morph license "MIT-style STK-4.3 license";
3888 svf_morph(f, q, _b, x) = svf.lp(f,q,x)*w_LP + svf.bp(f,q,x)*w_BP + svf.hp(f,q,x)*w_HP
3889 with {
3890   b = _b - 1; // b is now -1 to 1.
3891   w_LP = max(-b, 0);
3892   w_BP = sqrt(1 - b^2);
3893   w_HP = max(b, 0);
3894 };
3895
3896 //-----`(fi.)svf_notch_morph`-----
3897 // An SVF-based notch-filter that can smoothly morph between
3898 // being lowpass, notch, and highpass.
3899 //
3900 // #### Usage
3901 //
3902 // ...
3903 // _ : svf_notch_morph(freq, Q, blend) : _
3904 // ...
3905 //
3906 // Where:
3907 //
3908 // * `freq`: cutoff frequency
3909 // * `Q`: quality factor
3910 // * `blend`: [0..2] continuous, where 0 is `lowpass`, 1 is `notch`, and 2 is `highpass`. For
3911 //   performance, the value is not clamped to [0..2].
3912 //
3913 // #### Test
3914 // ...
3915 // fi = library("filters.lib");
3916 // os = library("oscillators.lib");
3917 // sig = os.osc(440);
3918 // svf_notch_morph_test = fi.svf_notch_morph(1000, 0.707, 1, sig);
3919 // ...
3920 //
3921 // #### Example test program
3922 //
3923 // ...
3924 // process = no.noise : svf_notch_morph(freq, q, blend)
3925 // with {
3926 //   blend = hslider("Blend", 0, 0, 2, .01) : si.smoo;
3927 //   q = hslider("Q", 1, 0.1, 10, .01) : si.smoo;
3928 //   freq = hslider("freq", 5000, 100, 18000, 1) : si.smoo;
3929 // };
3930 // ...
3931 //
3932 // #### References
3933 //
3934 // <https://github.com/mtytel/vital/blob/636ca0ef517a4db087a6a08a6a8a5e704e21f836/src/
3935 // synthesis/filters/digital_svf.cpp#L256C36-L263>
3936 //-----
3937 declare svf_notch_morph author "David Braun and Clarence W. Rowley";
3938 declare svf_notch_morph copyright "Copyright (C) 2024 David Braun <braun@ccrma.stanford.edu>";
3939 declare svf_notch_morph license "MIT-style STK-4.3 license";
3940 svf_notch_morph(f, q, _b, x) = svf.lp(f,q,x)*w_LP + svf.hp(f,q,x)*w_HP
3941 with {
3942   b = _b - 1; // b is now -1 to 1.
3943   w_LP = min(1-b, 1);
3944   w_HP = min(1+b, 1);
3945 };

```

```

3944
3945
3946 //-----`(fi.)SVFTPT`-----
3947 //
3948 // Topology-preserving transform implementation following Zavalishin's method.
3949 //
3950 // Outputs: lowpass, highpass, bandpass, normalised bandpass, notch, allpass,
3951 // peaking.
3952 //
3953 // Each individual output can be recalled with its name in the environment as in:
3954 // `SVFTPT.LP2(1000.0, .707)`.
3955 //
3956 // The 7 outputs can be recalled by using `SVF` name as in:
3957 // `SVFTPT.SVF(1000.0, .707)`.
3958 //
3959 // Even though the implementation is different, the characteristics of this
3960 // filter are comparable to those of the `svf` environment in this library.
3961 //
3962 // ### Usage:
3963 //
3964 // ```
3965 // _ : SVFTPT.xxx(CF, Q) : _
3966 // ```
3967 //
3968 // Where:
3969 //
3970 // * `xxx` can be one of the following: `LP2`, `HP2`, `BP2`, `BP2Norm`, `Notch2`, `AP2`, `
3971 //   Peaking2`
3972 // * `CF`: cutoff in Hz
3973 // * `Q`: resonance
3974 //
3975 // ### Test
3976 // ```
3977 // fi = library("filters.lib");
3978 // os = library("oscillators.lib");
3979 // sig = os.osc(440);
3980 // SVFTPT_SVF_test = fi.SVFTPT.SVF(1000, 0.707, sig);
3981 // ```
3982 //-----
3983 declare SVFTPT author "Dario Sanfilippo";
3984 declare SVFTPT copyright
3985   "Copyright (C) 2024 Dario Sanfilippo <sanfilippo.dario@gmail.com>";
3986 declare SVFTPT license "MIT License";
3987 SVFTPT = environment {
3988   SVF(CF, Q, x) = f ~ si.bus(2) : (!, !, _, _, _, _, _, _)
3989     with {
3990       g = tan(CF * ma.PI * ma.T);
3991       R2 = 1.0 / Q;
3992       gPlusR2 = g + R2;
3993       f(s0, s1) = u0, u1, LP, HP, BP, BPNorm, Notch, AP, Peaking
3994         with {
3995           HP = (x - s0 * gPlusR2 - s1) / (1.0 + g * gPlusR2);
3996           v0 = HP * g;
3997           BP = s0 + v0;
3998           v1 = BP * g;
3999           LP = s1 + v1;
4000           BPNorm = BP * R2;
4001           Notch = x - BPNorm;
4002           AP = x - BP * (R2 + R2);
4003           Peaking = LP - HP;
4004           u0 = v0 + BP;
4005           u1 = v1 + LP;
4006         };
4007   LP2(CF, Q, x) = SVF(CF, Q, x) : ba.selectn(7, 0);
4008   HP2(CF, Q, x) = SVF(CF, Q, x) : ba.selectn(7, 1);
4009   BP2(CF, Q, x) = SVF(CF, Q, x) : ba.selectn(7, 2);
4010   BP2Norm(CF, Q, x) = SVF(CF, Q, x) : ba.selectn(7, 3);

```

```

4011     Notch2(CF, Q, x) = SVF(CF, Q, x) : ba.selectn(7, 4);
4012     AP2(CF, Q, x) = SVF(CF, Q, x) : ba.selectn(7, 5);
4013     Peaking2(CF, Q, x) = SVF(CF, Q, x) : ba.selectn(7, 6);
4014 };
4015
4016 //-----`(fi.)dynamicSmoothing`-----
4017 //
4018 // Adaptive smoother based on Andy Simper's paper.
4019 //
4020 // This filter uses both the lowpass and bandpass outputs of a
4021 // state-variable filter. The lowpass is used to smooth out the input signal,
4022 // the bandpass, which is a smoothed out version of the highpass, provides
4023 // information on the rate of change of the input. Hence, the bandpass signal
4024 // can be used to adjust the cutoff of the filter to quickly follow the input's
4025 // fast and large variations while effectively filtering out local
4026 // perturbations.
4027 //
4028 // This implementation does not use an approximation for the CF computation,
4029 // and it deploys guards to prevent overshooting with extreme sensitivity
4030 // values.
4031 //
4032 // #### Usage:
4033 //
4034 // ...
4035 // _ : dynamicSmoothing(sensitivity, baseCF) : _
4036 // ...
4037 //
4038 // Where:
4039 //
4040 // * `sensitivity`: sensitivity to changes in the input signal.
4041 //   The range is, theoretically, from 0 to INF, though anything between
4042 //   0.0 and 1.0 should be reasonable
4043 // * `baseCF`: cutoff frequency, in Hz, when there is no variation in the
4044 //   input signal
4045 //
4046 // #### Test
4047 // ...
4048 // fi = library("filters.lib");
4049 // os = library("oscillators.lib");
4050 // sig = os.osc(440);
4051 // dynamicSmoothing_test = fi.dynamicSmoothing(0.5, 500, sig);
4052 // ...
4053 //
4054 // #### References
4055 //
4056 // * <https://cytomic.com/files/dsp/DynamicSmoothing.pdf>
4057 //-----
4058 declare dynamicSmoothing author "Dario Sanfilippo";
4059 declare dynamicSmoothing copyright
4060     "Copyright (C) 2024 Dario Sanfilippo <sanfilippo.dario@gmail.com>";
4061 declare dynamicSmoothing license "MIT License";
4062 dynamicSmoothing(sensitivity, baseCF, x) = f ~ _ : ! , _
4063     with {
4064         f(s) = SVFTPT.BP2(CF, .5 , x) , SVFTPT.LP2(CF, .5, x)
4065         with {
4066             CF = min(ma.SR * .125, baseCF + sensitivity * abs(s) * ma.SR * .5);
4067         };
4068     };
4069
4070
4071 //-----`(fi.)oneEuro`-----
4072 // The One Euro Filter €(1 Filter) is an adaptive lowpass filter.
4073 // This kind of filter is commonly used in object-tracking,
4074 // not necessarily audio processing.
4075 //
4076 // #### Usage
4077 //
4078 // ...

```

```

4079 // _ : oneEuro(derivativeCutoff, beta, minCutoff) : _
4080 // ...
4081 //
4082 // Where:
4083 //
4084 // * `derivativeCutoff`: Used to filter the first derivative of the input. 1 Hz is a good
4085 //   default.
4086 // * `beta`: "Speed" parameter where higher values reduce latency.
4087 // * `minCutoff`: Minimum cutoff frequency in Hz. Lower values remove more jitter.
4088 //
4089 // #### Test
4090 // ...
4091 // fi = library("filters.lib");
4092 // os = library("oscillators.lib");
4093 // sig = os.osc(440);
4094 // oneEuro_test = sig : fi.oneEuro(1, 0.5, 5);
4095 // ...
4096 //
4097 // #### References
4098 //
4099 // * <https://gery.casiez.net/1euro/>
4100 //-----
4101 declare oneEuro author "David Braun";
4102 declare oneEuro copyright "Copyright (C) 2024 by David Braun <braun@ccrma.stanford.edu>";
4103 declare oneEuro license "MIT";
4104
4105 oneEuro(derivativeCutoff, beta, minCutoff) = _oneEuro ~ _
4106 with {
4107   // exponential moving average that calculates its own alpha from a cutoff in Hz
4108   // _ : ema(cutoff) : _
4109   ema(cutoff) = tick ~ _
4110   with {
4111     alpha = 1.0 / (1.0 + ma.SR / (2 * ma.PI * cutoff));
4112     tick(prev, x) = prev*(1-alpha) + x*alpha;
4113   };
4114   _oneEuro(prev, x) = x : ema(adaptiveCutoff)
4115   with {
4116     derivative = (x - prev)*ma.SR;
4117     derivativeFiltered = derivative : ema(derivativeCutoff);
4118     adaptiveCutoff = minCutoff + beta * abs(derivativeFiltered);
4119   };
4120 };
4121
4122 //=====Linkwitz-Riley 4th-order 2-way, 3-way, and 4-way crossovers=====
4123 //
4124 // The Linkwitz-Riley (LR) crossovers are designed to produce a fully-flat
4125 // magnitude response when their outputs are combined. The 4th-order
4126 // LR filters (LR4) have a 24dB/octave slope and they are rather popular audio
4127 // crossovers used in multi-band processing.
4128 //
4129 // The LR4 can be constructed by cascading two second-order Butterworth
4130 // filters. For the second-order Butterworth filters, we will use the SVF
4131 // filter implemented above by setting the Q-factor to 1.0 / sqrt(2.0).
4132 // These will be cascaded in pairs to build the LR4 highpass and lowpass.
4133 // For the phase correction, we will use the 2nd-order Butterworth allpass.
4134 //
4135 //
4136 // #### References
4137 //
4138 // Zavalishin, Vadim. "The art of VA filter design." Native Instruments, Berlin, Germany
4139 //   (2012).
4140 //=====
4141
4142 //-----`(fi.)lowpassLR4`-----
4143 // 4th-order Linkwitz-Riley lowpass.
4144 //

```

```

4145 // #### Usage
4146 //
4147 // ```
4148 // _ : lowpassLR4(cf) : _
4149 // ```
4150 //
4151 // Where:
4152 //
4153 // * `cf`: lowpass cutoff in Hz
4154 //
4155 // #### Test
4156 // ```
4157 // fi = library("filters.lib");
4158 // os = library("oscillators.lib");
4159 // src = os.osc(440);
4160 // lowpassLR4_test = src : fi.lowpassLR4(1000);
4161 // ```
4162 //-----
4163 declare lowpassLR4 author "Dario Sanfilippo";
4164 declare lowpassLR4 copyright
4165   "Copyright (C) 2022 Dario Sanfilippo <sanfilippo.dario@gmail.com>";
4166 declare lowpassLR4 license "MIT-style STK-4.3 license";
4167 lowpassLR4(cf, x) = x : seq(i, 2, svf.lp(cf, 1.0 / sqrt(2.0)));
4168
4169 //-----
4170 //-----`(fi.)highpassLR4`-----
4171 // 4th-order Linkwitz-Riley highpass.
4172 //
4173 // #### Usage
4174 //
4175 // ```
4176 // _ : highpassLR4(cf) : _
4177 // ```
4178 //
4179 // Where:
4180 //
4181 // * `cf`: highpass cutoff in Hz
4182 //
4183 // #### Test
4184 // ```
4185 // fi = library("filters.lib");
4186 // os = library("oscillators.lib");
4187 // src = os.osc(440);
4188 // highpassLR4_test = src : fi.highpassLR4(1000);
4189 // ```
4190 //-----
4191 declare highpassLR4 author "Dario Sanfilippo";
4192 declare highpassLR4 copyright
4193   "Copyright (C) 2022 Dario Sanfilippo <sanfilippo.dario@gmail.com>";
4194 declare highpassLR4 license "MIT-style STK-4.3 license";
4195 highpassLR4(cf, x) = x : seq(i, 2, svf.hp(cf, 1.0 / sqrt(2.0)));
4196
4197 //-----
4198 //-----`(fi.)crossover2LR4`-----
4199 // Two-way 4th-order Linkwitz-Riley crossover.
4200 //
4201 // #### Usage
4202 //
4203 // ```
4204 // _ : crossover2LR4(cf) : si.bus(2)
4205 // ```
4206 //
4207 // Where:
4208 //
4209 // * `cf`: crossover split cutoff in Hz
4210 //
4211 // #### Test
4212 // ```

```



```

4213 // fi = library("filters.lib");
4214 // os = library("oscillators.lib");
4215 // src = os.osc(440);
4216 // crossover2LR4_test = src : fi.crossover2LR4(1000);
4217 // ```
4218 //-----
4219 declare crossover2LR4 author "Dario Sanfilippo";
4220 declare crossover2LR4 copyright
4221   "Copyright (C) 2022 Dario Sanfilippo <sanfilippo.dario@gmail.com>";
4222 declare crossover2LR4 license "MIT-style STK-4.3 license";
4223 crossover2LR4(cf, x) = lowpassLR4(cf, x) , highpassLR4(cf, x);
4224
4225
4226 //-----`(fi.)crossover3LR4`-----
4227 // Three-way 4th-order Linkwitz-Riley crossover.
4228 //
4229 // #### Usage
4230 //
4231 // ```
4232 // _ : crossover3LR4(cf1, cf2) : si.bus(3)
4233 // ```
4234 //
4235 // Where:
4236 //
4237 // * `cf1`: crossover lower split cutoff in Hz
4238 // * `cf2`: crossover upper split cutoff in Hz
4239 //
4240 // #### Test
4241 // ```
4242 // fi = library("filters.lib");
4243 // os = library("oscillators.lib");
4244 // src = os.osc(440);
4245 // crossover3LR4_test = src : fi.crossover3LR4(500, 2000);
4246 // ```
4247 //-----
4248 declare crossover3LR4 author "Dario Sanfilippo";
4249 declare crossover3LR4 copyright
4250   "Copyright (C) 2022 Dario Sanfilippo <sanfilippo.dario@gmail.com>";
4251 declare crossover3LR4 license "MIT-style STK-4.3 license";
4252 crossover3LR4(cf1, cf2, x) =
4253   crossover2LR4(cf1, x) : svf.ap(cf2, 1.0 / sqrt(2.0)) , crossover2LR4(cf2);
4254
4255
4256 //-----`(fi.)crossover4LR4`-----
4257 // Four-way 4th-order Linkwitz-Riley crossover.
4258 //
4259 // #### Usage
4260 //
4261 // ```
4262 // _ : crossover4LR4(cf1, cf2, cf3) : si.bus(4)
4263 // ```
4264 //
4265 // Where:
4266 //
4267 // * `cf1`: crossover lower split cutoff in Hz
4268 // * `cf2`: crossover mid split cutoff in Hz
4269 // * `cf3`: crossover upper split cutoff in Hz
4270 //
4271 // #### Test
4272 // ```
4273 // fi = library("filters.lib");
4274 // os = library("oscillators.lib");
4275 // src = os.osc(440);
4276 // crossover4LR4_test = src : fi.crossover4LR4(300, 1000, 3000);
4277 // ```
4278 //-----
4279 declare crossover4LR4 author "Dario Sanfilippo";
4280 declare crossover4LR4 copyright

```

```

4281 "Copyright (C) 2022 Dario Sanfilippo <sanfilippo.dario@gmail.com>;
4282 declare crossover4LR4 license "MIT-style STK-4.3 license";
4283 crossover4LR4(cf1, cf2, cf3, x) =
4284     crossover2LR4(cf2, x) :
4285         svf.ap(cf3, 1.0 / sqrt(2.0)) ,
4286         svf.ap(cf1, 1.0 / sqrt(2.0)) :
4287             crossover2LR4(cf1) ,
4288             crossover2LR4(cf3);
4289
4290
4291 //-----`(fi.)crossover8LR4`-----
4292 // Eight-way 4th-order Linkwitz-Riley crossover.
4293 //
4294 // ### Usage
4295 //
4296 // ```
4297 // _ : crossover8LR4(cf1, cf2, cf3, cf4, cf5, cf6, cf7) : si.bus(8)
4298 // ```
4299 //
4300 // Where:
4301 //
4302 // * `cf1`: lowest crossover cutoff frequency in Hz
4303 // * `cf2`: second crossover cutoff frequency in Hz
4304 // * `cf3`: third crossover cutoff frequency in Hz
4305 // * `cf4`: fourth crossover cutoff frequency in Hz
4306 // * `cf5`: fifth crossover cutoff frequency in Hz
4307 // * `cf6`: sixth crossover cutoff frequency in Hz
4308 // * `cf7`: highest crossover cutoff frequency in Hz
4309 //
4310 // ### Test
4311 // ```
4312 // fi = library("filters.lib");
4313 // os = library("oscillators.lib");
4314 // src = os.osc(440);
4315 // crossover8LR4_test = src : fi.crossover8LR4(100, 200, 400, 800, 1600, 3200, 6400);
4316 // ```
4317 //-----
4318 declare crossover8LR4 author "Dario Sanfilippo";
4319 declare crossover8LR4 copyright
4320     "Copyright (C) 2022 Dario Sanfilippo <sanfilippo.dario@gmail.com>";
4321 declare crossover8LR4 license "MIT-style STK-4.3 license";
4322 crossover8LR4(cf1, cf2, cf3, cf4, cf5, cf6, cf7, x) =
4323     crossover2LR4(cf4, x) :
4324         (ap(cf6) : ap(cf5) : ap(cf7)) ,
4325         (ap(cf2) : ap(cf1) : ap(cf3)) :
4326             crossover2LR4(cf2) ,
4327             crossover2LR4(cf6) :
4328                 ap(cf3) ,
4329                 ap(cf1) ,
4330                 ap(cf7) ,
4331                 ap(cf5) :
4332                     crossover2LR4(cf1) ,
4333                     crossover2LR4(cf3) ,
4334                     crossover2LR4(cf5) ,
4335                     crossover2LR4(cf7)
4336     with {
4337         ap(cf) = svf.ap(cf, 1.0 / sqrt(2.0));
4338     };
4339
4340
4341 //===== Standardized Filters =====
4342 //=====
4343 //
4344 // This section provides filters that are defined by national or
4345 // international standards, e.g. for measurement applications.
4346
4347 //-----`(fi.)itu_r_bs_1770_4_kfilter`-----
4348 // The prefilter from Recommendation ITU-R BS.1770-4 for loudness

```

```

4349 // measurement. Also known as "K-filter". The recommendation defines
4350 // biquad filter coefficients for a fixed sample rate of 48kHz (page
4351 // 4-5). Here, we construct biquads for arbitrary samplerates. The
4352 // resulting filter is normalized, such that the magnitude at 997Hz is
4353 // unity gain 1.0.
4354 //
4355 // Please note, the ITU-recommendation handles the normalization in
4356 // equation (2) by subtracting 0.691dB, which is not needed with
4357 // `itu_r_bs_1770_4_kfilter`.
4358 //
4359 // One option for future improvement might be, to round those filter
4360 // coefficients, that are almost equal to one. Second, the maximum
4361 // magnitude difference at 48kHz between the ITU-defined filter and
4362 // `itu_r_bs_1770_4_kfilter` is 0.001dB, which obviously could be
4363 // less.
4364 //
4365 // #### Usage
4366 //
4367 // ...
4368 // _ : itu_r_bs_1770_4_kfilter : _
4369 // ...
4370 //
4371 // #### Test
4372 //
4373 // fi = library("filters.lib");
4374 // os = library("oscillators.lib");
4375 // src = os.osc(440);
4376 // itu_r_bs_1770_4_kfilter_test = src : fi.itu_r_bs_1770_4_kfilter;
4377 // ...
4378 //
4379 // #### References
4380 //
4381 // * <https://www.itu.int/rec/R-REC-BS.1770>
4382 // * <https://gist.github.com/jkdb/07521a98f7873a2dc3dbe16417930791>
4383 //-----
4384 declare itu_r_bs_1770_4_kfilter author "Jakob D  bel";
4385 declare itu_r_bs_1770_4_kfilter copyright "Copyright (C) 2022 Jakob D  bel";
4386 declare itu_r_bs_1770_4_kfilter license "ISC license";
4387
4388 itu_r_bs_1770_4_kfilter = stage1 : stage2 : normalize997Hz
4389 with {
4390   freq2k(f_c) = tan((ma.PI * f_c)/ma.SR);
4391
4392   stage1 = tf22t(b0,b1,b2,a1,a2)
4393   with {
4394     f_c = 1681.7632251028442; // Hertz
4395     gain = 3.9997778685513232; // Decibel
4396     K = freq2k(f_c);
4397     V_0 = pow(10, (gain/20.0));
4398
4399     denominator = 1.0 + sqrt(2.0)*K + K^2;
4400     b0 = (V_0 + sqrt((2.0*V_0))*K + K^2) / denominator;
4401     b1 = 2.0*(K^2 - V_0) / denominator;
4402     b2 = (V_0 - sqrt(2.0*V_0)*K + K^2) / denominator;
4403
4404     a1 = 2*(K^2 - 1) / denominator;
4405     a2 = (1 - sqrt(2.0)*K + K^2) / denominator;
4406   };
4407
4408   stage2 = tf22t(b0,b1,b2,a1,a2)
4409   with {
4410     f_c = 38.135470876002174; // Hertz
4411     Q = 0.5003270373223665;
4412     K = freq2k(f_c);
4413
4414     denominator = (K^2) * Q + K + Q;
4415     b0 = Q / denominator;
4416     b1 = -2*Q / denominator;

```

```

4417     b2 = b0;
4418
4419     a1 = (2*Q * (K^2 - 1)) / denominator;
4420     a2 = ((K^2) * Q - K + Q) / denominator;
4421 };
4422
4423     normalize997Hz = *(0.9273671710547968);
4424 };
4425
4426
4427 //=====Averaging Functions=====
4428 //=====
4429 //
4430 // These are a set of samplerate independent averaging functions based on
4431 // moving-average and one-pole filters with specific response characteristics.
4432
4433 //-----`(fi.)avg_rect`-----
4434 // Moving average.
4435 //
4436 // #### Usage
4437 //
4438 // ```
4439 // _ : avg_rect(period) : _
4440 // ```
4441 //
4442 // Where:
4443 //
4444 // * `period`: averaging frame duration in seconds
4445 //-----
4446 declare avg_rect author "Dario Sanfilippo and Julius O. Smith III";
4447 declare avg_rect copyright "Copyright (C) 2020 Dario Sanfilippo
4448     <sanfilippo.dario@gmail.com> and
4449     2003-2020 by Julius O. Smith III <jos@ccrma.stanford.edu>";
4450 declare avg_rect license "MIT-style STK-4.3 license";
4451 avg_rect(period, x) = x : ba.slidingMean(rint(period * ma.SR));
4452
4453
4454 //-----`(fi.)avg_tau`-----
4455 // Averaging function based on a one-pole filter and the tau response time.
4456 // Tau represents the effective length of the one-pole impulse response,
4457 // that is, tau is the integral of the filter's impulse response. This
4458 // response is slower to reach the final value but has less ripples in
4459 // non-steady signals.
4460 //
4461 // #### Usage
4462 //
4463 // ```
4464 // _ : avg_tau(period) : _
4465 // ```
4466 //
4467 // Where:
4468 //
4469 // * `period`: time, in seconds, for the system to decay by 1/e,
4470 // or to reach 1-1/e of its final value.
4471 //
4472 // #### References
4473 //
4474 // * <https://ccrma.stanford.edu/~jos/mdft/Exponentials.html>
4475 //-----
4476 declare avg_tau author "Dario Sanfilippo and Julius O. Smith III";
4477 declare avg_tau copyright "Copyright (C) 2020 Dario Sanfilippo
4478     <sanfilippo.dario@gmail.com> and
4479     2003-2020 by Julius O. Smith III <jos@ccrma.stanford.edu>";
4480 declare avg_tau license "MIT-style STK-4.3 license";
4481 avg_tau(period, x) = fi.lptau(period, x);
4482
4483
4484 //-----`(fi.)avg_t60`-----

```

```

4485 // Averaging function based on a one-pole filter and the t60 response time.
4486 // This response is particularly useful when the system is required to
4487 // reach the final value after about `period` seconds.
4488 //
4489 // #### Usage
4490 //
4491 // ...
4492 // _ : avg_t60(period) : _
4493 // ...
4494 //
4495 // Where:
4496 //
4497 // * `period`: time, in seconds, for the system to decay by 1/1000,
4498 // or to reach 1-1/1000 of its final value.
4499 //
4500 // #### References
4501 //
4502 // <https://ccrma.stanford.edu/~jos/mdft/Audio\_Decay\_Time\_T60.html>
4503 //-----
4504 declare avg_t60 author "Dario Sanfilippo and Julius O. Smith III";
4505 declare avg_t60 copyright "Copyright (C) 2020 Dario Sanfilippo
4506     <sanfilippo.dario@gmail.com> and
4507     2003-2020 by Julius O. Smith III <jos@ccrma.stanford.edu>";
4508 declare avg_t60 license "MIT-style STK-4.3 license";
4509 avg_t60(period, x) = fi.lpt60(period, x);
4510
4511 //-----`(fi.)avg_t19`-----
4512 // Averaging function based on a one-pole filter and the t19 response time.
4513 // This response is close to the moving-average algorithm as it roughly reaches
4514 // the final value after `period` seconds and shows about the same
4515 // oscillations for non-steady signals.
4516 //
4517 // #### Usage
4518 //
4519 // ...
4520 // _ : avg_t19(period) : _
4521 // ...
4522 //
4523 // Where:
4524 //
4525 // * `period`: time, in seconds, for the system to decay by 1/e2.2,
4526 // or to reach 1-1/e2.2 of its final value.
4527 //
4528 // #### References
4529 //
4530 // Zölzer, U. (2008). Digital audio signal processing (Vol. 9). New York: Wiley.
4531 //-----
4532 declare avg_t19 author "Dario Sanfilippo and Julius O. Smith III";
4533 declare avg_t19 copyright "Copyright (C) 2020 Dario Sanfilippo
4534     <sanfilippo.dario@gmail.com> and
4535     2003-2020 by Julius O. Smith III <jos@ccrma.stanford.edu>";
4536 declare avg_t19 license "MIT-style STK-4.3 license";
4537 avg_t19(period, x) = fi.lpt19(period, x);
4538
4539 //=====Kalman Filters=====
4540 //=====
4541 //
4542 // Functions related to the Kalman filter.
4543
4544 kalmanEnv = environment {
4545     matMul = la.matMul;
4546
4547     // Predicts the next state.
4548     // * `N`: State size
4549     // * `M`: Measurement size

```

```

4553 // * `F`: State transition matrix (NxN)
4554 // * `B`: Control input matrix (NxM)
4555 // Implicit arguments:
4556 // * `x`: Current state (Nx1)
4557 // * `u`: Control input (Mx1)
4558 predictState(N, M, F, B) = si.vecOp((Fx, Bu), +)
4559 with {
4560     // Fx = F @ x, where F is NxN and x is Nx1
4561     Fx = matMul(N, N, N, 1, F, si.bus(N));
4562
4563     // Bu = B @ u, where B is NxM and u is Mx1
4564     Bu = matMul(N, M, M, 1, B, si.bus(M));
4565 };
4566
4567 // Predicts the covariance matrix for the next state.
4568 // * `N`: State size
4569 // * `F`: State transition matrix (NxN)
4570 // * `Q`: Process noise covariance matrix (NxN)
4571 // * `P`: Current state covariance matrix (NxN)
4572 // No implicit arguments.
4573 predictCovariance(N, F, Q, P) = si.vecOp((FPF_T, Q), +)
4574 with {
4575     // FP = F @ P
4576     FP = matMul(N, N, N, N, F, P);
4577
4578     // FPF_T = FP @ F^T, where FP is NxN and F^T is NxN
4579     FPF_T = matMul(N, N, N, N, FP, la.transpose2(N, N, F));
4580 };
4581
4582 // Calculates the Kalman gain (NxM).
4583 // * `N`: State size
4584 // * `M`: Measurement size
4585 // * `H`: Observation matrix (MxN)
4586 // * `R`: Measurement noise covariance matrix (MxM)
4587 // Implicit arguments:
4588 // * `P_pred`: Predicted covariance matrix (NxN)
4589 kalmanGain(N, M, H, R) = P_pred <:
4590     matMul(N, M, M, M, matMul(N, N, N, M, P_pred, H_T), S_inv)
4591 with {
4592     P_pred = si.bus(N*N);
4593     // H_T = Transpose of H
4594     H_T = la.transpose2(M, N, H);
4595
4596     // HP = H * P_pred, where H is MxN and P_pred is NxN
4597     HP = matMul(M, N, N, N, H, P_pred);
4598
4599     // HPH_T = HP @ H^T, where HP is MxN and H^T is NxM, resulting in MxM
4600     HPH_T = matMul(M, N, N, M, HP, H_T);
4601
4602     // S_inv = inverse(HPH_T + R)
4603     S_inv = si.vecOp((HPH_T, R), +) : la.inverse(M);
4604 };
4605
4606 // Updates the state estimate based on the Kalman gain and measurement.
4607 // * `N`: State size
4608 // * `M`: Measurement size
4609 // * `H`: Observation matrix (MxN)
4610 // Implicit arguments:
4611 // * `K`: Kalman gain matrix (NxM)
4612 // * `x_pred`: (Nx1)
4613 // * `z`: (Mx1)
4614 updateState(N, M, H) = si.bus(N*M), si.bus(N), si.bus(M) <:
4615     si.vecOp((get_x_pred, K_mul_residual(N, M, H)), +)
4616 with {
4617     get_x_pred = par(i, N*M, !), par(i, N, _), par(i, M, !);
4618 };
4619
4620 // todo: find a way to not use mySwap.

```

```

4621 // Its purpose is to route K, x, z into K, z, x
4622 mySwap(N, M) = si.bus(N*M), si.bus(N), si.bus(M) <:
4623   select_K, select_z, select_x_pred
4624 with {
4625   select_K = par(i, N*M, _), par(i, N, !), par(i, M, !);
4626   select_x_pred = par(i, N*M, !), par(i, N, _), par(i, M, !);
4627   select_z = par(i, N*M, !), par(i, N, !), par(i, M, _);
4628 };
4629
4630 K_mul_residual(N, M, H) = mySwap(N, M) : matMul(N, M, M, 1, si.bus(N*M), residual)
4631 with {
4632   // residual = z - H @ x_pred, where z is Mx1 and H @ x_pred is Mx1
4633   residual = si.vecOp((si.bus(M), matMul(M, N, N, 1, H, si.bus(N))), -);
4634 };
4635
4636 // Updates the covariance matrix based on the Kalman gain and observation matrix.
4637 // * `N`: State size
4638 // * `M`: Measurement size
4639 // * `H`: Observation matrix (MxN)
4640 // * `K`: Kalman gain matrix (NxM)
4641 // * `P_pred`: Predicted covariance matrix (NxN)
4642 // No implicit arguments.
4643 updateCovariance(N, M, H, K, P_pred) =
4644   matMul(N, N, N, N, I_minus_KH, P_pred)
4645 with {
4646   // KH = K @ H, where K is NxM and H is MxN
4647   KH = matMul(N, M, M, N, K, H);
4648
4649   // I_minus_KH = I - K @ H, where both are NxN
4650   I_minus_KH = si.vecOp((la.identity(N), KH), -);
4651 };
4652
4653 // implicit arguments:
4654 // * `u`: Control input (Mx1)
4655 // * `z`: Measurement signal (Mx1)
4656 kalmanFilter(N, M, B, R, H, Q, F, reset) = (p, x, u, z <: pNew, xNew) ~ (pBus, xBus) :
4657   pCut, xBus
4658 with {
4659   doReset = reset > 0;
4660
4661   pBus = si.bus(N*N);
4662   pCut = par(i, N*N, !);
4663   pInit = la.diag(N, par(i, N, 100)); // large value for initial uncertainty in P.
4664   p = pBus, pInit : ba.selectbus(N*N, 2, doReset);
4665
4666   xBus = si.bus(N);
4667   xCut = par(i, N, !);
4668   xInit = par(i, N, 0); // initialize with zeros.
4669   x = xBus, xInit : ba.selectbus(N, 2, doReset);
4670
4671   u = si.bus(M);
4672   z = si.bus(M);
4673
4674   uCut = par(i, M, !);
4675   zCut = par(i, M, !);
4676
4677   predCov = predictCovariance(N, F, Q, pBus);
4678   gain = kalmanGain(N, M, H, R);
4679
4680   xNew = pBus, xBus, u, z : updateState(N, M, H, gain(predCov), predictState(N, M, F, B,
4681     xBus, u), z);
4682   pNew = pBus, xCut, uCut, zCut : predCov <: updateCovariance(N, M, H, gain(pBus), pBus)
4683     ;
4684 };
4685
4686 //-----`(fi.)kalman`-----
4687 // The Kalman filter. It returns the state (a bus of size `N`).

```

```

4686 // Note that the only compile-time constant arguments are `N` and `M`.
4687 // Other arguments are capitalized because they're matrices, and it makes
4688 // reading them much easier.
4689 //
4690 // #### Usage
4691 // ```
4692 // kalman(N, M, B, R, H, Q, F, reset, u, z) : si.bus(N)
4693 // ```
4694 //
4695 // Where:
4696 //
4697 // * `N`: State size (constant int)
4698 // * `M`: Measurement size (constant int)
4699 // * `B`: Control input matrix (NxM)
4700 // * `R`: Measurement noise covariance matrix (MxM)
4701 // * `H`: Observation matrix (MxN)
4702 // * `Q`: Process noise covariance matrix (NxN)
4703 // * `F`: State transition matrix (NxN)
4704 // * `reset`: Reset trigger. Whenever `reset>0`, the internal state `x` and covariance matrix
4705 //   `P` are reset.
4706 // * `u`: Control input (Mx1)
4707 // * `z`: Measurement signal (Mx1)
4708 //
4709 // #### Example test programs
4710 // Demo 1 `(N=1, M=1)` (don't listen, just use oscilloscope):
4711 // ```
4712 // process = fi.kalman(N, M, B, R, H, Q, F, reset, u, z) : it.interpolate_linear(filteredAmt,
4713 //   z)
4714 // with {
4715 //   B = 1.;
4716 //   R = 0.1;
4717 //   H = 1;
4718 //   Q = .01;
4719 //   F = la.identity(N);
4720 //   reset = button("reset");
4721 //
4722 //   // Dimensions
4723 //   N = 1; // State size
4724 //   M = 1; // Measurement size
4725 //
4726 //   freq = hslider("Freq", 1, 0.01, 10, .01);
4727 //   u = 0.; // constant input
4728 //   trueState = os.osc(freq)*.5 + u;
4729 //   noiseGain = hslider("Noise Gain", .1, 0, 1, .01);
4730 //
4731 //   filteredAmt = hslider("Filter Amount", 1, 0, 1, .01) : si.smoo;
4732 //
4733 //   measurementNoise = no.noise*noiseGain;
4734 //   z = trueState + measurementNoise; // Observed state
4735 // };
4736 // ```
4737 // Demo 2 `(N=2, M=1)` (don't listen, just use oscilloscope)
4738 // ```
4739 // process = fi.kalman(N, M, B, R, H, Q, F, reset, u, z)
4740 // with {
4741 //   B = par(i, N, 0);
4742 //   R = (0.1);
4743 //   H = (1, 0);
4744 //   Q = la.diag(2, par(i, N, .1));
4745 //   F = la.identity(N);
4746 //   reset = 0;
4747 //   u = si.bus(M);
4748 //   z = si.bus(M);
4749 //
4750 //   // Dimensions

```



```

4752 //      N = 2; // State size
4753 //      M = 1; // Measurement size
4754 // };
4755 // ...
4756 // #### References
4757 //
4758 // * <https://en.wikipedia.org/wiki/Kalman_filter>
4759 // * <https://www.cs.unc.edu/~welch/kalman/index.html>
4760 //-----
4761 declare kalman author "David Braun";
4762 declare kalman license "MIT License";
4763 kalman = kalmanEnv.kalmanFilter;
4764
4765
4766 /*****
4767 # Licenses
4768
4769 ## STK 4.3 License
4770
4771 Permission is hereby granted, free of charge, to any person obtaining a copy of
4772 this software and associated documentation files (the "Software"), to deal in
4773 the Software without restriction, including without limitation the rights to
4774 use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies
4775 of the Software, and to permit persons to whom the Software is furnished to do
4776 so, subject to the following conditions:
4777
4778 The above copyright notice and this permission notice shall be included in all
4779 copies or substantial portions of the Software.
4780
4781 Any person wishing to distribute modifications to the Software is asked to send
4782 the modifications to the original developer so that they can be incorporated
4783 into the canonical version. For software copyrighted by Julius O. Smith III,
4784 email your modifications to <jos@ccrma.stanford.edu>. This is, however, not a
4785 binding provision of this license.
4786
4787 THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
4788 IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
4789 FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
4790 AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
4791 LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
4792 OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
4793 SOFTWARE.
4794
4795 -----
4796
4797 ## LGPL License
4798
4799 This program is free software; you can redistribute it and/or modify it under
4800 the terms of the GNU Lesser General Public License as published by the Free
4801 Software Foundation; either version 2.1 of the License, or (at your option) any
4802 later version.
4803
4804 This program is distributed in the hope that it will be useful, but WITHOUT ANY
4805 WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A
4806 PARTICULAR PURPOSE. See the GNU Lesser General Public License for more details.
4807
4808 You should have received a copy of the GNU Lesser General Public License along
4809 with the GNU C Library; if not, write to the Free Software Foundation, Inc.,
4810 59 Temple Place, Suite 330, Boston, MA 02111-1307 USA.
4811
4812 -----
4813
4814 ## ISC License
4815
4816 Permission to use, copy, modify, and/or distribute this software for any
4817 purpose with or without fee is hereby granted, provided that the above
4818 copyright notice and this permission notice appear in all copies.
4819

```

```

4820 THE SOFTWARE IS PROVIDED "AS IS" AND THE AUTHOR DISCLAIMS ALL WARRANTIES WITH
4821 REGARD TO THIS SOFTWARE INCLUDING ALL IMPLIED WARRANTIES OF MERCHANTABILITY
4822 AND FITNESS. IN NO EVENT SHALL THE AUTHOR BE LIABLE FOR ANY SPECIAL, DIRECT,
4823 INDIRECT, OR CONSEQUENTIAL DAMAGES OR ANY DAMAGES WHATSOEVER RESULTING FROM
4824 LOSS OF USE, DATA OR PROFITS, WHETHER IN AN ACTION OF CONTRACT, NEGLIGENCE OR
4825 OTHER TORTIOUS ACTION, ARISING OUT OF OR IN CONNECTION WITH THE USE OR
4826 PERFORMANCE OF THIS SOFTWARE.
4827
4828 *****/

```

Listing 7: maths.lib

```

1 //##### maths.lib #####
2 // Maths library. Its official prefix is `ma`.
3 //
4 // This library provides mathematical functions and utilities for numerical
5 // computations in Faust. It includes trigonometric, exponential, logarithmic, and
6 // statistical functions, constants, and complex-number
7 // operations used throughout Faust DSP and control code.
8 //
9 // The Maths library is organized into 1 section:
10 //
11 // * [Functions Reference] (#functions-reference)
12 //
13 // Some functions are implemented as Faust foreign functions of `math.h` functions
14 // that are not part of Faust's primitives. Defines also various constants and several
15 // utilities.
16 //
17 // ### References
18 //
19 // * <https://github.com/grame-cncm/faustlibraries/blob/master/maths.lib>
20 //#####
21
22 // ## History
23 // * 06/13/2016 [RM] normalizing and integrating to new libraries
24 // * 07/08/2015 [YO] documentation comments
25 // * 20/06/2014 [SL] added FTZ function
26 // * 22/06/2013 [YO] added float/double/quad variants of some foreign functions
27 // * 28/06/2005 [YO] postfix functions with 'f' to force float version instead of double
28 // * 28/06/2005 [YO] removed 'modf' because it requires a pointer as argument
29
30 /*****
31 *****/
32 FAUST library file
33 Copyright (C) 2003-2016 GRAME, Centre National de Creation Musicale
34 -----
35 This program is free software; you can redistribute it and/or modify
36 it under the terms of the GNU Lesser General Public License as
37 published by the Free Software Foundation; either version 2.1 of the
38 License, or (at your option) any later version.
39
40 This program is distributed in the hope that it will be useful,
41 but WITHOUT ANY WARRANTY; without even the implied warranty of
42 MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
43 GNU Lesser General Public License for more details.
44
45 You should have received a copy of the GNU Lesser General Public
46 License along with the GNU C Library; if not, write to the Free
47 Software Foundation, Inc., 59 Temple Place, Suite 330, Boston, MA
48 02111-1307 USA.
49
50 EXCEPTION TO THE LGPL LICENSE : As a special exception, you may create a
51 larger FAUST program which directly or indirectly imports this library
52 file and still distribute the compiled code generated by the FAUST

```

```

53 compiler, or a modified version of this compiled code, under your own
54 copyright and license. This EXCEPTION TO THE LGPL LICENSE explicitly
55 grants you the right to freely choose the license for the resulting
56 compiled code. In particular the resulting compiled code has no obligation
57 to be LGPL or GPL. For example you are free to choose a commercial or
58 closed source license or any other license if you decide so.
59 ****
60 *****/
61
62 // This library contains platform specific constants
63 ba = library("basics.lib");
64 pl = library("platform.lib");
65 ma = library("maths.lib"); // for compatible copy/paste out of this file
66
67 declare name "Faust Math Library";
68 declare version "2.9.0";
69 declare author "GRAMME";
70 declare copyright "GRAMME";
71 declare license "LGPL with exception";
72
73 //=====Functions Reference=====
74 //=====
75
76
77 //-----`(ma.)SR`-----
78 // Current sampling rate given at init time. Constant during program execution.
79 //
80 // ### Usage
81 //
82 // ``
83 // SR : _
84 // ``
85 //
86 // Where:
87 //
88 // * `SR`: initialization-time sampling rate constant
89 //
90 // ### Test
91 //
92 // ``
93 // ma = library("maths.lib");
94 // SR_test = ma.SR;
95 // ``
96 //-----
97 SR = pl.SR;
98
99 //-----`(ma.)T`-----
100 // Current sample duration in seconds computed from the sampling rate given at init time.
101 // Constant during program execution.
102 //
103 // ### Usage
104 //
105 // ``
106 // T : _
107 // ``
108 //
109 // Where:
110 //
111 // * `T`: sample duration ( $1/SR$ ) constant
112 //
113 // ### Test
114 //
115 // ``
116 // ma = library("maths.lib");
117 // T_test = ma.T;
118 // ``
119 //-----
120 T = 1.0 / SR;
121
122 //-----`(ma.)BS`-----

```

```

120 // Current block-size. Can change during the execution at each block.
121 //
122 // #### Usage
123 //
124 // ```
125 // BS : _
126 // ```
127 //
128 // Where:
129 //
130 // * `BS`: current processing block size
131 //
132 // #### Test
133 //
134 // ma = library("maths.lib");
135 // BS_test = ma.BS;
136 // ```
137 //-----
138 BS = pl.BS;
139
140
141 //-----`(ma.)PI`-----
142 // Constant PI in double precision.
143 //
144 // #### Usage
145 //
146 // ```
147 // PI : _
148 // ```
149 //
150 // Where:
151 //
152 // * `PI`: double-precision constant
153 //
154 // #### Test
155 //
156 // ma = library("maths.lib");
157 // PI_test = ma.PI;
158 // ```
159 //-----
160 PI = 3.14159265358979323846;
161
162
163 //-----`(ma.)deg2rad`-----
164 // Convert degrees to radians.
165 //
166 // #### Usage
167 //
168 // ```
169 // 45. : deg2rad
170 // ```
171 //
172 // Where:
173 //
174 // * input: angle in degrees to convert
175 //
176 // #### Test
177 //
178 // ma = library("maths.lib");
179 // deg2rad_test = 45.0 : ma.deg2rad;
180 // ```
181 //-----
182 deg2rad = _ * PI / 180.;
183
184
185 //-----`(ma.)rad2deg`-----
186 // Convert radians to degrees.
187 //

```

```

188 // #### Usage
189 //
190 // ```
191 // ma.PI : rad2deg
192 // ```
193 //
194 // Where:
195 //
196 // * input: angle in radians to convert
197 //
198 // #### Test
199 // ```
200 // ma = library("maths.lib");
201 // rad2deg_test = ma.PI : ma.rad2deg;
202 // ```
203 //-----
204 rad2deg = _ * 180. / PI;
205
206
207 //-----`(ma.)E`-----
208 // Constant e in double precision.
209 //
210 // #### Usage
211 //
212 // ```
213 // E : _
214 // ```
215 //
216 // Where:
217 //
218 // * `E`: double-precision Euler's number constant
219 //
220 // #### Test
221 // ```
222 // ma = library("maths.lib");
223 // E_test = ma.E;
224 // ```
225 //-----
226 E = 2.71828182845904523536;
227
228
229 //-----`(ma.)EPSILON`-----
230 // Constant EPSILON available in simple/double/quad precision,
231 // as defined in the [floating-point standard](https://en.wikipedia.org/wiki/IEEE_754)
232 // and [machine epsilon](https://en.wikipedia.org/wiki/Machine_epsilon),
233 // that is smallest positive number such that `1.0 + EPSILON != 1.0`.
234 //
235 // #### Usage
236 //
237 // ```
238 // EPSILON : _
239 // ```
240 //
241 // Where:
242 //
243 // * `EPSILON`: machine epsilon constant for the current floating-point precision
244 //
245 // #### Test
246 // ```
247 // ma = library("maths.lib");
248 // EPSILON_test = ma.EPSILON;
249 // ```
250 //-----
251 singleprecision EPSILON = 1.192092896e-07;
252 doubleprecision EPSILON = 2.2204460492503131e-016;
253 quadprecision EPSILON = 1.084202172485504434007452e-019;
254 fixedpointprecision EPSILON = 2.2204460492503131e-016;
255

```

```

256 //-----`(ma.)MIN`-----
257 // Constant MIN available in simple/double/quad precision (minimal positive value).
258 //
259 // #### Usage
260 //
261 // ```
262 // MIN : _
263 // ```
264 //
265 // Where:
266 //
267 // * `MIN`: minimal positive normalized value for the current precision
268 //
269 // #### Test
270 //
271 // ```
272 // ma = library("maths.lib");
273 // MIN_test = ma.MIN * 1e307;
274 // ```
275 //-----
276 singleprecision MIN = 1.175494351e-38;
277 doubleprecision MIN = 2.2250738585072014e-308;
278 quadprecision MIN = 2.2250738585072014e-308;
279 fixedpointprecision MIN = 2.2250738585072014e-308;
280
281 //-----`(ma.)MAX`-----
282 // Constant MAX available in simple/double/quad precision (maximal positive value).
283 //
284 // #### Usage
285 //
286 // ```
287 // MAX : _
288 // ```
289 //
290 // Where:
291 //
292 // * `MAX`: maximal finite value for the current precision
293 //
294 // #### Test
295 //
296 // ```
297 // ma = library("maths.lib");
298 // MAX_test = ma.MAX;
299 // ```
300 //-----
301 singleprecision MAX = 3.402823466e+38;
302 doubleprecision MAX = 1.7976931348623158e+308;
303 quadprecision MAX = 1.7976931348623158e+308;
304 fixedpointprecision MAX = 1.7976931348623158e+308;
305
306 // Obsolete, kept for compatibility reasons
307 INFINITY = MAX;
308
309 //-----`(ma.)FTZ`-----
310 // Flush to zero: force samples under the "maximum subnormal number"
311 // to be zero. Usually not needed in C++ because the architecture
312 // file take care of this, but can be useful in JavaScript for instance.
313 //
314 // #### Usage
315 //
316 // ```
317 // _ : FTZ : _
318 // ```
319 //
320 // Where:
321 //
322 // * `x`: input signal to flush if its magnitude is subnormal
323 //

```

```

324 // #### Test
325 // ```
326 // ma = library("maths.lib");
327 // FTZ_test = ((ma.MIN * 0.5) : ma.FTZ), ((ma.MIN * 1e307) : ma.FTZ);
328 // ```
329 //
330 // #### References
331 //
332 // * <http://docs.oracle.com/cd/E19957-01/806-3568/ncg\_math.html>
333 //-----
334 FTZ(x) = x * (abs(x) > MIN);
335
336
337 //-----`(ma.)copysign`-----
338 // Changes the sign of x (first input) to that of y (second input).
339 //
340 // #### Usage
341 //
342 // ```
343 // _ : copysign : _
344 // ```
345 //
346 // Where:
347 //
348 // * `x`: value whose magnitude is preserved
349 // * `y`: value providing the sign
350 //
351 // #### Test
352 // ```
353 // ma = library("maths.lib");
354 // copysign_test = (-1.0, 2.0) : ma.copysign;
355 // ```
356 //-----
357 copysign = ffunction(float copysignf|copysign|copysignl (float, float), <math.h>,"");
358
359
360 //-----`(ma.)neg`-----
361 // Invert the sign (-x) of a signal.
362 //
363 // #### Usage
364 //
365 // ```
366 // _ : neg : _
367 // ```
368 //
369 // Where:
370 //
371 // * `x`: value to negate
372 //
373 // #### Test
374 // ```
375 // ma = library("maths.lib");
376 // neg_test = 3.5 : ma.neg;
377 // ```
378 //-----
379 neg(x) = -x;
380
381
382 //-----`(ma.)not`-----
383 // Bitwise `not` implemented with [xor](https://faustdoc.grame.fr/manual/syntax/#xor-primitive) as `not(x) = x xor -1;`.
384 // So working regardless of the size of the integer, assuming negative numbers in two's complement.
385 //
386 // #### Usage
387 //
388 // ```
389 // _ : not : _

```

```

390 // ``
391 //
392 // Where:
393 //
394 // * `x`: integer input value
395 //
396 // #### Test
397 // ``
398 // ma = library("maths.lib");
399 // not_test = 5 : ma.not;
400 // ``
401 //-----
402 not(x) = x xor -1;
403
404
405 //-----`(ma.)sub(x,y)`-----
406 // Subtract `x` and `y`.
407 //
408 // #### Usage
409 //
410 // ``
411 // _ : sub : _
412 // ``
413 //
414 // Where:
415 //
416 // * `x`: first operand
417 // * `y`: second operand
418 //
419 // #### Test
420 // ``
421 // ma = library("maths.lib");
422 // sub_test = (3, 10) : ma.sub;
423 // ``
424 //-----
425 sub(x,y) = y-x;
426
427
428 //-----`(ma.)inv`-----
429 // Compute the inverse (1/x) of the input signal.
430 //
431 // #### Usage
432 //
433 // ``
434 // _ : inv : _
435 // ``
436 //
437 // Where:
438 //
439 // * `x`: denominator input (non-zero)
440 //
441 // #### Test
442 // ``
443 // ma = library("maths.lib");
444 // inv_test = 4.0 : ma.inv;
445 // ``
446 //-----
447 inv(x) = 1/x;
448
449
450 //-----`(ma.)cbrt`-----
451 // Computes the cube root of of the input signal.
452 //
453 // #### Usage
454 //
455 // ``
456 // _ : cbrt : _
457 // ``

```



```

458 //
459 // Where:
460 //
461 // * `x`: value whose cube root is computed
462 //
463 // #### Test
464 // ```
465 // ma = library("maths.lib");
466 // cbrt_test = 8.0 : ma.cbrt;
467 // ```
468 //-----
469 cbrt = ffunction(float cbrtf|cbrt|cbrtl (float), <math.h>,"");
470
471
472 //-----`(ma.)hypot`-----
473 // Computes the euclidian distance of the two input signals
474 // sqrt(x*x+y*y) without undue overflow or underflow.
475 //
476 // #### Usage
477 //
478 // ```
479 // _ : hypot : _
480 // ```
481 //
482 // Where:
483 //
484 // * `x`: first operand
485 // * `y`: second operand
486 //
487 // #### Test
488 // ```
489 // ma = library("maths.lib");
490 // hypot_test = (3.0, 4.0) : ma.hypot;
491 // ```
492 //-----
493 hypot = ffunction(float hypotf|hypot|hypotl (float, float), <math.h>,"");
494
495
496 //-----`(ma.)ldexp`-----
497 // Takes two input signals: x and n, and multiplies x by 2 to the power n.
498 //
499 // #### Usage
500 //
501 // ```
502 // _ : ldexp : _
503 // ```
504 //
505 // Where:
506 //
507 // * `x`: significand input
508 // * `n`: exponent (integer) input
509 //
510 // #### Test
511 // ```
512 // ma = library("maths.lib");
513 // ldexp_test = (1.5, 3) : ma.ldexp;
514 // ```
515 //-----
516 ldexp = ffunction(float ldexpf|ldexp|ldexpl (float, int), <math.h>,"");
517
518
519 //-----`(ma.)scalb`-----
520 // Takes two input signals: x and n, and multiplies x by 2 to the power n.
521 //
522 // #### Usage
523 //
524 // ```
525 // _ : scalb : _

```

```

526 // ``
527 //
528 // Where:
529 //
530 // * `x`: significand input
531 // * `n`: exponent (integer) input
532 //
533 // #### Test
534 // ``
535 // ma = library("maths.lib");
536 // scalb_test = (2.0, -1) : ma.scalb;
537 // ``
538 //-----
539 scalb = ffunction(float scalbnf|scalbn|scalbnl (float, int), <math.h>,"");
540
541
542 //-----`(ma.)logip`-----
543 // Computes  $\log(1 + x)$  without undue loss of accuracy when  $x$  is nearly zero.
544 //
545 // #### Usage
546 //
547 // ``
548 // _ : logip : _
549 // ``
550 //
551 // Where:
552 //
553 // * `x`: offset used in  $\log(1 + x)$  (must be greater than -1)
554 //
555 // #### Test
556 // ``
557 // ma = library("maths.lib");
558 // logip_test = 0.5 : ma.logip;
559 // ``
560 //-----
561 logip = ffunction(float logipf|logip|logipl (float), <math.h>,"");
562
563
564 //-----`(ma.)logb`-----
565 // Return exponent of the input signal as a floating-point number.
566 //
567 // #### Usage
568 //
569 // ``
570 // _ : logb : _
571 // ``
572 //
573 // Where:
574 //
575 // * `x`: positive value whose exponent part is returned
576 //
577 // #### Test
578 // ``
579 // ma = library("maths.lib");
580 // logb_test = 8.0 : ma.logb;
581 // ``
582 //-----
583 logb = ffunction(float logbf|logb|logbl (float), <math.h>,"");
584
585
586 //-----`(ma.)ilogb`-----
587 // Return exponent of the input signal as an integer number.
588 //
589 // #### Usage
590 //
591 // ``
592 // _ : ilogb : _
593 // ``

```

```

594 //
595 // Where:
596 //
597 // * `x`: positive value whose exponent part is returned
598 //
599 // #### Test
600 // ```
601 // ma = library("maths.lib");
602 // ilogb_test = 8.0 : ma.ilogb;
603 // ```
604 //-----
605 ilogb = ffunction(int ilogbf|ilogb|ilogbl (float), <math.h>,"");
606
607
608 //-----`(ma.)log2`-----
609 // Returns the base 2 logarithm of x.
610 //
611 // #### Usage
612 //
613 // ```
614 // _ : log2 : _
615 // ```
616 //
617 // Where:
618 //
619 // * `x`: positive value whose base-2 logarithm is computed
620 //
621 // #### Test
622 // ```
623 // ma = library("maths.lib");
624 // log2_test = 8.0 : ma.log2;
625 // ```
626 //-----
627 log2(x) = log(x)/log(2.0);
628
629
630 //-----`(ma.)expm1`-----
631 // Return exponent of the input signal minus 1 with better precision.
632 //
633 // #### Usage
634 //
635 // ```
636 // _ : expm1 : _
637 // ```
638 //
639 // Where:
640 //
641 // * `x`: input value used for the `exp(x) - 1` computation
642 //
643 // #### Test
644 // ```
645 // ma = library("maths.lib");
646 // expm1_test = 0.5 : ma.expm1;
647 // ```
648 //-----
649 expm1 = ffunction(float expm1f|expm1|expm1l (float), <math.h>,"");
650
651
652 //-----`(ma.)acosh`-----
653 // Computes the principle value of the inverse hyperbolic cosine
654 // of the input signal.
655 //
656 // #### Usage
657 //
658 // ```
659 // _ : acosh : _
660 // ```
661 //

```

```

662 // Where:
663 //
664 // * `x`: input value (greater than or equal to 1)
665 //
666 // #### Test
667 // ```
668 // ma = library("maths.lib");
669 // acosh_test = 1.5 : ma.acosh;
670 // ```
671 //-----
672 acosh = ffunction(float acoshf|acosh|acoshl (float), <math.h>, "");
673
674
675 //-----`(ma.)asinh`-----
676 // Computes the inverse hyperbolic sine of the input signal.
677 //
678 // #### Usage
679 //
680 // ```
681 // _ : asinh : _
682 // ```
683 //
684 // Where:
685 //
686 // * `x`: input value
687 //
688 // #### Test
689 // ```
690 // ma = library("maths.lib");
691 // asinh_test = 0.5 : ma.asinh;
692 // ```
693 //-----
694 asinh = ffunction(float asinhf|asinh|asinh1 (float), <math.h>, "");
695
696
697 //-----`(ma.)atanh`-----
698 // Computes the inverse hyperbolic tangent of the input signal.
699 //
700 // #### Usage
701 //
702 // ```
703 // _ : atanh : _
704 // ```
705 //
706 // Where:
707 //
708 // * `x`: input value in (-1, 1)
709 //
710 // #### Test
711 // ```
712 // ma = library("maths.lib");
713 // atanh_test = 0.5 : ma.atanh;
714 // ```
715 //-----
716 atanh = ffunction(float atanhf|atanh|atanh1 (float), <math.h>, "");
717
718
719 //-----`(ma.)sinh`-----
720 // Computes the hyperbolic sine of the input signal.
721 //
722 // #### Usage
723 //
724 // ```
725 // _ : sinh : _
726 // ```
727 //
728 // Where:
729 //

```

```

730 // * `x`: input value
731 //
732 // #### Test
733 // ```
734 // ma = library("maths.lib");
735 // sinh_test = 0.5 : ma.sinh;
736 // ```
737 //-----
738 sinh = ffunction(float sinhf|sinh|sinhl (float), <math.h>, "");
739
740
741 //-----`(ma.)cosh`-----
742 // Computes the hyperbolic cosine of the input signal.
743 //
744 // #### Usage
745 //
746 // ```
747 // _ : cosh : _
748 // ```
749 //
750 // Where:
751 //
752 // * `x`: input value
753 //
754 // #### Test
755 // ```
756 // ma = library("maths.lib");
757 // cosh_test = 0.5 : ma.cosh;
758 // ```
759 //-----
760 cosh = ffunction(float coshf|cosh|coshl (float), <math.h>, "");
761
762
763 //-----`(ma.)tanh`-----
764 // Computes the hyperbolic tangent of the input signal.
765 //
766 // #### Usage
767 //
768 // ```
769 // _ : tanh : _
770 // ```
771 //
772 // Where:
773 //
774 // * `x`: input value
775 //
776 // #### Test
777 // ```
778 // ma = library("maths.lib");
779 // tanh_test = 0.5 : ma.tanh;
780 // ```
781 //-----
782 tanh = ffunction(float tanhf|tanh|tanhl (float), <math.h>,"");
783
784
785 //-----`(ma.)erf`-----
786 // Computes the error function of the input signal.
787 //
788 // #### Usage
789 //
790 // ```
791 // _ : erf : _
792 // ```
793 //
794 // Where:
795 //
796 // * `x`: input value
797 //

```

```

798 // #### Test
799 // ...
800 // ma = library("maths.lib");
801 // erf_test = 0.5 : ma.erf;
802 // ...
803 //-----
804 erf = ffunction(float erff|erf|erfl(float), <math.h>,"");
805
806
807 //-----`(ma.)erfc`-----
808 // Computes the complementary error function of the input signal.
809 //
810 // #### Usage
811 //
812 // ...
813 // _ : erfc : _
814 // ...
815 //
816 // Where:
817 //
818 // * `x`: input value
819 //
820 // #### Test
821 // ...
822 // ma = library("maths.lib");
823 // erfc_test = 0.5 : ma.erfc;
824 // ...
825 //-----
826 erfc = ffunction(float erfcf|erfc|erfcl(float), <math.h>,"");
827
828
829 //-----`(ma.)gamma`-----
830 // Computes the gamma function of the input signal.
831 //
832 // #### Usage
833 //
834 // ...
835 // _ : gamma : _
836 // ...
837 //
838 // Where:
839 //
840 // * `x`: positive input value
841 //
842 // #### Test
843 // ...
844 // ma = library("maths.lib");
845 // gamma_test = 3.0 : ma.gamma;
846 // ...
847 //-----
848 gamma = ffunction(float tgammaf|tgamma|tgammal(float), <math.h>,"");
849
850
851 //-----`(ma.)lgamma`-----
852 // Calculates the natural logarithm of the absolute value of
853 // the gamma function of the input signal.
854 //
855 // #### Usage
856 //
857 // ...
858 // _ : lgamma : _
859 // ...
860 //
861 // Where:
862 //
863 // * `x`: positive input value
864 //
865 // #### Test

```

```

866 // ``
867 // ma = library("maths.lib");
868 // lgamma_test = 3.0 : ma.lgamma;
869 // ``
870 //-----
871 lgamma = ffunction(float lgammaf|lgamma|lgammal(float), <math.h>,"");
872
873
874 //-----`(ma.)J0`-----
875 // Computes the Bessel function of the first kind of order 0
876 // of the input signal.
877 //
878 // #### Usage
879 //
880 // ``
881 // _ : J0 : _
882 // ``
883 //
884 // Where:
885 //
886 // * `x`: input value
887 //
888 // #### Test
889 //
890 // ma = library("maths.lib");
891 // J0_test = 1.0 : ma.J0;
892 // ``
893 //-----
894 J0 = ffunction(float j0(float), <math.h>,"");
895
896
897 //-----`(ma.)J1`-----
898 // Computes the Bessel function of the first kind of order 1
899 // of the input signal.
900 //
901 // #### Usage
902 //
903 // ``
904 // _ : J1 : _
905 // ``
906 //
907 // Where:
908 //
909 // * `x`: input value
910 //
911 // #### Test
912 //
913 // ma = library("maths.lib");
914 // J1_test = 1.0 : ma.J1;
915 // ``
916 //-----
917 J1 = ffunction(float j1(float), <math.h>,"");
918
919
920 //-----`(ma.)Jn`-----
921 // Computes the Bessel function of the first kind of order n
922 // (first input signal) of the second input signal.
923 //
924 // #### Usage
925 //
926 // ``
927 // _ : Jn : _
928 // ``
929 //
930 // Where:
931 //
932 // * `n`: integer order
933 // * `x`: input value

```

```

934 //
935 // #### Test
936 // ```
937 // ma = library("maths.lib");
938 // Jn_test = (2, 1.0) : ma.Jn;
939 // ```
940 //-----
941 Jn = ffunction(float jn(int, float), <math.h>,"");
942
943
944 //-----`(ma.)Y0`-----
945 // Computes the linearly independent Bessel function of the second kind
946 // of order 0 of the input signal.
947 //
948 // #### Usage
949 //
950 // ```
951 // _ : Y0 : _
952 // ```
953 //
954 // Where:
955 //
956 // * `x`: positive input value
957 //
958 // #### Test
959 // ```
960 // ma = library("maths.lib");
961 // Y0_test = 1.0 : ma.Y0;
962 // ```
963 //-----
964 Y0 = ffunction(float y0(float), <math.h>,"");
965
966
967 //-----`(ma.)Y1`-----
968 // Computes the linearly independent Bessel function of the second kind
969 // of order 1 of the input signal.
970 //
971 // #### Usage
972 //
973 // ```
974 // _ : Y0 : _
975 // ```
976 //
977 // Where:
978 //
979 // * `x`: positive input value
980 //
981 // #### Test
982 // ```
983 // ma = library("maths.lib");
984 // Y1_test = 1.0 : ma.Y1;
985 // ```
986 //-----
987 Y1 = ffunction(float y1(float), <math.h>,"");
988
989
990 //-----`(ma.)Yn`-----
991 // Computes the linearly independent Bessel function of the second kind
992 // of order n (first input signal) of the second input signal.
993 //
994 // #### Usage
995 //
996 // ```
997 // _ : Yn : _
998 // ```
999 //
1000 // Where:
1001 //

```



```

1002 // * `n`: integer order
1003 // * `x`: positive input value
1004 //
1005 // #### Test
1006 // ```
1007 // ma = library("maths.lib");
1008 // Yn_test = (2, 1.0) : ma.Yn;
1009 // ```
1010 //-----
1011 Yn = ffunction(float yn(int, float), <math.h>,"");
1012
1013
1014 //-----`(ma.)fabs`, `(ma.)fmax`, `(ma.)fmin`
1015 // Just for compatibility...
1016 //
1017 // ```
1018 // fabs = abs
1019 // fmax = max
1020 // fmin = min
1021 // ```
1022 //-----
1023 fabs = abs;
1024 fmax = max;
1025 fmin = min;
1026
1027 //-----`(ma.)np2`-----
1028 // Gives the next power of 2 of x.
1029 //
1030 // #### Usage
1031 //
1032 // ```
1033 // np2(n) : _
1034 // ```
1035 //
1036 // Where:
1037 //
1038 // * `n`: an integer
1039 //
1040 // #### Test
1041 // ```
1042 // ma = library("maths.lib");
1043 // np2_test = 5 : ma.np2;
1044 // ```
1045 //-----
1046 np2 = -(1) <: >>(1)|_ <: >>(2)|_ <: >>(4)|_ <: >>(8)|_ <: >>(16)|_ : +(1);
1047
1048
1049 //-----`(ma.)frac`-----
1050 // Gives the fractional part of n.
1051 //
1052 // #### Usage
1053 //
1054 // ```
1055 // frac(n) : _
1056 // ```
1057 //
1058 // Where:
1059 //
1060 // * `n`: a decimal number
1061 //
1062 // #### Test
1063 // ```
1064 // ma = library("maths.lib");
1065 // frac_test = 3.75 : ma.frac;
1066 // ```
1067 //-----
1068 frac(n) = n - floor(n);

```

```

1069 decimal = frac;
1070
1071 // NOTE: decimal does the same thing as frac but using floor instead. JOS uses frac a lot
1072 // in filters.lib so we decided to keep that one... decimal is declared though for
1073 // backward compatibility.
1074 // decimal(n) = n - floor(n);
1075
1076
1077 //-----`(ma.)modulo`-----
1078 // Modulus operation using the `(x%y+y)%y` formula to ensures the result is always non-
// negative, even if `x` is negative.
1079 //
1080 // ### Usage
1081 //
1082 // ```
1083 // modulo(x,y) : _
1084 // ```
1085 //
1086 // Where:
1087 //
1088 // * `x`: the numerator
1089 // * `y`: the denominator
1090 //
1091 // ### Test
1092 // ```
1093 // ma = library("maths.lib");
1094 // modulo_test = (-3, 4) : ma.modulo;
1095 // ```
1096 //-----
1097 modulo(x,y) = (x % y + y) % y;
1098
1099
1100 //-----`(ma.)isnan`-----
1101 // Return non-zero if x is a NaN.
1102 //
1103 // ### Usage
1104 //
1105 // ```
1106 // isnan(x)
1107 // _ : isnan : _
1108 // ```
1109 //
1110 // Where:
1111 //
1112 // * `x`: signal to analyse
1113 //
1114 // ### Test
1115 // ```
1116 // ma = library("maths.lib");
1117 // os = library("oscillators.lib");
1118 // isnan_test = (os.osc(1) - 2.0) : sqrt : ma.isnan;
1119 // ```
1120 //-----
1121 isnan = ffunction(int isnanf|isnan|isnanl (float),<math.h>,"");
1122
1123
1124 //-----`(ma.)isinf`-----
1125 // Return non-zero if x is a positive or negative infinity.
1126 //
1127 // ### Usage
1128 //
1129 // ```
1130 // isinf(x)
1131 // _ : isinf : _
1132 // ```
1133 //
1134 // Where:
1135 //

```

```

1136 // * `x`: signal to analyse
1137 //
1138 // #### Test
1139 // ```
1140 // ma = library("maths.lib");
1141 // os = library("oscillators.lib");
1142 // isinf_test = (os.impulse - os.impulse) : log : ma.isinf;
1143 // ```
1144 //-----
1145 isinf = ffunction(int isinff|isinfl(float),<math.h>,"");
1146
1147 nextafter = ffunction(float nextafter(float, float),<math.h>,"");
1148
1149
1150 //-----`(ma.)chebychev`-----
1151 // Chebychev transformation of order N.
1152 //
1153 // #### Usage
1154 //
1155 // ```
1156 // _ : chebychev(N) : _
1157 // ```
1158 //
1159 // Where:
1160 //
1161 // * `N`: the order of the polynomial, a constant numerical expression
1162 //
1163 // #### Semantics
1164 //
1165 // ```
1166 // T[0](x) = 1,
1167 // T[1](x) = x,
1168 // T[n](x) = 2x*T[n-1](x) - T[n-2](x)
1169 // ```
1170 //
1171 // #### Test
1172 // ```
1173 // ma = library("maths.lib");
1174 // chebychev_test = 0.5 : ma.chebychev(3);
1175 // ```
1176 //
1177 // #### References
1178 //
1179 // <http://en.wikipedia.org/wiki/Chebyshev\_polynomial>
1180 //-----
1181 chebychev(0,x) = 1;
1182 chebychev(1,x) = x;
1183 chebychev(n,x) = 2*x*chebychev(n-1, x) - chebychev(n-2, x);
1184
1185
1186 //-----`(ma.)chebychevpoly`-----
1187 // Linear combination of the first Chebyshev polynomials.
1188 //
1189 // #### Usage
1190 //
1191 // ```
1192 // _ : chebychevpoly((c0,c1,...,cn)) : _
1193 // ```
1194 //
1195 // Where:
1196 //
1197 // * `cn`: the different Chebyshev polynomials such that:
1198 // chebychevpoly((c0,c1,...,cn)) = Sum of chebychev(i)*ci
1199 //
1200 // #### Test
1201 // ```
1202 // ma = library("maths.lib");
1203 // chebychevpoly_test = 0.5 : ma.chebychevpoly((1, 0, 1));

```

```

1204 // ``
1205 //
1206 // #### References
1207 //
1208 // * <http://www.csounds.com/manual/html/chebyshevpoly.html>
1209 //-----
1210 chebychevpoly(lcoef) = _ <: L(0,lcoef) :> _
1211   with {
1212     L(n,(c,cs)) = chebychev(n)*c, L(n+1,cs);
1213     L(n,c)      = chebychev(n)*c;
1214   };
1215
1216 //-----`ma.)diffn`-----
1217 // Negated first-order difference.
1218 //
1219 // #### Usage
1220 //
1221 // ``
1222 // _ : diffn : _
1223 // ``
1224 //
1225 //
1226 // Where:
1227 //
1228 // * `x`: input signal
1229 //
1230 // #### Test
1231 // ``
1232 // ma = library("maths.lib");
1233 // os = library("oscillators.lib");
1234 // diffn_test = os.osc(440) : ma.diffn;
1235 // ``
1236 //-----
1237 diffn(x) = x' - x; // negated first-order difference
1238
1239 //-----`ma.)signum`-----
1240 // The signum function signum(x) is defined as
1241 // -1 for x<0, 0 for x==0, and 1 for x>0.
1242 //
1243 // #### Usage
1244 //
1245 // ``
1246 // _ : signum : _
1247 // ``
1248 //
1249 // Where:
1250 //
1251 // * `x`: input value
1252 //
1253 // #### Test
1254 // ``
1255 // ma = library("maths.lib");
1256 // signum_test = (-5.0) : ma.signum;
1257 // ``
1258 //-----
1259 signum(x) = (x>0)-(x<0);
1260
1261 //-----`ma.)nextpow2`-----
1262 // The nextpow2(x) returns the lowest integer m such that
1263 // 2^m >= x.
1264 //
1265 // #### Usage
1266 //
1267 // ``
1268 // 2^nextpow2(n) : _
1269 // ``
1270 //
1271 //

```

```

1272 // Useful for allocating delay lines, e.g.,
1273 // ```
1274 // delay(2^nextpow2(maxDelayNeeded), currentDelay);
1275 // ```
1276 //
1277 // Where:
1278 //
1279 // * `n`: positive value whose next power-of-two exponent is computed
1280 //
1281 // #### Test
1282 // ```
1283 // ma = library("maths.lib");
1284 // nextpow2_test = 10.0 : ma.nextpow2;
1285 // ```
1286 //-----
1287 nextpow2(x) = ceil(log(x)/log(2.0));
1288
1289
1290 //-----`(ma.)zc`-----
1291 // Indicator function for zero-crossing: it returns 1 if a zero-crossing
1292 // occurs, 0 otherwise.
1293 //
1294 // #### Usage
1295 //
1296 // ```
1297 // _ : zc : _
1298 // ```
1299 //
1300 // Where:
1301 //
1302 // * `x`: input signal to monitor for zero crossings
1303 //
1304 // #### Test
1305 // ```
1306 // ma = library("maths.lib");
1307 // os = library("oscillators.lib");
1308 // zc_test = os.osc(440) : ma.zc;
1309 // ```
1310 //-----
1311 zc = ba.tAndH(!=(0)) <: *(mem) < 0;
1312
1313 //-----`(ma.)unwrap`-----
1314 // Unwrap the input signal so that successive output values never differ
1315 // by more than `pi`, switching to a 2*pi-complementary value when needed.
1316 //
1317 // #### Usage
1318 //
1319 // ```
1320 // _ : unwrap(pi) : _
1321 // ```
1322 //
1323 // Where:
1324 //
1325 // * `pi`: maximum discontinuity between the output values (typically `ma.PI`)
1326 //
1327 // #### Test
1328 // ```
1329 // ma = library("maths.lib");
1330 // os = library("oscillators.lib");
1331 // unwrap_test = os.oscsrc(100) : ma.unwrap(ma.PI);
1332 // ```
1333 //
1334 // #### Example test program
1335 //
1336 // ```
1337 // process = 0 - os.oscsrc(1000) // the true phase is either -PI or +PI
1338 // : an.resonator(1,1000) : !,_ // oscillates between -PI and +PI
1339 // : ma.unwrap(ma.PI); // oscillates near +PI

```

```

1340 // ---
1341 //
1342 //-----
1343 unwrap(pi,x) = (_ <: +(dz)) ~ _ with {
1344     dz(z) = wrap(x - z, 2*pi) <: ba.ifNc(1, >(pi),-(2*pi));
1345     wrap(x,d) = x - d * floor(x / d);
1346 };
1347
1348 //-----`ma.`primes`-----
1349 // Return the n-th prime using a waveform primitive. Note that primes(0) is 2,
1350 // primes(1) is 3, and so on. The waveform is length 2048, so the largest
1351 // precomputed prime is primes(2047) which is 17863.
1352 //
1353 // #### Usage
1354 //
1355 // ---
1356 // _ : primes : _
1357 // ---
1358 //
1359 // Where:
1360 //
1361 // * `x`: index of the prime number sequence (0-based).
1362 //
1363 // #### Test
1364 // ---
1365 // ma = library("maths.lib");
1366 // primes_test = 10 : ma.primes;
1367 // ---
1368 //-----
1369 primes(x) = rdttable(waveform{
1370     2,3,5,7,11,13,17,19,23,29,
1371     31,37,41,43,47,53,59,61,67,71,
1372     73,79,83,89,97,101,103,107,109,113,
1373     127,131,137,139,149,151,157,163,167,173,
1374     179,181,191,193,197,199,211,223,227,229,
1375     233,239,241,251,257,263,269,271,277,281,
1376     283,293,307,311,313,317,331,337,347,349,
1377     353,359,367,373,379,383,389,397,401,409,
1378     419,421,431,433,439,443,449,457,461,463,
1379     467,479,487,491,499,503,509,521,523,541,
1380     547,557,563,569,571,577,587,593,599,601,
1381     607,613,617,619,631,641,643,647,653,659,
1382     661,673,677,683,691,701,709,719,727,733,
1383     739,743,751,757,761,769,773,787,797,809,
1384     811,821,823,827,829,839,853,857,859,863,
1385     877,881,883,887,907,911,919,929,937,941,
1386     947,953,967,971,977,983,991,997,1009,1013,
1387     1019,1021,1031,1033,1039,1049,1051,1061,1063,1069,
1388     1087,1091,1093,1097,1103,1109,1117,1123,1129,1151,
1389     1153,1163,1171,1181,1187,1193,1201,1213,1217,1223,
1390     1229,1231,1237,1249,1259,1277,1279,1283,1289,1291,
1391     1297,1301,1303,1307,1319,1321,1327,1361,1367,1373,
1392     1381,1399,1409,1423,1427,1429,1433,1439,1447,1451,
1393     1453,1459,1471,1481,1483,1487,1489,1493,1499,1511,
1394     1523,1531,1543,1549,1553,1559,1567,1571,1579,1583,
1395     1597,1601,1607,1609,1613,1619,1621,1627,1637,1657,
1396     1663,1667,1669,1693,1697,1699,1709,1721,1723,1733,
1397     1741,1747,1753,1759,1777,1783,1787,1789,1801,1811,
1398     1823,1831,1847,1861,1867,1871,1873,1877,1879,1889,
1399     1901,1907,1913,1931,1933,1949,1951,1973,1979,1987,
1400     1993,1997,1999,2003,2011,2017,2027,2029,2039,2053,
1401     2063,2069,2081,2083,2087,2089,2099,2111,2113,2129,
1402     2131,2137,2141,2143,2153,2161,2179,2203,2207,2213,
1403     2221,2237,2239,2243,2251,2267,2269,2273,2281,2287,
1404     2293,2297,2309,2311,2333,2339,2341,2347,2351,2357,
1405     2371,2377,2381,2383,2389,2393,2399,2411,2417,2423,
1406     2437,2441,2447,2459,2467,2473,2477,2503,2521,2531,
1407     2539,2543,2549,2551,2557,2579,2591,2593,2609,2617,

```

1408	2621,2633,2647,2657,2659,2663,2671,2677,2683,2687,
1409	2689,2693,2699,2707,2711,2713,2719,2729,2731,2741,
1410	2749,2753,2767,2777,2789,2791,2797,2801,2803,2819,
1411	2833,2837,2843,2851,2857,2861,2879,2887,2897,2903,
1412	2909,2917,2927,2939,2953,2957,2963,2969,2971,2999,
1413	3001,3011,3019,3023,3037,3041,3049,3061,3067,3079,
1414	3083,3089,3109,3119,3121,3137,3163,3167,3169,3181,
1415	3187,3191,3203,3209,3217,3221,3229,3251,3253,3257,
1416	3259,3271,3299,3301,3307,3313,3319,3323,3329,3331,
1417	3343,3347,3359,3361,3371,3373,3389,3391,3407,3413,
1418	3433,3449,3457,3461,3463,3467,3469,3491,3499,3511,
1419	3517,3527,3529,3533,3539,3541,3547,3557,3559,3571,
1420	3581,3583,3593,3607,3613,3617,3623,3631,3637,3643,
1421	3659,3671,3673,3677,3691,3697,3701,3709,3719,3727,
1422	3733,3739,3761,3767,3769,3779,3793,3797,3803,3821,
1423	3823,3833,3847,3851,3853,3863,3877,3881,3889,3907,
1424	3911,3917,3919,3923,3929,3931,3943,3947,3967,3989,
1425	4001,4003,4007,4013,4019,4021,4027,4049,4051,4057,
1426	4073,4079,4091,4093,4099,4111,4127,4129,4133,4139,
1427	4153,4157,4159,4177,4201,4211,4217,4219,4229,4231,
1428	4241,4243,4253,4259,4261,4271,4273,4283,4289,4297,
1429	4327,4337,4339,4349,4357,4363,4373,4391,4397,4409,
1430	4421,4423,4441,4447,4451,4457,4463,4481,4483,4493,
1431	4507,4513,4517,4519,4523,4547,4549,4561,4567,4583,
1432	4591,4597,4603,4621,4637,4639,4643,4649,4651,4657,
1433	4663,4673,4679,4691,4703,4721,4723,4729,4733,4751,
1434	4759,4783,4787,4789,4793,4799,4801,4813,4817,4831,
1435	4861,4871,4877,4889,4903,4909,4919,4931,4933,4937,
1436	4943,4951,4957,4967,4969,4973,4987,4993,4999,5003,
1437	5009,5011,5021,5023,5039,5051,5059,5077,5081,5087,
1438	5099,5101,5107,5113,5119,5147,5153,5167,5171,5179,
1439	5189,5197,5209,5227,5231,5233,5237,5261,5273,5279,
1440	5281,5297,5303,5309,5323,5333,5347,5351,5381,5387,
1441	5393,5399,5407,5413,5417,5419,5431,5437,5441,5443,
1442	5449,5471,5477,5479,5483,5501,5503,5507,5519,5521,
1443	5527,5531,5557,5563,5569,5573,5581,5591,5623,5639,
1444	5641,5647,5651,5653,5657,5659,5669,5683,5689,5693,
1445	5701,5711,5717,5737,5741,5743,5749,5779,5783,5791,
1446	5801,5807,5813,5821,5827,5839,5843,5849,5851,5857,
1447	5861,5867,5869,5879,5881,5897,5903,5923,5927,5939,
1448	5953,5981,5987,6007,6011,6029,6037,6043,6047,6053,
1449	6067,6073,6079,6089,6091,6101,6113,6121,6131,6133,
1450	6143,6151,6163,6173,6197,6199,6203,6211,6217,6221,
1451	6229,6247,6257,6263,6269,6271,6277,6287,6299,6301,
1452	6311,6317,6323,6329,6337,6343,6353,6359,6361,6367,
1453	6373,6379,6389,6397,6421,6427,6449,6451,6469,6473,
1454	6481,6491,6521,6529,6547,6551,6553,6563,6569,6571,
1455	6577,6581,6599,6607,6619,6637,6653,6659,6661,6673,
1456	6679,6689,6691,6701,6703,6709,6719,6733,6737,6761,
1457	6763,6779,6781,6791,6793,6803,6823,6827,6829,6833,
1458	6841,6857,6863,6869,6871,6883,6899,6907,6911,6917,
1459	6947,6949,6959,6961,6967,6971,6977,6983,6991,6997,
1460	7001,7013,7019,7027,7039,7043,7057,7069,7079,7103,
1461	7109,7121,7127,7129,7151,7159,7177,7187,7193,7207,
1462	7211,7213,7219,7229,7237,7243,7247,7253,7283,7297,
1463	7307,7309,7321,7331,7333,7349,7351,7369,7393,7411,
1464	7417,7433,7451,7457,7459,7477,7481,7487,7489,7499,
1465	7507,7517,7523,7529,7537,7541,7547,7549,7559,7561,
1466	7573,7577,7583,7589,7591,7603,7607,7621,7639,7643,
1467	7649,7669,7673,7681,7687,7691,7699,7703,7717,7723,
1468	7727,7741,7753,7757,7759,7789,7793,7817,7823,7829,
1469	7841,7853,7867,7873,7877,7879,7883,7901,7907,7919,
1470	7927,7933,7937,7949,7951,7963,7993,8009,8011,8017,
1471	8039,8053,8059,8069,8081,8087,8089,8093,8101,8111,
1472	8117,8123,8147,8161,8167,8171,8179,8191,8209,8219,
1473	8221,8231,8233,8237,8243,8263,8269,8273,8287,8291,
1474	8293,8297,8311,8317,8329,8353,8363,8369,8377,8387,
1475	8389,8419,8423,8429,8431,8443,8447,8461,8467,8501,

1476 8513,8521,8527,8537,8539,8543,8563,8573,8581,8597,
1477 8599,8609,8623,8627,8629,8641,8647,8663,8669,8677,
1478 8681,8689,8693,8699,8707,8713,8719,8731,8737,8741,
1479 8747,8753,8761,8779,8783,8803,8807,8819,8821,8831,
1480 8837,8839,8849,8861,8863,8867,8887,8893,8923,8929,
1481 8933,8941,8951,8963,8969,8971,8999,9001,9007,9011,
1482 9013,9029,9041,9043,9049,9059,9067,9091,9103,9109,
1483 9127,9133,9137,9151,9157,9161,9173,9181,9187,9199,
1484 9203,9209,9221,9227,9239,9241,9257,9277,9281,9283,
1485 9293,9311,9319,9323,9337,9341,9343,9349,9371,9377,
1486 9391,9397,9403,9413,9419,9421,9431,9433,9437,9439,
1487 9461,9463,9467,9473,9479,9491,9497,9511,9521,9533,
1488 9539,9547,9551,9587,9601,9613,9619,9623,9629,9631,
1489 9643,9649,9661,9677,9679,9689,9697,9719,9721,9733,
1490 9739,9743,9749,9767,9769,9781,9787,9791,9803,9811,
1491 9817,9829,9833,9839,9851,9857,9859,9871,9883,9887,
1492 9901,9907,9923,9929,9931,9941,9949,9967,9973,10007,
1493 10009,10037,10039,10061,10067,10069,10079,10091,10093,10099,
1494 10103,10111,10133,10139,10141,10151,10159,10163,10169,10177,
1495 10181,10193,10211,10223,10243,10247,10253,10259,10267,10271,
1496 10273,10289,10301,10303,10313,10321,10331,10333,10337,10343,
1497 10357,10369,10391,10399,10427,10429,10433,10453,10457,10459,
1498 10463,10477,10487,10499,10501,10513,10529,10531,10559,10567,
1499 10589,10597,10601,10607,10613,10627,10631,10639,10651,10657,
1500 10663,10667,10687,10691,10709,10711,10723,10729,10733,10739,
1501 10753,10771,10781,10789,10799,10831,10837,10847,10853,10859,
1502 10861,10867,10883,10889,10891,10903,10909,10937,10939,10949,
1503 10957,10973,10979,10987,10993,11003,11027,11047,11057,11059,
1504 11069,11071,11083,11087,11093,11113,11117,11119,11131,11149,
1505 11159,11161,11171,11173,11177,11197,11213,11239,11243,11251,
1506 11257,11261,11273,11279,11287,11299,11311,11317,11321,11329,
1507 11351,11353,11369,11383,11393,11399,11411,11423,11437,11443,
1508 11447,11467,11471,11483,11489,11491,11497,11503,11519,11527,
1509 11549,11551,11579,11587,11593,11597,11617,11621,11633,11657,
1510 11677,11681,11689,11699,11701,11717,11719,11731,11743,11777,
1511 11779,11783,11789,11801,11807,11813,11821,11827,11831,11833,
1512 11839,11863,11867,11887,11897,11903,11909,11923,11927,11933,
1513 11939,11941,11953,11959,11969,11971,11981,11987,12007,12011,
1514 12037,12041,12043,12049,12071,12073,12097,12101,12107,12109,
1515 12113,12119,12143,12149,12157,12161,12163,12197,12203,12211,
1516 12227,12239,12241,12251,12253,12263,12269,12277,12281,12289,
1517 12301,12323,12329,12343,12347,12373,12377,12379,12391,12401,
1518 12409,12413,12421,12433,12437,12451,12457,12473,12479,12487,
1519 12491,12497,12503,12511,12517,12527,12539,12541,12547,12553,
1520 12569,12577,12583,12589,12601,12611,12613,12619,12637,12641,
1521 12647,12653,12659,12671,12689,12697,12703,12713,12721,12739,
1522 12743,12757,12763,12781,12791,12799,12809,12821,12823,12829,
1523 12841,12853,12889,12893,12899,12907,12911,12917,12919,12923,
1524 12941,12953,12959,12967,12973,12979,12983,13001,13003,13007,
1525 13009,13033,13037,13043,13049,13063,13093,13099,13103,13109,
1526 13121,13127,13147,13151,13159,13163,13171,13177,13183,13187,
1527 13217,13219,13229,13241,13249,13259,13267,13291,13297,13309,
1528 13313,13327,13331,13337,13339,13367,13381,13397,13399,13411,
1529 13417,13421,13441,13451,13457,13463,13469,13477,13487,13499,
1530 13513,13523,13537,13553,13567,13577,13591,13597,13613,13619,
1531 13627,13633,13649,13669,13679,13681,13687,13691,13693,13697,
1532 13709,13711,13721,13723,13729,13751,13757,13759,13763,13781,
1533 13789,13799,13807,13829,13831,13841,13859,13873,13877,13879,
1534 13883,13901,13903,13907,13913,13921,13931,13933,13963,13967,
1535 13997,13999,14009,14011,14029,14033,14051,14057,14071,14081,
1536 14083,14087,14107,14143,14149,14153,14159,14173,14177,14197,
1537 14207,14221,14243,14249,14251,14281,14293,14303,14321,14323,
1538 14327,14341,14347,14369,14387,14389,14401,14407,14411,14419,
1539 14423,14431,14437,14447,14449,14461,14479,14489,14503,14519,
1540 14533,14537,14543,14549,14551,14557,14561,14563,14591,14593,
1541 14621,14627,14629,14633,14639,14653,14657,14669,14683,14699,
1542 14713,14717,14723,14731,14737,14741,14747,14753,14759,14767,
1543 14771,14779,14783,14797,14813,14821,14827,14831,14843,14851,


```

1544 14867,14869,14879,14887,14891,14897,14923,14929,14939,14947,
1545 14951,14957,14969,14983,15013,15017,15031,15053,15061,15073,
1546 15077,15083,15091,15101,15107,15121,15131,15137,15139,15149,
1547 15161,15173,15187,15193,15199,15217,15227,15233,15241,15259,
1548 15263,15269,15271,15277,15287,15289,15299,15307,15313,15319,
1549 15329,15331,15349,15359,15361,15373,15377,15383,15391,15401,
1550 15413,15427,15439,15443,15451,15461,15467,15473,15493,15497,
1551 15511,15527,15541,15551,15559,15569,15581,15583,15601,15607,
1552 15619,15629,15641,15643,15647,15649,15661,15667,15671,15679,
1553 15683,15727,15731,15733,15737,15739,15749,15761,15767,15773,
1554 15787,15791,15797,15803,15809,15817,15823,15859,15877,15881,
1555 15887,15889,15901,15907,15913,15919,15923,15937,15959,15971,
1556 15973,15991,16001,16007,16033,16057,16061,16063,16067,16069,
1557 16073,16087,16091,16097,16103,16111,16127,16139,16141,16183,
1558 16187,16189,16193,16217,16223,16229,16231,16249,16253,16267,
1559 16273,16301,16319,16333,16339,16349,16361,16363,16369,16381,
1560 16411,16417,16421,16427,16433,16447,16451,16453,16477,16481,
1561 16487,16493,16519,16529,16547,16553,16561,16567,16573,16603,
1562 16607,16619,16631,16633,16649,16651,16657,16661,16673,16691,
1563 16693,16699,16703,16729,16741,16747,16759,16763,16787,16811,
1564 16823,16829,16831,16843,16871,16879,16883,16889,16901,16903,
1565 16921,16927,16931,16937,16943,16963,16979,16981,16987,16993,
1566 17011,17021,17027,17029,17033,17041,17047,17053,17077,17093,
1567 17099,17107,17117,17123,17137,17159,17167,17183,17189,17191,
1568 17203,17207,17209,17231,17239,17257,17291,17293,17299,17317,
1569 17321,17327,17333,17341,17351,17359,17377,17383,17387,17389,
1570 17393,17401,17417,17419,17431,17443,17449,17467,17471,17477,
1571 17483,17489,17491,17497,17509,17519,17539,17551,17569,17573,
1572 17579,17581,17597,17599,17609,17623,17627,17657,17659,17669,
1573 17681,17683,17707,17713,17729,17737,17747,17749,17761,17783,
1574 17789,17791,17807,17827,17837,17839,17851,17863
1575 }, x);

```